

Apuntes de Ingeniería de Software

Juan Pablo Sandoval y Jaime Navon

Derechos de Autor 2015–2019 por Juan Pablo Sandoval y Jaime Navon.

Los contenidos de este libro están protegidos bajo la licencia Creative Commons Atribución-CompartirIgual 3.0 Unported.

Usted tiene el derecho de:

- **Compartir:** copiar, distribuir y transmitir la obra.
- **Remezclar:** adaptar la obra.

Bajo las siguientes condiciones:

Atribución. Debe dar crédito al autor de la manera especificada por el autor o el licenciante (pero no de manera que sugiera que le respaldan a usted o al uso que hace de la obra).

Compartir Igual. Si altera, transforma o crea a partir de esta obra, solo puede distribuir la obra resultante bajo una licencia idéntica, similar o compatible.

Para cualquier reutilización o distribución, debe dejar en claro a otros los términos de esta licencia. La mejor manera de hacerlo es mediante un enlace a esta página web: <http://creativecommons.org/licenses/by-sa/3.0/>

Cualquiera de las condiciones anteriores puede ser renunciada si obtiene permiso del titular de los derechos de autor. Nada en esta licencia afecta o restringe los derechos morales del autor.



Sus derechos de uso justo y otros derechos no se ven afectados de ninguna manera por lo anterior. Este es un resumen legible por humanos del Código Legal (la licencia completa):

<http://creativecommons.org/licenses/by-sa/3.0/legalcode>

Contents

Illustrations	ii
Agradecimientos	1
Fundamentos en Ingeniería de Software	2
1 Introducción	3
1.1 ¿Qué es la Ingeniería de Software?	3
1.2 Importancia de la Ingeniería de Software	3
1.3 Principios Fundamentales	4
1.4 Estructura del Libro	4
2 Procesos de Desarrollo de Software	5
2.1 La idea de Proceso	5
2.2 El Primer Modelo de Proceso: Cascada	7
2.3 Procesos Iterativos	8
2.4 Procesos Iterativos Incrementales	12
2.5 Ejercicios	14
3 Scrum: Una Pequeña Introducción	16
3.1 Introducción	16
3.2 Solo 3 roles	17
3.3 El Importantísimo <i>Product Owner</i>	18
3.4 El <i>Scrum Master</i> no es el Jefe de Proyecto	19
3.5 Actividades y Artefactos	19
3.6 El <i>Sprint</i>	21
3.7 Scrum y Principios Ágiles	21
3.8 Ejercicios	22
4 Relatos de Usuario	24
4.1 Antecedentes	24
4.2 Relatos de Usuario	25
4.3 Estructura de un relato de usuario	27
4.4 Nivel de detalle	28
4.5 Directrices para escribir buenos relatos	29
4.6 Épicas, Temas y Relatos	29
4.7 Ejercicios	30

5	Planificación en Scrum	31
5.1	Introducción	31
5.2	Planeación de la Iteración (Sprint)	32
5.3	Desglose Táctico en Tareas	34
5.4	Planificación a Nivel de <i>Release</i>	35
5.5	Estrategias de Ajuste en la Planificación	35
6	Equipos de Desarrollo de Software	37
6.1	Las fases de maduración de un Equipo	38
6.2	Modelos de Equipo según Tarea Predominante	39
6.3	Modelo de Equipo Atlético vs Modelo de Grupo de Teatro	40
6.4	El modelo de programador jefe (<i>Chief programming team</i>)	40
6.5	Los equipos deben ser pequeños	41
6.6	Topología de Equipos de Desarrollo	42
6.7	Modos de Colaboración	46
7	Estimaciones	49
7.1	Lo que deseamos estimar	49
7.2	Las Magnitudes y sus Métricas	49
7.3	Del tamaño a la Duración y al Esfuerzo	50
7.4	Evitar la respuesta improvisada	51
7.5	Problemas de Subestimar y Sobreestimar	53
7.6	El Cono de la Incertidumbre	54
7.7	Fuentes de Error en las Estimaciones	55
7.8	Precisión, exactitud y uso de las métricas correctas	55
7.9	Todo parte con el tamaño	56
7.10	Del tamaño al esfuerzo	60
8	UML El Lenguaje de Modelación Unificado	64
8.1	Diseño en el contexto de procesos ágiles	64
8.2	Modelos de Dominio	64
8.3	Cómo encontrar las clases (objetos) de dominio	65
8.4	Clase, responsabilidad, colaborador (CRC)	66
8.5	Del análisis al diseño	67
8.6	Acoplamiento y Cohesión	67
8.7	Un lenguaje visual para el software	68
8.8	El diagrama de clases	70
8.9	Visibilidad	70
8.10	Asociaciones	72
8.11	Interfaces	74
8.12	El diagrama de secuencia	75
9	Patrones de Diseño	77
9.1	¿Por qué vale la pena estudiar estos patrones?	79
9.2	El Patrón Observador (<i>Observer</i>)	79
9.3	El Patrón Método Plantilla (<i>Template Method</i>)	82

9.4	El Patrón Estrategia (<i>Strategy</i>)	86
9.5	El Patrón Decorador (<i>Decorator</i>)	88
9.6	El Patrón <i>Composite</i>	91
9.7	Para aprender más sobre patrones de diseño	96
Apendice - Ruby on Rails		97
10	Ruby: una revisión rápida	98
10.1	Comentarios	98
10.2	Aspectos Básicos	99
10.3	Variables	100
10.4	Funciones	100
10.5	Clases y Objetos	101
10.6	Herencia	103
10.7	Arreglos y Hashes	106
10.8	Estructuras de Control	107
10.9	Convenciones	108
10.10	Material Adicional	109
10.11	Ejercicios	109
11	RoR: Active Records	110
11.1	Maqueo Objeto Relacional	110
12	RoR: Mi primera API	113
12.1	Protocolo de Transferencia de HiperTexto	113
12.2	Postman	115
12.3	Hello World API	117
12.4	API – Estudiantes	119
12.5	Ejercicios	124

Illustrations

2-1	Ilustración: Proceso de Construcción de un Automovil	6
2-2	Ilustración: Modelo en Cascada	7
2-3	Ilustración: Modelo en Espiral	9
2-4	Ilustración: Proceso Unificado	11
2-5	Riesgo a lo largo del desarrollo iterativo. RUP versus Cascada	11
3-1	Ilustración: Scrum	17
4-1	Resultado del requerimiento: Escribe 'hasta pronto alicia' en morado y coloa estrellas al redor	26

Agradecimientos

Fundamentos en Ingenieria de Software

Introducción

La Ingeniería de Software es una disciplina esencial en la era digital en la que vivimos. En un mundo donde la tecnología está en constante evolución y donde las aplicaciones digitales impactan prácticamente todos los aspectos de nuestras vidas, comprender y dominar los principios de la Ingeniería de Software es crucial.

1.1 ¿Qué es la Ingeniería de Software?

La Ingeniería de Software se define como la aplicación de principios de ingeniería para el diseño, desarrollo, mantenimiento y gestión de software de alta calidad. Aunque puede parecer abstracto, podemos pensar en el software como las instrucciones digitales que permiten a las computadoras realizar tareas específicas. La Ingeniería de Software nos brinda las herramientas y técnicas para crear y mantener estas instrucciones de manera sistemática y eficiente.

1.2 Importancia de la Ingeniería de Software

La importancia de la Ingeniería de Software radica en su capacidad para abordar los desafíos complejos asociados con el desarrollo de software a gran escala. A medida que las aplicaciones se vuelven más grandes y sofisticadas, es fundamental tener un enfoque estructurado para garantizar que el software sea confiable, seguro y cumpla con los requisitos del usuario.

1.3 Principios Fundamentales

La Ingeniería de Software se basa en varios principios fundamentales que guían el proceso de desarrollo. Estos principios incluyen:

- **Calidad:** El software debe cumplir con altos estándares de calidad para garantizar su funcionalidad, confiabilidad y seguridad.
- **Mantenibilidad:** El software debe poder modificarse y adaptarse fácilmente a medida que cambian los requisitos y las tecnologías.
- **Eficiencia:** Los recursos, como el tiempo y el esfuerzo, deben utilizarse de manera eficiente durante todo el ciclo de vida del software.
- **Colaboración:** El desarrollo de software a menudo es un esfuerzo colaborativo que requiere comunicación y trabajo en equipo efectivos.

1.4 Estructura del Libro

Este libro está diseñado para brindar una introducción completa a los conceptos básicos de la Ingeniería de Software. Cada capítulo abordará un tema específico, desde la planificación y el análisis de requisitos hasta el diseño, la implementación y las pruebas. También exploraremos metodologías de desarrollo, como el enfoque en cascada y las metodologías ágiles, y analizaremos cómo gestionar proyectos de software de manera efectiva.

La Ingeniería de Software es una disciplina en constante evolución que desempeña un papel fundamental en la creación de soluciones tecnológicas innovadoras. A medida que te sumerjas en los capítulos siguientes, obtendrás una base sólida en los conceptos esenciales que te permitirán abordar proyectos de desarrollo de software con confianza y comprensión.

Procesos de Desarrollo de Software

2.1 La idea de Proceso

Imagina que deseas preparar una deliciosa torta. No basta con tener todos los ingredientes requeridos, sino que también necesitas seguir un procedimiento detallado para obtener el resultado deseado. Desde mezclar la masa hasta hornearla adecuadamente y agregar la decoración final, cada paso tiene su propia secuencia y herramientas específicas. El proceso debe ser seguido con precisión para garantizar que la torta resultante sea consistente en sabor y apariencia.

Este mismo principio se aplica en la industria, donde la construcción de casi cualquier objeto, ya sea un automóvil completo o incluso un simple neumático (puedes observar este proceso en este video), sigue un proceso meticuloso y organizado.

Por lo tanto, es lógico deducir que el desarrollo de software, siendo un producto potencialmente más complejo que un automóvil, también debe basarse en un proceso bien definido. De hecho, a lo largo de los años, se han identificado y delineado las principales etapas involucradas en este proceso:

- **Estudio de Factibilidad:** Determinar si el proyecto es viable en términos económicos, riesgos y necesidades.
- **Ingeniería de Requisitos:** Especificación exhaustiva de los requisitos del software.
- **Diseño de la Interfaz de Usuario:** Creación de la experiencia visual y funcional para los usuarios.



Figure 2-1 Ilustración: Proceso de Construcción de un Automovil

- **Diseño de la Arquitectura:** Definir la estructura y componentes del software.
- **Diseño Detallado (Módulos):** Desglose detallado de componentes individuales.
- **Programación (Codificación):** Escribir el código según las especificaciones.
- **Integración (Construcción):** Ensamblar y combinar los módulos en un sistema completo.
- **Verificación (Pruebas Unitarias y de Sistema):** Evaluar cada componente y el sistema en su conjunto.
- **Validación (Pruebas de Aceptación):** Confirmar que el software cumple con los requisitos.
- **Puesta en Producción:** Implementar el software en el entorno de uso real.
- **Mantenimiento:** Realizar actualizaciones, correcciones y mejoras.
- **Documentación:** Crear manuales y documentación técnica.
- **Gestión del Proyecto:** Supervisar y coordinar todas las etapas del proceso.

A pesar de esta estructura, durante mucho tiempo se ha desarrollado software sin seguir un proceso definido. A menudo denominado “hacking”, este enfoque solía requerir individuos altamente talentosos para lograr resultados exitosos. Sin embargo, su mayor desventaja radica en su falta de repetibilidad; repetir el mismo proceso podría no generar los mismos resultados exitosos.

En las siguientes secciones, nos sumergiremos en ejemplos específicos de procesos de desarrollo de software. Analizaremos detenidamente las ventajas y desventajas de cada enfoque, lo que te permitirá comprender mejor



Figure 2-2 Ilustración: Modelo en Cascada

cómo elegir el proceso más adecuado según las necesidades del proyecto y los objetivos comerciales.

2.2 El Primer Modelo de Proceso: Cascada

El Modelo en Cascada, también conocido como el enfoque secuencial lineal, se compone de una serie de etapas interdependientes (cada etapa depende de la anterior) que guían el desarrollo de proyectos. Estas etapas, que incluyen la definición de requisitos, diseño, implementación, pruebas y mantenimiento, se despliegan de manera secuencial, donde cada etapa sienta las bases para las siguientes. A lo largo de más de tres décadas, este modelo ha sido ampliamente utilizado y, sorprendentemente, continúa aplicándose en algunas ocasiones particulares a pesar de sus notables limitaciones.

¿Por qué goza de tanta popularidad?

- Su simplicidad conceptual y facilidad de comprensión.
- Analogía con procesos de manufactura o construcción.
- Claridad en cada fase con entradas y salidas definidas.
- Facilita la asignación de recursos.
- Permite el seguimiento del progreso y la estimación de lo que falta.

¿Por qué abandonar este enfoque?

- Su eficacia es cuestionable en la práctica, excepto para proyectos muy breves.
- Los requisitos iniciales no siempre son completamente conocidos.
- Incluso si se conocen, pueden cambiar a lo largo del proyecto.

- Existe un alto riesgo hasta las etapas avanzadas del proyecto.

2.3 Procesos Iterativos

Las actividades de desarrollo avanzan en forma de una secuencia de iteraciones, a través de las cuales nos aproximamos gradualmente a nuestro objetivo. Cada iteración no siempre resulta en la generación de nuevo código o en un incremento visible y útil. En este capítulo veremos tres ejemplos de modelos de procesos iterativos comprenden:

- El Modelo Espiral.
- El Modelo de Prototipos.
- El Modelo Unificado (inicialmente denominado RUP).

Modelo en Espiral

El modelo en espiral puede considerarse una evolución del modelo en cascada. Se caracteriza por su capacidad de operar en ciclos y fases iterativas. Una de sus contribuciones más destacadas es haber sido el primero en introducir consideraciones de riesgo de manera explícita. De acuerdo con este enfoque, cada iteración se compone de cuatro fases fundamentales: análisis de requisitos, construcción, evaluación y análisis de riesgo, y planificación de la siguiente iteración.

La espiral se divide en cuatro cuadrantes:

- **Determinación de objetivos:** En este cuadrante, se recopilan los requisitos provenientes de los clientes.
- **Identificación y resolución de riesgos:** Aquí se evalúan todas las posibles soluciones para seleccionar la más adecuada.
- **Desarrollo y pruebas:** Durante el tercer cuadrante, se desarrollan las funcionalidades identificadas y se verifican a través de pruebas exhaustivas. Al final de esta fase, se busca liberar una nueva versión del software.
- **Planificación para la siguiente iteración:** En este cuadrante, los clientes evalúan el progreso realizado en la nueva versión del software.

Algunas ventajas del modelo en espiral son:

- Manejo de riesgo.
- Flexibilidad en los requerimientos. Los cambios de requerimientos pueden ser presentados e incorporados.
- Involucra retroalimentación del cliente.
- Desarrollo en partes donde el riesgo se puede manejar de forma separada.

Algunas desventajas:

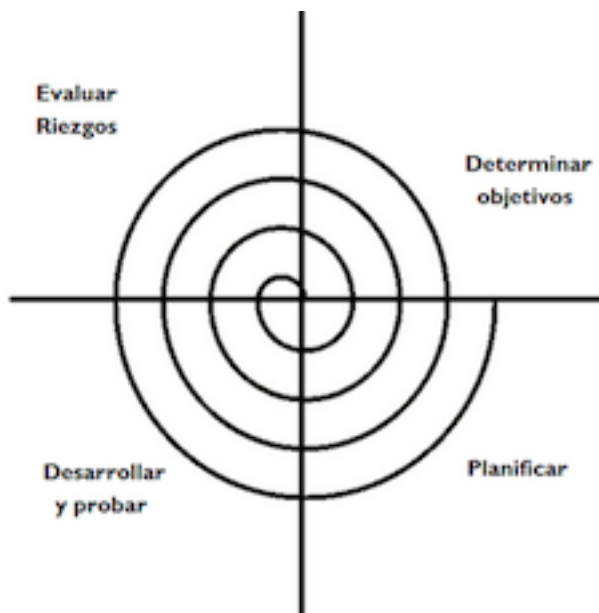


Figure 2-3 Ilustración: Modelo en Espiral

- Depende mucho del análisis de riesgo.
- Dificultades en el manejo del tiempo. El numero de fases es desconocido, por lo que es difícil estimar el tiempo de finalización del proyecto.
- Requiere una inversión importante en la planeación, análisis de riesgo y evaluaciones.

Modelo basado en Prototipos

En un proceso basado en prototipos, la idea central es la construcción física de un producto que “luce” similar al objetivo pero que en realidad está bastante distante de serlo. La idea es que el usuario pueda tener una mirada concreta del producto en forma temprana y resolver el dilema de “el usuario solo sabrá lo que quería cuando le muestres el resultado”.

Hay dos formas de trabajar con prototipos: prototipos desechables y prototipos evolutivos. En el primer caso, el prototipo solo es usado para asegurar que el usuario valide lo que se está construyendo y el prototipo se desecha comenzando el desarrollo desde cero. En el segundo caso, el mismo prototipo evoluciona hasta convertirse en el producto final (discutir ventajas y desventajas de uno y otro enfoque)

Algunas ventajas de desarrollar con prototipos son:

- permite enfrentar falta de claridad en requerimientos
- menor tiempo de desarrollo porque no hay grandes sorpresas al final

- involucramiento del usuario es más fácil de lograr
- menor frustración y ansiedad durante el desarrollo
- permite introducir nuevo sistema gradualmente
- facilita entrenamiento de usuarios
- aumento de satisfacción del usuario

Pero hay también desventajas:

- puede generar expectativas de que el *software* está “casi listo”
- producto final no queda igual que prototipo que se desecha
- usuario puede no estar nunca satisfecho (proyecto no tiene fin)
- análisis incompleto de requisitos (superficialmente entendido)
- omisión de requerimientos no funcionales
- difícil de planificar (cuantos prototipos, cuanto tiempo)
- no hay entregables claros
- poco apropiado para sistemas embebidos, sistemas de tiempo real, *software* científico, etc.

Proceso de Desarrollo Unificado

El Proceso de Desarrollo Unificado, anteriormente conocido como RUP (*Rational Unified Process*), se fundamenta en un enfoque que comprende cuatro fases distintivas: inicio, elaboración, construcción y transición. En cada una de estas fases, se ejecutan diversas actividades que abarcan desde la definición de requisitos hasta el diseño, la programación y otros procesos afines. Cada fase, no obstante, se caracteriza por la predominancia de ciertas actividades específicas. Por ejemplo, en la etapa de inicio, el desarrollo o la programación son limitados.

Estas fases se estructuran en base a objetivos fundamentales:

- **Inception** (Inicio) - Establecimiento del caso y la viabilidad del sistema propuesto.
- **Elaboration** (Elaboración) - Verificación de la capacidad para construir el sistema dentro de las restricciones existentes.
- **Construction** (Construcción) - Edificación de un sistema que funcione de manera exitosa (versión beta).
- **Transition** (Transición) - Entrega de un sistema completamente funcional a los usuarios.

Cada fase puede atravesar múltiples iteraciones hasta alcanzar los objetivos mencionados anteriormente. La figura siguiente proporciona una representación visual de esta idea:

El Modelo Unificado proporciona una definición exhaustiva de múltiples *workflows*, en los cuales se establecen roles sumamente específicos, artefactos de entrada, artefactos de salida y otros componentes clave. Este enfoque

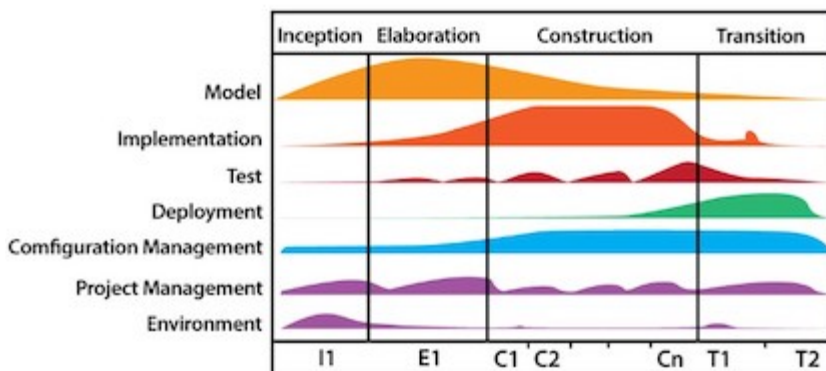


Figure 2-4 Ilustración: Proceso Unificado

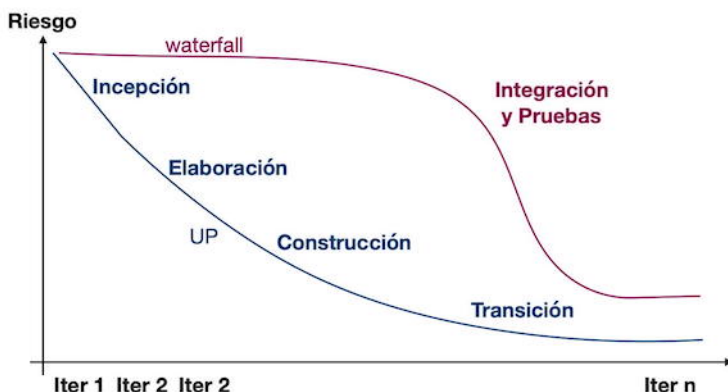


Figure 2-5 Riesgo a lo largo del desarrollo iterativo. RUP versus Cascada

marca uno de los primeros intentos por crear un proceso de desarrollo iterativo de manera sumamente completa.

Uno de los aspectos en los que este proceso demuestra una gran eficacia es en la mitigación temprana del riesgo. Al priorizar el desarrollo de los casos de uso más arriesgados durante las primeras iteraciones, es posible reducir la exposición al riesgo sin necesidad de esperar hasta que el proyecto alcance etapas avanzadas (en contraposición al enfoque en Cascada).

Este modelo enfrenta ciertos desafíos. Por un lado, su complejidad puede resultar abrumadora para equipos de desarrollo reducidos, ya que una sola persona podría tener que asumir múltiples roles. Por otro lado, a pesar de ser un proceso iterativo, su enfoque no siempre es incremental, lo que limita su adaptabilidad a las metodologías ágiles, que son las más ampliamente reconocidas y utilizadas en la actualidad.

2.4 Procesos Iterativos Incrementales

En un proceso iterativo incremental, cada iteración genera un aumento de valor tangible para el cliente, presentando funcionalidades que están listas para ser utilizadas. El producto se considera completo con la entrega del último de estos incrementos. Estos incrementos, en general, se relacionan con la implementación de uno o más nuevos relatos de usuario (detallados más adelante).

Procesos ágiles

Los modelos de proceso caracterizados por su enfoque iterativo incremental suelen recibir la denominación de “ágiles”. El origen de estos procesos ágiles se encuentra en el Manifiesto Ágil, un documento concebido por un grupo destacado de 17 desarrolladores que respondieron a los problemas evidentes en los procesos tradicionales, como el modelo en cascada. Este manifiesto se basa en cuatro valores fundamentales:

- Priorizamos individuos e interacciones sobre procesos y herramientas.
- Valoramos el software funcionando más que la documentación exhaustiva.
- Fomentamos la colaboración con el cliente en lugar de la negociación contractual.
- Nos adaptamos a los cambios en vez de aferrarnos a un plan rígido.

Además, se presenta una serie de 12 principios que profundizan en diversos aspectos:

- **Satisfacción del cliente:** Priorizamos la satisfacción del cliente a través de la entrega temprana y continua de software valioso.
- **Cambios en los requisitos:** Apreciamos los cambios en los requisitos, incluso en etapas avanzadas, y utilizamos estos cambios para entregar un producto competitivo.
- **Entrega frecuente:** Entregamos frecuentemente software funcionando, con preferencia por intervalos más cortos.
- **Colaboración con el cliente:** Colaboramos estrechamente con los clientes y usuarios a lo largo del proceso de desarrollo.
- **Motivación de individuos:** Generamos un entorno en el que los individuos motivados pueden dar lo mejor de sí.
- **Comunicación cara a cara:** La comunicación directa y en persona es esencial para un entendimiento efectivo.
- **Software funcionando:** El software funcionando es la medida principal de progreso.
- **Desarrollo sostenible:** Se enfoca en un ritmo sostenible de trabajo, promoviendo la agilidad a largo plazo.
- **Excelencia técnica:** La atención constante a la excelencia técnica y al buen diseño mejora la agilidad.

- **Simplicidad:** Se valora la simplicidad y se busca maximizar la cantidad de trabajo no realizado.
- **Equipos autoorganizados:** Los equipos tienen la autoridad para tomar decisiones y adaptarse a las circunstancias cambiantes.
- **Reflexión y ajuste continuo:** A intervalos regulares, el equipo reflexiona sobre su desempeño y ajusta su comportamiento en consecuencia.

Extreme Programming

Uno de los primeros modelos de desarrollo ágil, *Extreme Programming* (XP), no es tanto un modelo de proceso en sí, sino más bien una colección de buenas prácticas alineadas con los valores y principios del Manifiesto Ágil. Entre sus pilares se encuentran:

- Mantener al cliente como parte integral del equipo de desarrollo.
- Formular casos de uso concisos (tres frases).
- Que el cliente especifique los casos de prueba.
- Buscar la simplicidad máxima en el software.
- Fomentar la programación en pares.
- Fomentar la propiedad compartida del código.
- Mantener pruebas continuas.

La programación en pares implica que dos desarrolladores trabajen simultáneamente en el mismo código, compartiendo un solo teclado y ratón. Uno actúa como conductor y el otro como revisor. Esta práctica ha demostrado ser muy eficiente. Ver video.

Entre sus ventajas se incluyen:

- La revisión de código por pares detecta errores tempranamente.
- Al menos dos personas comprenden profundamente el código.
- El código tiende a ser más legible y de mejor calidad.
- Facilita la aplicación de estándares de codificación.

Modelos Ágiles Destacados

Dos de los modelos ágiles más populares son Scrum y Kanban. Scrum ha alcanzado tal popularidad que, en muchos casos, se ha convertido en sinónimo de metodología ágil.

En resumen, Scrum opera bajo un modelo iterativo incremental, donde en cada iteración de duración fija (generalmente 2 o 3 semanas), el equipo de desarrollo elige un nuevo incremento basado en prioridades establecidas por el representante del cliente o usuario (*product owner*). El equipo tiene amplia libertad para gestionar la iteración y, al final, se llevan a cabo dos reuniones fundamentales: revisión del producto y retrospectiva del proceso.

Por otro lado, Kanban no emplea iteraciones de longitud fija. Su enfoque radica en optimizar el flujo de tareas desde la fase de “por hacer” hasta “hecho”.

Esto se logra mediante una visualización efectiva del progreso y la limitación del trabajo en curso.

Estos dos modelos de proceso serán analizados con mayor profundidad en las siguientes secciones.

La Esencia de la Agilidad y la Agilidad Moderna

Alistair Cockburn, uno de los autores del Manifiesto Ágil, ha expresado su decepción ante la evolución que ha experimentado la sencilla idea de agilidad. Según Cockburn, se ha complicado en exceso una idea que, en su origen, era muy simple: colaborar, entregar, reflexionar, mejorar. Su propuesta se resume en el concepto del corazón de la agilidad.

Siguiendo una línea similar, Joshua Kerievsky ha simplificado la agilidad en su propuesta de agilidad moderna, que se basa en solo cuatro prácticas:

- Potenciar al equipo para alcanzar su máximo potencial.
- Entregar valor de manera continua.
- Establecer la seguridad como requisito previo.
- Experimentar y aprender de manera ágil.

2.5 Ejercicios

Preguntas Cerradas:

- El Modelo en Cascada es un proceso iterativo e incremental. (Falso/Verdadero)
- El desarrollo de software puede llevarse a cabo siguiendo un proceso caótico sin ninguna estructura definida. (Falso/Verdadero)
- El Modelo en Espiral fue el primero en introducir consideraciones de riesgo de manera explícita. (Falso/Verdadero)
- En el Modelo en Espiral, cada iteración consta de cuántas fases fundamentales?
- El Proceso de Desarrollo Unificado (RUP) se divide en cuántas fases distintas?
- El desarrollo de prototipos evolutivos implica desechar el prototipo original y comenzar desde cero. (Falso/Verdadero)
- Extreme Programming (XP) es un modelo de proceso altamente estructurado y rígido. (Falso/Verdadero)
- En Kanban, las iteraciones tienen una duración fija y predefinida. (Falso/Verdadero)
- En Scrum, el product owner establece las prioridades para los nuevos incrementos. (Falso/Verdadero)
- El Manifiesto Ágil se basa en valores y principios que promueven la documentación exhaustiva como prioridad. (Falso/Verdadero)

Preguntas Abiertas:

- ¿Cuál es la importancia de contar con un proceso bien definido en el desarrollo de software, similar al proceso de preparar una torta?
- ¿Qué ventajas y desventajas ves en la programación en pares como se presenta en Extreme Programming?
- ¿Cuál crees que es la principal razón por la que el Modelo en Cascada sigue siendo utilizado en ciertos contextos a pesar de sus limitaciones?
- ¿Por qué crees que el enfoque de desarrollo basado en prototipos puede ser especialmente adecuado para proyectos con requisitos poco claros?
- ¿Cómo consideras que los valores y principios del Manifiesto Ágil se alinean con la filosofía de desarrollo de software?



Scrum: Una Pequeña Introducción

3.1 Introducción

Scrum, sin lugar a dudas, es el paradigma más destacado entre los modelos de procesos ágiles. Desarrollado a fines de los años 90 por Ken Schwaber, se destaca por su enfoque iterativo e incremental que promueve la flexibilidad y la colaboración en el desarrollo de software.

La esencia de Scrum puede ser comprendida a través de los siguientes puntos clave:

- El desarrollo de software se lleva a cabo en ciclos repetitivos llamados “sprints”, que representan períodos de tiempo específicos.
- Cada sprint agrega un incremento de funcionalidad al producto y genera un resultado “potencialmente entregable”, es decir, un incremento de software funcional.
- El contenido del sprint se define en gran medida mediante una negociación activa con el representante del cliente, conocido como “product owner”.
- Al concluir cada sprint, se realizan dos reuniones esenciales antes de embarcarse en el siguiente ciclo:
- La reunión de “review”, que implica evaluar y presentar lo que se ha logrado durante el sprint, así como identificar lo que aún debe abordarse.
- La “retrospectiva”, una sesión reflexiva en la que el equipo analiza y discute cómo funcionó el proceso durante el sprint anterior y cómo se pueden realizar mejoras.

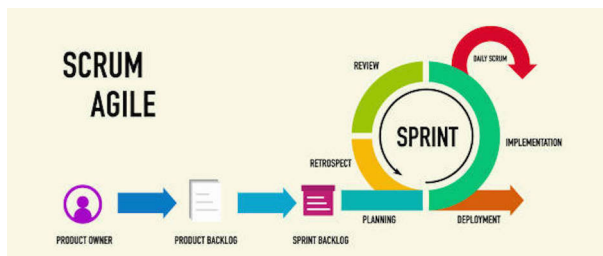


Figure 3-1 Ilustración: Scrum

Scrum se caracteriza por su estructura organizativa y sus roles bien definidos, que incluyen al “product owner”, el equipo de desarrollo y el “Scrum Master”, quien actúa como facilitador y defensor del proceso. A través de estos roles y prácticas, Scrum fomenta la transparencia, la adaptabilidad y la colaboración continua, permitiendo que los equipos entreguen resultados de alta calidad en ciclos cortos y controlados.

A pesar de sus numerosas ventajas, implementar Scrum de manera efectiva requiere una comprensión sólida de sus principios y prácticas, así como una comunicación abierta y una colaboración constante entre los miembros del equipo y los stakeholders. En última instancia, Scrum ofrece un marco poderoso para el desarrollo de software ágil, alineado con los valores y principios fundamentales de la metodología ágil.

3.2 Solo 3 roles

En este modelo de proceso solo existen 3 roles distintos:

- **Product Owner.** Este rol asume la responsabilidad total del producto, lo que incluye su éxito o fracaso en el proyecto. El Product Owner es el encargado de tomar decisiones clave sobre el producto y trabaja en estrecha colaboración con los stakeholders para identificar y priorizar las características esenciales que se incorporarán. Su enfoque se centra en garantizar que el equipo esté construyendo el producto adecuado y que las necesidades del cliente se satisfagan plenamente.
- **Scrum Master.** El Scrum Master desempeña un papel multifacético como coach y líder del proceso. Su función principal es facilitar y guiar al equipo en la implementación efectiva de Scrum. Actúa como un facilitador que remueve obstáculos y promueve un ambiente propicio para la colaboración y la mejora continua. Además, el Scrum Master asume la responsabilidad de resolver problemas y asegurar que el equipo siga las prácticas y los principios de Scrum de manera adecuada.
- **Team.** El núcleo del proceso recae en el equipo de desarrollo, el cual está compuesto por individuos altamente colaborativos y con habili-

dades diversas. Los equipos en Scrum son autónomos y autogestionados, y típicamente constan de 5 a 8 personas. Estos equipos cross-funcionales tienen la capacidad de planificar, diseñar, implementar y probar las características del producto. Trabajan de manera colaborativa, promoviendo la comunicación abierta y la responsabilidad compartida para alcanzar los objetivos del sprint.

3.3 El Importantísimo *Product Owner*

El Product Owner generalmente desempeña un papel clave en el proyecto, siendo un líder de usuarios que posee un profundo conocimiento sobre los usuarios, el mercado y las tendencias relevantes para el dominio en cuestión. Este rol requiere disponibilidad, experiencia en el negocio y un compromiso sólido con el proyecto.

La figura del Product Owner debe tener una visión clara de los objetivos del proyecto y ser capaz de comunicar eficazmente esa visión al equipo. Su responsabilidad principal radica en la priorización del backlog del producto, aunque no decide la cantidad de trabajo que se aborda en cada sprint.

El Product Owner asume la responsabilidad final del proyecto, desempeñando diversos roles clave. Como comunicador, el Product Owner informa a los interesados sobre el progreso del proyecto y se involucra en negociaciones con los stakeholders, colaborando con ellos para definir y establecer el valor que será entregado. Además, fomenta la creación de conocimiento compartido y se mantiene accesible para el equipo en todo momento. También desempeña un papel activo en los eventos del equipo, participando activamente en ellos.

En su función como planificador, el Product Owner diseña y planifica los sprints, buscando maximizar el valor de los releases. Asimismo, establece prioridades en el backlog y toma decisiones fundamentales para el avance y el éxito del proyecto.

Como visionario, el Product Owner posee una visión clara y precisa de los requisitos y objetivos del proyecto. Promueve esta visión ante la organización y se esfuerza por definir metas claras y alcanzables que guíen el desarrollo del software.

En su rol de experto, el Product Owner demuestra un profundo conocimiento del ámbito del proyecto. Goza de respeto y reconocimiento en toda la organización, desempeñando también el papel de representante del negocio en el proceso de desarrollo.

Finalmente, como miembro valioso del equipo (Team player), el Product Owner motiva al equipo estableciendo metas concretas y comprensibles. Además, suministra al equipo la información necesaria para que pueda en-

tregar valor de manera efectiva y eficiente. Su colaboración y participación son fundamentales para el éxito general del proyecto.

3.4 El Scrum Master no es el Jefe de Proyecto

El Scrum Master tiene la responsabilidad primordial de respaldar y fomentar el proceso Scrum en todas sus dimensiones: teoría, prácticas, reglas y valores. Su alcance de acción no se limita solo al equipo de desarrollo, sino que también se extiende al Product Owner y a la organización en su conjunto.

Sus funciones clave abarcan:

- Ofrecer apoyo al equipo de desarrollo mediante el ejercicio de coaching en la autorganización y la multifuncionalidad. Además, se encarga de eliminar obstáculos que puedan dificultar el avance del equipo y brindar orientación para la creación de productos de alto valor.
- Brindar soporte al Product Owner en diversos aspectos, como asegurarse de que todos comprendan las metas establecidas, sugerir técnicas para la gestión eficiente del backlog y garantizar que las prioridades del backlog sean adecuadamente establecidas.
- Contribuir al éxito organizacional al acelerar la adopción de procesos ágiles en la organización. Esto implica planificar y supervisar la implementación del modelo Scrum en la estructura organizativa, así como proporcionar orientación a los empleados y partes interesadas sobre cómo llevar a cabo procesos con Scrum de manera efectiva.

3.5 Actividades y Artefactos

La siguiente gráfica sintetiza todas las actividades y los artefactos involucrados en un proceso Scrum.

A continuación se describen brevemente cada una de las actividades en el marco de Scrum:

- **Sprint:** Es un período de tiempo fijo y corto en el que se lleva a cabo el trabajo de desarrollo. Durante el sprint, se crea un incremento del producto potencialmente entregable que agrega valor al producto en desarrollo.
- **Sprint Planning:** Es una reunión al comienzo de cada sprint donde el equipo de desarrollo y el Product Owner colaboran para definir qué elementos del backlog se trabajarán durante el sprint y cómo se llevará a cabo el trabajo.
- **Daily Scrum:** Una reunión diaria de corta duración en la que el equipo de desarrollo sincroniza sus actividades, revisa el progreso y discute cualquier obstáculo que pueda estar afectando su avance.

- **Sprint Execution:** Durante esta actividad, el equipo de desarrollo trabaja en la implementación de las tareas definidas en el backlog durante el sprint planificado.
- **Sprint Review:** Al final de cada sprint, se lleva a cabo una reunión para demostrar y revisar el incremento del producto desarrollado. Los stakeholders proporcionan retroalimentación y se identifican posibles ajustes para el backlog.
- **Sprint Retrospective:** Otra reunión al final del sprint en la que el equipo de desarrollo reflexiona sobre el proceso, identifica lo que funcionó bien y las áreas de mejora, y ajusta su enfoque para futuros sprints.
- **Add/Take from Product Backlog:** Durante cualquier momento, el Product Owner puede agregar nuevos elementos o ajustar los elementos existentes en el backlog del producto, en función de las necesidades cambiantes del proyecto y la retroalimentación recibida.

Estas actividades en conjunto forman la estructura esencial de Scrum, permitiendo la entrega incremental y continua de valor mientras se adapta y mejora el proceso a lo largo del tiempo.

A continuación una breve descripción de cada uno de los artefactos en Scrum:

- **Product Backlog:** Es una lista priorizada de todas las funcionalidades, requisitos, mejoras y correcciones que se planea desarrollar en el producto a lo largo del tiempo. Se mantiene y actualiza constantemente para reflejar las necesidades cambiantes del proyecto y proporciona una visión general de alto nivel de los elementos a ser trabajados. Esta lista está constantemente siendo revisada y priorizada por el *product owner*.
- **Sprint Backlog:** Después de seleccionar los elementos del Product Backlog que se han comprometido a ser desarrollados durante un sprint específico. Contiene tareas más detalladas y estimadas que el equipo de desarrollo debe completar durante el sprint para cumplir con los objetivos establecidos. Recordemos que una vez iniciado el *sprint* no se pueden agregar ni quitar nuevas tareas.
- **Potentially Shippable Product:** Al final de cada sprint, el equipo de desarrollo entrega un incremento de producto potencialmente entregable. Esto significa que el incremento contiene todas las funcionalidades completas y probadas que podrían ser entregadas al cliente si fuera necesario, aunque la decisión de hacerlo depende del Product Owner.

3.6 El *Sprint*

El *sprint* es una iteración, típicamente de dos a cuatro semanas de duración. Todas las iteraciones en Scrum son del mismo largo. Lo importante es que al final del *sprint* debe haberse creado algo de valor para el cliente y por eso es fundamental el rol de *product owner*.

- **Planeación del *Sprint*.** Es la etapa inicial antes de comenzar la ejecución. Comienza con la selección de relatos o funcionalidades que serán desarrolladas. Esto lo hace el equipo de desarrollo a partir del *backlog* del producto priorizado por el *product owner*. Estas prestaciones son divididas en tareas más pequeñas para poder hacer estimaciones de tiempo y esfuerzo y se asignan a los miembros del equipo disponibles.
- **Ejecución del *Sprint*.** Corresponde a la ejecución de las tareas en el orden que el equipo decida. Nadie le dice al equipo cómo llevar a cabo las tareas y no existen cosas “casi hechas”, está hecha o no está hecha. Siempre está el *Scrum Master* a mano para guías y consultas sobre el proceso.
- **El *Daily Meeting*.** Es una reunión de pié cada día temprano de 15 - 20 minutos. Los miembros del equipo indican que hicieron desde ayer, que pretendo hacer hoy y posibles obstáculos visibles. Puede decidirse que temporalmente una persona apoye a alguien que está teniendo problemas o cambios en los equipos. No es momento de resolver problemas
- **El *Sprint Review*.** Esta reunión es muy importante, porque permite inspeccionar y adaptar el producto a medida que transcurren las iteraciones. La conversación incluye al equipo, a los interesados, a *sponsors*, y a clientes y representa una gran oportunidad para que equipo adquiera *feedback* de la gente relacionada con el negocio. De esta reunión puede surgir la necesidad de repriorizar el *backlog* del producto o incluso agregar o eliminar algunas tareas.
- **El *Sprint Retrospective*.** La segunda reunión del final de *sprint*. A diferencia del *Sprint Review*, esta está orientada a reflexionar y ver formas de mejorar el proceso. Permite revisar en forma crítica el proceso y como funcionó en el *sprint* reciente para ver posibles formas de pulir los ripios, eliminar conflictos y mejorar así el proceso dentro de los principios ágiles. Participan principalmente el *Scrum Master* y los miembros del equipo de desarrollo, pero no es raro que se una el *product owner* en algunos casos.

3.7 Scrum y Principios Ágiles

Scrum responde a los principios ágiles al

- **Abrazar la variabilidad:** En lugar de tratar de eliminar por completo la incertidumbre y la variabilidad, los enfoques ágiles reconocen que el

cambio y la adaptación son inevitables en el desarrollo de software. En lugar de resistir la variabilidad, los equipos ágiles la abrazan y la incorporan en su enfoque, permitiendo una mayor flexibilidad y capacidad de respuesta a medida que surgen nuevos desafíos y requisitos.

- **Desarrollar en forma iterativa e incremental:** En lugar de tratar de completar todo el desarrollo de una vez, los métodos ágiles fomentan la construcción del producto en pequeños incrementos funcionales a lo largo de iteraciones periódicas. Esto permite obtener retroalimentación temprana, ajustar el enfoque según sea necesario y entregar valor de manera continua en lugar de esperar hasta que todo esté terminado.
- **Inspeccionar y adaptar constantemente:** Los equipos ágiles se comprometen a una mejora continua mediante la revisión constante de su trabajo y procesos. Inspeccionan su progreso, identifican áreas de mejora y ajustan su enfoque para optimizar la eficiencia y la calidad del producto.
- **Fomentar la transparencia (todos saben lo que está pasando):** La transparencia es fundamental en los métodos ágiles. Todos los aspectos del trabajo, desde el progreso del proyecto hasta los desafíos y problemas, se comparten de manera abierta y accesible. Esto fomenta la confianza, la colaboración y la toma de decisiones informadas en todos los niveles, lo que contribuye a un ambiente de trabajo más efectivo y a la alineación en los objetivos del proyecto.
- **Reducir las incertezas:** Los enfoques ágiles buscan reducir las incertidumbres y los riesgos al abordar el desarrollo en pasos pequeños y manejables. Al construir de manera incremental y obtener retroalimentación temprana, los equipos pueden abordar y resolver problemas a medida que surgen, en lugar de esperar a que se conviertan en obstáculos insuperables.

3.8 Ejercicios

Preguntas de verdadero o falso:

- Scrum es un modelo de proceso ágil desarrollado por Ken Schwaber a fines de los años 90.
- En Scrum, los sprints son períodos de tiempo fijos y cortos en los que se lleva a cabo el trabajo de desarrollo.
- Durante un sprint, se crea un incremento de funcionalidad potencialmente entregable.
- El product owner es responsable de priorizar el backlog del producto y decide cuánto trabajo se aborda en cada sprint.
- La reunión de sprint review implica evaluar y presentar lo que se ha logrado durante el sprint y definir el valor que falta.

- El Scrum Master es el líder del equipo de desarrollo y tiene la responsabilidad total del producto.
- El Scrum Master elimina obstáculos y promueve un ambiente propicio para la colaboración y la mejora continua.
- El sprint backlog contiene tareas detalladas y estimadas que el equipo debe completar durante el sprint.
- El sprint execution es la fase en la que se planifica qué elementos del backlog se trabajarán durante el sprint.
- En Scrum, el Product Owner es responsable de hacer coaching en autoorganización y multifuncionalidad al equipo de desarrollo.

Preguntas abiertas de reflexión:

- ¿Cómo contribuye Scrum a la flexibilidad y la colaboración en el desarrollo de software?
- ¿Cuál es el papel y la importancia del Product Owner en el proceso Scrum?
- ¿Cómo se diferencia el rol del Scrum Master del rol de un jefe de proyecto tradicional?
- ¿Por qué es esencial la inspección y adaptación constante en Scrum?
- ¿Cómo promueve Scrum la transparencia en todas las etapas del desarrollo?



Relatos de Usuario

4.1 Antecedentes

El proceso de levantamiento de requisitos, también conocido como requerimientos, marca el inicio de un proyecto de desarrollo de software. Su esencia radica en descubrir las verdaderas necesidades que se deben abordar. Aunque esta tarea puede parecer sencilla en teoría, a menudo resulta complicada, ya que aquellos que están interesados en el producto o los futuros usuarios a veces carecen de claridad sobre lo que realmente se requiere, o enfrentan dificultades al transmitir sus necesidades a quienes son responsables de la construcción.

Existen dos categorías principales de requisitos: los funcionales y los no funcionales. Los requisitos funcionales se centran en definir QUÉ capacidades debe poseer el sistema. Por ejemplo, podría ser necesario que el sistema permita verificar la cantidad de productos en inventario. Por otro lado, los requisitos no funcionales imponen restricciones o condiciones adicionales, como rendimiento y usabilidad. Siguiendo el ejemplo de la consulta de inventario, un requisito no funcional podría establecer que la respuesta debe ser generada en menos de un segundo.

En el pasado, en las metodologías de desarrollo tradicionales, la etapa de levantamiento de requisitos culminaba con la creación de un documento de requisitos, que servía como un contrato entre los usuarios y los desarrolladores. Sin embargo, esta aproximación presentaba dos desafíos significativos: (1) Era inviable congelar los requisitos en un momento temprano del proyecto, cuando aún no se había iniciado; (2) Los documentos escritos solían estar plagados de ambigüedades.

Las metodologías ágiles abordan el primer inconveniente permitiendo la adición, eliminación o modificación de requisitos en cada iteración. En cuanto a

la ambigüedad en las especificaciones escritas, se asume que lo plasmado en papel es incompleto y que requerirá ajustes a medida que avance el proceso.

Un cambio fundamental en la forma de abordar el levantamiento de requisitos es la orientación hacia el usuario en lugar de centrarse exclusivamente en el sistema en desarrollo. La premisa clave es capturar cómo el sistema ayudará a cada tipo de usuario a alcanzar sus objetivos y realizar sus tareas de manera efectiva.

En esta línea, dos enfoques principales destacan:

- **Relatos de Usuario (User Stories):** Descripciones breves y sencillas de las características desde la perspectiva del usuario, enfocadas en la entrega de valor y la satisfacción de necesidades concretas.
- **Casos de Uso (Use Cases):** Escenarios detallados que narran las interacciones entre los usuarios y el sistema, ilustrando cómo se logran los objetivos del usuario a través de la aplicación.

Estos cambios en el enfoque y las metodologías permiten abordar los desafíos del levantamiento de requisitos de manera más dinámica y centrada en el usuario, contribuyendo a una comprensión más profunda de las necesidades reales y a una mayor agilidad en el proceso de desarrollo de software.

4.2 Relatos de Usuario

Dos conceptos fundamentales emergen en este contexto: la orientación hacia el usuario en los requisitos y la noción de que lo que consignamos por escrito constituye solo el punto de partida para un diálogo más amplio.

Ya hemos discutido la inherente ambigüedad del lenguaje natural al expresar requerimientos. Ilustrando esto, consideremos el caso de alguien que solicita un pastel de despedida para su amiga Alicia. Lo que el pastelero recibe es la siguiente indicación:

“Por favor, escribe ‘Hasta pronto Alicia’ en morado y coloca estrellas alrededor”. La imagen muestra la creación resultante, que cumple perfectamente con los requerimientos.

El enfoque de los relatos de usuario surge de la observación de que tener un documento compartido no garantiza un entendimiento compartido. El propósito detrás de redactar los relatos radica en cultivar, con el tiempo, una comprensión colectiva de los requerimientos. La segunda premisa subyacente, arraigada en la filosofía ágil, sostiene que el objetivo no es simplemente maximizar la cantidad de código producido (*software*), sino más bien maximizar lo que se conoce como el *outcome* y el impacto.

La propuesta es emplear una “conversación” como medio para capturar los requerimientos del usuario. De hecho, un relato puede considerarse una promesa de diálogo con el usuario.



Figure 4-1 Resultado del requerimiento: Escribe 'hasta pronto alicia' en morado y coloa estrellas al rededor

En sus inicios, se emplearon tarjetas de 3x5 pulgadas (muy populares en los Estados Unidos) para plasmar los relatos.

Es fundamental entender que los relatos de usuario NO CONSTITUYEN un nuevo formato de requerimientos escritos, sino más bien un vehículo para construir un entendimiento colectivo. Representan deliberaciones sobre las soluciones a los problemas que enfrentan nuestros usuarios y clientes, lo que conduce a acuerdos sobre qué construir.

Los cimientos de esta noción se sustentan en los siguientes pilares:

- Los requerimientos deben surgir de una colaboración activa.
- Los requerimientos consignados en documentos concluyen con todas las decisiones tomadas.
- Dada la naturaleza de plazos de entrega breves, es irreal especificar todos los detalles de antemano.
- La responsabilidad de plasmar los requerimientos de manera precisa se distribuye entre los actores involucrados.

Nos enfrentamos, por tanto, no únicamente a una estructura diferente respecto a los convencionales documentos de requerimientos, sino a una filosofía enteramente divergente que armoniza con los valores y principios que definen el movimiento ágil. La figura resalta de manera explícita esta contrastante perspectiva.

4.3 Estructura de un relato de usuario

Una tarjeta para cada relato. En el anverso incluir el título y la descripción. En el reverso las condiciones de satisfacción.

El título debe comenzar con un verbo y debe describir en forma muy clara el objetivo. Por ejemplo: Encontrar Cines donde se esté dando una película.

La descripción tiene una estructura que se repite para cada relato que es la siguiente:

Yo, como _____ necesito _____ para _____

El primer espacio debe ser llenado con un rol específico de entre los diversos roles que existen para los usuarios. Por ejemplo, si una aplicación va a ser usada por alumnos, ayudantes y profesores, allí pondremos “yo como alumno” o “yo como profesor”.

El segundo espacio es la meta del usuario. Qué es lo que realmente necesita que este trozo de funcionalidad realice. Por ejemplo, “yo como profesor necesito revisar las notas de mis alumnos”.

Finalmente, la última parte es la justificación. Es muy importante por dos razones:

- evita agregar funcionalidades que no son necesarias
- ayuda a entender mejor la funcionalidad

Ejemplo 1

- **Título.** Encontrar reseñas de restaurantes cercanos.
- **Descripción.** Yo, como simple usuario de la aplicación necesito revisar reseñas de los restaurantes cercanos a donde me encuentro para seleccionar un lugar adecuado para cenar.
- **Condiciones de aceptación.**
 - Escenario 1:
 - * Dado un usuario que ingresa a la aplicación.
 - * Cuando el usuario comparte su ubicación a la aplicación.
 - * Entonces el sistema debe mostrar los restaurantes más cercanos a su ubicación ordenados por orden de recomendación.

Ejemplo 2

- **Título.** Recuperar Contraseña
- **Descripción.** Como usuario quiero recuperar mi contraseña para que pueda ingresar al sistema en caso de olvidarla.
- **Condiciones de aceptación.**
 - Escenario 1:
 - * Dado un usuario que navega a la página de inicio.

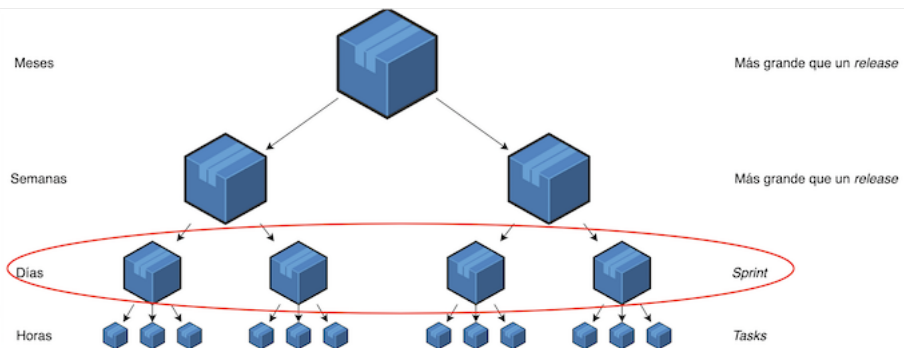
- * Cuando el usuario selecciona la opción “recuperar contraseña” e ingresa un email valido.
- * Entonces el sistema debe enviar un enlace al email ingresado.
- * Cuando el usuario entra al sistema a través del enlace.
- * Entonces el sistema le permite al usuario establecer su nueva contraseña.

Hay múltiples enfoques para redactar las condiciones de aceptación de historias de usuario; en este curso, nos centraremos en la estrategia popularmente denominada ‘Given, When, Then’ (Dado, Cuando, Entonces). En esencia, esta estructura nos capacita para plasmar las condiciones de aceptación de una historia de usuario de manera que una vez concluida la historia, sea posible verificar su funcionalidad, asegurando así que cumple con los resultados esperados a partir de un punto de partida específico.

4.4 Nivel de detalle

Una de las cosas más difíciles para quienes están comenzando a escribir relatos de usuario es encontrar el nivel de detalle que es apropiado para un relato. Por ejemplo, supongamos que mi meta es ir de paseo a San Francisco, eso parece demasiado complejo para un relato, ya que puede descomponerse en varias partes como comprar el ticket de avión, hacer la reserva de hotel, reservar un auto en el aeropuerto, etc. Si tomamos la compra de un ticket de avión puede incluso ser demasiado para un relato, ya que posiblemente tendré que evaluar posibilidades, seleccionar aerolíneas y horarios, etc.

Un criterio útil es considerar un relato cuando puede implementarse en unos pocos días y, por lo tanto, puede ser desarrollado fácilmente en un sprint. Tampoco nos interesa seguir dividiendo hasta llegar a subtarefas elementales porque el número de relatos se elevaría demasiado. Ver figura.



4.5 Directrices para escribir buenos relatos

Pautas para la redacción de relatos efectivos:

- Idealmente, deben ser independientes (permitiendo una reorganización flexible según la prioridad).
- Deben aportar valor **AL CLIENTE**.
- Deben permitir al equipo de desarrollo estimar con facilidad el esfuerzo de implementación.

Prácticas que suelen ser consideradas como “malos hábitos”:

- Evitar la expresión genérica “as a user” (ser más específico en la descripción).
- Evitar la concisión extrema en los relatos.
 - Ejemplo: “Los resultados se deben guardar en formato XML”.
 - Ejemplo: “Los resultados se deben almacenar en formato HTML”.
- Evitar relatos con interdependencias (dificultan la planificación).
- Evitar el *goldplating* (añadir características innecesarias).
 - Ejemplo: Además de las pestañas, permitir el arrastre de elementos.
- Evitar un exceso de detalle (ya que se utilizan tarjetas pequeñas para la descripción).
- Retrasar detalles específicos de la interfaz de usuario en el proceso.
- No transferir requisitos formales a los relatos de usuario.

4.6 Épicas, Temas y Relatos

La idea consiste en considerar los relatos de usuario dentro de un contexto más amplio en lugar de abordarlos de manera fragmentada. Para lograr esto, existen dos enfoques principales: las épicas y los temas.

El concepto de una épica busca describir un objetivo de mayor envergadura, por ejemplo: la compra de un boleto de avión a San Francisco o la inscripción en los cursos del semestre 1'2020 utilizando Banner. Esta meta abarcará una serie de objetivos más pequeños que se traducirán en relatos de usuario.

De esta manera, una épica se refiere a un objetivo principal que debe descomponerse en diversos relatos a ser realizados secuencialmente. Por lo general, los relatos que forman parte de una épica no tienen un valor significativo por sí mismos.

En contraposición a las épicas, un tema puede abarcar cualquier agrupación de relatos (tales como variaciones de un mismo objetivo, entre otros). En este caso, los relatos que conforman un tema poseen valor de manera independiente al tema en sí.

Para simplificarlo, podríamos considerar un tema como una recopilación de relatos relacionados de alguna manera, mientras que una épica se refiere a un relato amplio que puede dividirse en múltiples etapas.

Ejemplo de temas son por ejemplo:

- Todos los relatos que tienen que ver con análisis de datos de los clientes
- Todos los relatos que tiene que ver con acceso a la aplicación
- Todos los relatos que tienen que ver con manejo de productos

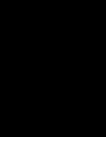
4.7 Ejercicios

Preguntas de falso y verdadero

- El levantamiento de requisitos es el inicio de un proyecto de desarrollo de software.
- Los requisitos funcionales se centran en las restricciones y condiciones adicionales de un sistema.
- Los relatos de usuario y los casos de uso son dos enfoques principales para capturar requerimientos.
- Los relatos de usuario se basan en una conversación constante para comprender los requerimientos.
- Las metodologías ágiles permiten la adición, eliminación o modificación de requisitos en cada iteración.
- Los relatos de usuario buscan maximizar la cantidad de código producido.
- El nivel de detalle apropiado para un relato de usuario es aquel que puede implementarse en un solo sprint.
- Las épicas son objetivos principales que se descomponen en varios relatos secuenciales.
- Los temas son agrupaciones de relatos que tienen valor independiente al tema en sí.
- Los relatos de usuario constituyen un nuevo formato de requerimientos escritos.

Preguntas abiertas de reflexión:

- ¿Por qué es importante orientarse hacia el usuario al definir requisitos de software?
- ¿Qué ventajas ofrecen los relatos de usuario sobre los documentos tradicionales de requerimientos?
- ¿Cómo puede un equipo asegurarse de que los relatos de usuario tengan el nivel de detalle adecuado?
- ¿Qué significa que los relatos de usuario son una “promesa de diálogo con el usuario”?



Planificación en Scrum

5.1 Introducción

La planificación efectiva es esencial para el éxito de cualquier tarea o proyecto, y el desarrollo de software no es una excepción. En Scrum, un marco de trabajo ágil ampliamente utilizado, la planificación es un proceso continuo y adaptativo que abarca varios niveles, desde la iteración hasta el *release* y más allá. En este capítulo, exploraremos cómo se planifica en Scrum, considerando analogías sugerentes y estrategias clave para abordar los desafíos inherentes a la planificación en el desarrollo de software.

Imagina el atraco meticulosamente planeado en la famosa serie de TV “Casa de Papel”. Un grupo liderado por un “profesor” superinteligente ejecuta un robo de gran complejidad, que requirió años de planificación meticulosa. De manera similar, cualquier tarea de envergadura necesita ser minuciosamente planificada para alcanzar el éxito. Este principio se aplica con fuerza al desarrollo de software, donde la incertidumbre y la evolución constante son la norma.

En proyectos de otros ámbitos, las “Cartas Gantt” suelen estructurar la planificación, pero en el desarrollo de software es difícil establecer un plan rígido desde el principio. Aquí es donde entran en juego los modelos de proceso ágiles. Un ejemplo elocuente es el esquí en terreno desconocido: los esquiadores se enfrentan a un terreno cambiante y se adaptan en tiempo real. De manera análoga, los equipos ágiles ajustan su enfoque según las circunstancias cambiantes del proyecto.

5.2 Planeación de la Iteración (Sprint)

La iteración, o *sprint*, es una unidad fundamental de tiempo en Scrum. Durante la planeación del sprint, el equipo selecciona historias de usuario de acuerdo con la prioridad y la capacidad del equipo. Las historias de usuario, expresadas en Puntos de Historia, son estimaciones relativas de complejidad. El equipo equilibra la cantidad de trabajo con la capacidad, asegurando un compromiso realista.

Puntos de Historia

Los Puntos de Historia constituyen una técnica valiosa para evaluar la complejidad relativa de las historias de usuario. Esta estimación se efectúa en una unidad abstracta, lo que facilita la comparación del esfuerzo involucrado en diferentes historias. Una estrategia efectiva para estimar mediante puntos de historia es comenzar seleccionando una historia que la mayoría de los miembros del equipo comprenda en términos de su complejidad de desarrollo. Para esta historia de referencia, el equipo asigna un número específico, conocido como “puntos de historia”, que puede ser cualquier valor. Luego, utilizando esta referencia, el equipo atribuye puntos de historia a las demás historias en función de su complejidad relativa. Por ejemplo, si una historia se considera más desafiante que la historia de referencia, podría asignársele más puntos, mientras que si se la percibe como menos compleja, podría recibir menos puntos.

Estimación con Scrum Poker

La destreza esencial de priorizar historias de usuario se potencia a través de Scrum Poker, una técnica colaborativa que permite al equipo de desarrollo y al Product Owner estimar la complejidad de las historias de manera conjunta. En una sesión de Scrum Poker, cada miembro del equipo elige una carta que representa su percepción de la cantidad de esfuerzo necesario para completar una historia en particular. Luego, se revelan las cartas y se abordan las diferencias en las estimaciones. Esta colaboración en la estimación a través de Scrum Poker facilita la convergencia hacia un consenso acerca de la complejidad de la historia, permitiendo al equipo asignar puntos de historia de manera más precisa y colectiva.

Velocidad de Desarrollo

La velocidad de desarrollo en el contexto de Scrum se refiere a la medida del trabajo que un equipo ágil puede completar en un sprint. Representa la cantidad de puntos de historia, tareas o funcionalidades que el equipo puede entregar de manera exitosa y consistente en un período de tiempo determinado, generalmente en una iteración. La velocidad de desarrollo es una métrica clave que proporciona una comprensión cuantitativa de la capacidad

y eficiencia del equipo, lo que permite una planificación más precisa de futuros sprints y entregas. A medida que el equipo avanza en su colaboración y optimiza sus procesos, la velocidad de desarrollo puede ajustarse y utilizarse como referencia para estimar la cantidad de trabajo que se puede abordar en ciclos futuros.

El cálculo de la velocidad de desarrollo en Scrum se basa en la recopilación de datos de sprints anteriores. Para determinarla, se suman los puntos de historia de las historias de usuario que se completaron en cada uno de los sprints previos. La velocidad de desarrollo se obtiene al calcular el promedio de estos puntos de historia completados en los sprints anteriores. Es fundamental tener en cuenta que este cálculo únicamente considera los puntos de historia que se completaron al 100% durante los sprints anteriores, excluyendo aquellos que quedaron parcialmente resueltos o no se abordaron por completo. Esta métrica proporciona una estimación valiosa de la capacidad del equipo para realizar trabajo en un sprint y es una guía fundamental para la selección de historias y la planificación del próximo sprint.

Selección de Historias de Usuario

La selección de historias de usuario para un sprint conlleva una evaluación integral, considerando tanto la prioridad de las historias como sus estimaciones en Puntos de Historia. Este proceso requiere una combinación de factores clave: la velocidad de desarrollo del equipo, las estimaciones en puntos de historia de las historias en el backlog y la priorización basada en el valor para el cliente. Con estos elementos en mente, el equipo de desarrollo y el Product Owner colaboran para determinar qué historias son viables y estratégicas para ser abordadas en el próximo sprint. Idealmente, se seleccionan historias de menor complejidad (menos Puntos de Historia) y alta prioridad, asegurando un enfoque en aquellas que generan un mayor impacto para el cliente y pueden ser entregadas con éxito en el período del sprint. La suma total de los puntos de historia de las historias seleccionadas debe aproximarse a la velocidad de desarrollo del equipo, lo que garantiza que el conjunto de historias sea manejable y que el equipo tenga la capacidad de completarlas en el sprint. En última instancia, esta meticulosa selección se traduce en un plan de trabajo realista y enfocado, optimizando la probabilidad de éxito en la entrega y el cumplimiento de los objetivos establecidos.

Ejemplo

Consideremos las siguientes historias de usuario, ordenadas por prioridad en el backlog:

- Historia A: 4 puntos de historia
- Historia B: 8 puntos de historia
- Historia C: 2 puntos de historia
- Historia D: 12 puntos de historia

- Historia E: 4 puntos de historia
- Historia F: 16 puntos de historia

Si nuestra velocidad de desarrollo es de 12 puntos, entonces en el siguiente sprint sería conveniente abordar las historias A y B. Sin embargo, la historia C no podría ser incluida, dado que nuestra capacidad es de 12 puntos por sprint.

5.3 Desglose Táctico en Tareas

Las historias de usuario se desglosan en tareas concretas durante la reunión de planificación del sprint. Cada tarea representa un paso hacia la finalización de una historia. Estas tareas se estiman en horas o días y se asignan a miembros del equipo. Este desglose permite un monitoreo detallado del progreso y proporciona una estructura clara para la ejecución. A continuación, se muestra un desglose de tareas y sus estimaciones en horas para la historia de usuario: “Crear una página de inicio de sesión para los usuarios”.

Tareas:

- Diseñar la interfaz de la página de inicio de sesión.
- Configurar el controlador de sesiones en Ruby on Rails.
- Crear una vista para el formulario de inicio de sesión.
- Implementar la lógica de autenticación en el controlador.
- Establecer las rutas necesarias para la página de inicio de sesión.
- Agregar validaciones de entrada y manejo de errores.
- Crear pruebas unitarias para verificar la funcionalidad del inicio de sesión.
- Integrar estilos CSS para mejorar la apariencia visual.
- Configurar las sesiones de usuario y el almacenamiento seguro de contraseñas.
- Realizar pruebas de integración para asegurarse de que el flujo de inicio de sesión funcione correctamente.

Estimaciones:

- Diseño de la interfaz: 4 horas.
- Configuración del controlador de sesiones: 2 horas.
- Creación de la vista del formulario: 3 horas.
- Implementación de la lógica de autenticación: 6 horas.
- Establecimiento de rutas: 1 hora.
- Validaciones y manejo de errores: 3 horas.
- Pruebas unitarias: 5 horas.
- Integración de estilos CSS: 2 horas.
- Configuración de sesiones y seguridad de contraseñas: 4 horas.
- Pruebas de integración: 4 horas.

Este desglose de tareas proporciona una visión detallada de los pasos necesarios para completar la historia de usuario y permite al equipo estimar el esfuerzo requerido para cada tarea. Cada tarea puede asignarse a un miembro del equipo, lo que facilita el seguimiento del progreso y la ejecución eficiente del trabajo.

5.4 Planificación a Nivel de *Release*

La planificación a nivel de *release* es crucial para garantizar la visión a largo plazo. Implica estimar las iteraciones necesarias y las funcionalidades que compondrán el entregable final. Aquí, la colaboración entre el equipo de desarrollo y el Product Owner es vital. A pesar de la adaptabilidad inherente, se debe mantener una visión clara de las funcionalidades esenciales que conformarán el *release*.

Un *release* implica la entrega de un conjunto de historias de usuario. Por lo general, estas historias deben desarrollarse a lo largo de varios sprints. Por ejemplo, un *release* puede abarcar 4 sprints, en los cuales se trabajarán en 10 historias de usuario. Si cada sprint tiene una duración de 2 semanas, el *release* se completará en un período de 8 semanas.

5.5 Estrategias de Ajuste en la Planificación

Como ya hemos dicho antes, por diversas razones, la gente que hace esta planeación se ve sujeta a restricciones que muchas veces no están basadas en estimaciones reales o evidencia técnica, sino a factores comerciales u otro tipo de restricciones: debemos salir con esto en septiembre, tienes 3 meses para lograrlo, etc.

Debido a lo anterior es muy difícil que se logre construir el software con todas las prestaciones solicitadas en la fecha inicialmente prevista y se hace necesario ajustar a medio camino. Hay varias formas de ajustar, cada una con sus ventajas y desventajas.

Ajuste por Tiempo

Implica renegociar la fecha de entrega (extensión). Digamos que se planificaron 8 sprints de 2 semanas en que se iban a implementar 40 relatos de usuario. Al cerrar el sprint 4, vemos solo se han completado 16. A ese ritmo es probable que los 24 siguientes nos tome 6 sprints y no 4 por lo que negociamos un alargue de un mes adicional. Este tipo de ajuste, que suele parecerles muy lógico a los desarrolladores, generalmente tiene una muy mala recepción porque “no se cumple con lo prometido”. A veces de verdad hay una necesidad de tener el producto en una fecha dada y el atrasarse un mes no es una opción.

Ajuste por Alcance

Implica renegociar las funcionalidades que serán incluidas en el *release*. Tomando el mismo ejemplo anterior, al cerrar el sprint 4 vemos que a ese ritmo lograremos completar solo 32 de los 40 relatos. Dado que el desarrollo se ha llevado a cabo priorizando las funcionalidades más importantes para el negocio, esos 32 relatos deben tenerle 90% o más de las cosas que realmente le importan al cliente. Entonces ofrecemos entregar el producto en la fecha acordada, pero con un pequeño porcentaje de funcionalidades que quedarán para un *release* futuro del producto. Este tipo de ajuste es por lo general mucho mejor recibido. De hecho, si se ha priorizado correctamente, es posible que al cliente ni siquiera le importe demasiado lo que quedará fuera del *release*.

Ajuste por Calidad

Este es el tipo de ajuste que **NUNCA** se debería hacer, y sin embargo, el que más se hace. Tomando el mismo ejemplo anterior, el equipo de desarrollo, al darse cuenta del atraso al final del sprint 4 decide que “hay que apurar”. Esto suele traer consigo el sacrificio de cosas que permiten asegurar la calidad del producto como una reducción en *testing*, eliminar las revisiones de código, eliminar la programación en pares, eliminar las revisiones de producto y retrospectiva, etc. Finalmente, se entrega el producto a tiempo con todas las funcionalidades prometidas, pero con dos efectos laterales indeseados importantes: un compromiso serio de la calidad (puede tener serias consecuencias más adelante) y un desgaste y frustración extrema del equipo de desarrollo.

En estas situaciones se habla de haber contraído una deuda técnica. Al igual que una deuda financiera, una deuda técnica implica un compromiso a pagar más adelante lo que no puedo pagar hoy, pero con los intereses acumulados. Por ejemplo, un diseño apresurado puede significar un ahorro de tiempo hoy que tendrá que ser pagado más adelante cuando nos demos cuenta del error. Un *bug* no detectado puede posteriormente significar muchas horas de *debugging* para corregir.

Equipos de Desarrollo de *Software*

Como tan bien lo expresó quien probablemente sea el mejor jugador de basketbol de la historia: “el talento individual te puede hacer ganar partidos, pero es el trabajo en equipo lo que te hace ganar campeonatos”. Bien lo sabe la selección de futbol de Argentina que reúne muchísimo talento individual, pero no ha tenido un gran desempeño en los últimos campeonatos mundiales.

Entonces, si estamos de acuerdo en que el equipo es importante vale la pena tratar de definir lo que es un equipo. Un equipo no es simplemente un conjunto de individuos. Eso sería un grupo. Un equipo es un conjunto de personas con habilidades complementarias comprometidas con un propósito o meta bajo un enfoque en que todos son responsables.

El desarrollo de *software* se hace en equipo. Primero porque la tareas pueden ser de enorme envergadura y una persona sola no sería capaz de hacerlo. El trabajo de equipo permite lograr construir mucho más dado un tiempo disponible, permite contar con gente que tiene habilidades diversas, permite contar con gente que tiene ideas diversas y finalmente el equipo permite mantener la moral alta incluso cuando uno de los miembros sufre un bajón.

La gente del mundo del *software* es capaz de reconocer equipos de alto rendimiento. Generalmente, son equipos en que hay una alta sinergia (el todo es mayor que la suma de cada uno), donde el logro del equipo es más importante que el logro individual y donde hay una excelente comunicación entre los miembros. Más aún existen roles y responsabilidades claras y sobre todo hay un alto nivel de confianza.

Se han realizado estudios de cientos de equipos de alto rendimiento y hay cosas que se repiten en muchos de ellos:

- visión u objetivo compartido
- sentido de identidad
- miembros competentes
- compromiso con el equipo
- confianza mutua
- interdependencia entre miembros
- comunicación efectiva
- sentido de refuerzo (apoyados desde arriba)
- tamaño del equipo pequeño (máximo 10)
- sus miembros disfrutan de su trabajo

Visión u objetivo compartido se refiere a que exista una meta motivadora (un *driver*) bajo el cual se mueva el equipo. En un equipo de fútbol puede ser obtener el campeonato nacional o la “*champions*”. En un grupo de desarrollo de *software* puede ser, por ejemplo, construir el mejor navegador de la historia o una aplicación que puede ser usada con facilidad por personas desde 10 a 80 años.

Sentido de identidad se refiere a lo que la mayoría entendemos por “camiseta”. Sentirse parte de un equipo en que se comparte un nombre, un logo e incluso algunos chistes. Para ellos muchas veces los equipos usan poleras asociadas al proyecto o se mandan a hacer tazones alusivos.

6.1 Las fases de maduración de un Equipo

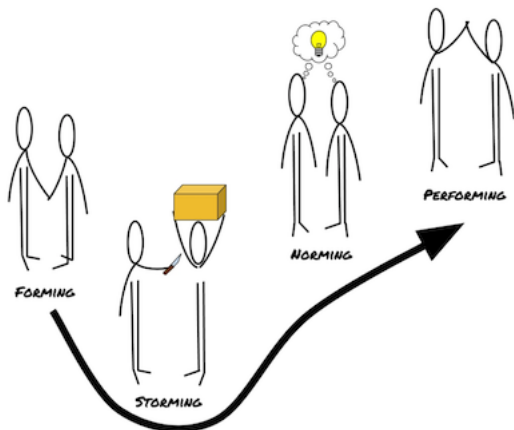
Un equipo de alto rendimiento no se puede armar de la noche a la mañana. Principalmente, porque hay que desarrollar la confianza. La metáfora correcta es la de un jardín que requiere un tiempo para que sea realmente hermoso y armónico. Toma tiempo y cuidados lograrlo. En ese proceso cada uno de los miembros del equipo debe procurar conocerse a si mismo y a los compañeros y cultivar una cultura de confianza.

Se ha estudiado el proceso de maduración de un equipo encontrándose 4 fases diferentes por las que deberá pasar hasta llegar a ser un equipo ideal: *forming*, *storming*, *norming* y *performing*.

La fase de *forming* corresponde a un período exploratorio, de mucha cautela y de exploración de límites. En la fase de *storming*, por lo general la etapa más compleja y desafiante, se definen aspectos de poder, control y liderazgo: pueden surgir conflictos e incluso peleas muy duras. Sin embargo, después del temporal viene la calma y a la etapa de *storming* la sucede la etapa de *norming*, donde finalmente se establece cohesión, se comienzan a apreciar las diferencias y nace la confianza entre los miembros del equipo. Desde allí es cosa de tiempo y de perfeccionamiento hasta llegar a la etapa de *performing*

en que se aprecia un claro sentido de identidad, un alto nivel de trabajo, y en general, un funcionamiento óptimo como equipo.

Es difícil saltarse una etapa, pero es posible acelerar el proceso. Por ejemplo, en la etapa inicial, el tener claridad en las metas u objetivos y también en las responsabilidades junto al conocimiento y validación de las habilidades individuales puede ayudar. En la difícil etapa de *storming* establecer normas claras de discusión, y sobre todo de la forma en que los conflictos van a ser resueltos, puede ser crucial.



6.2 Modelos de Equipo según Tarea Predominante

El desarrollar un producto de *software* requiere que el equipo pase por distintos modos. Por ejemplo, en el inicio cuando estamos imaginando el producto es bueno fomentar la creatividad, poner pocas restricciones. Más adelante, posiblemente el grupo se dedique la mayor parte del tiempo a la ejecución del proyecto con etapas en que debe actuar en modo de resolver dificultades o problemas.

Tenemos entonces a lo menos 3 tipos de tarea predominante: creatividad, resolución de problemas y ejecución táctica. Un equipo que está trabajando en modo creativo va a necesitar explorar con mucha libertad posibilidades y alternativas sin restricciones.

Un equipo trabajando en modo *problem solving* se puede ver con varios miembros del equipo mirando algunos diagramas o fórmulas en un *whiteboard* para tratar de solucionar algún obstáculo.

Finalmente, los miembros de un equipo en modo ejecución táctica trabajan concentrados en hacer un trabajo que está muy claro en la forma más rápida y eficiente posible.

6.3 Modelo de Equipo Atlético vs Modelo de Grupo de Teatro

Aquí hay otra analogía que puede resultar útil para entender la dinámica de algunos grupos de desarrollo de *software*. En el modelo de equipo atlético, por ejemplo, un equipo de fútbol, los jugadores son las estrellas y tienen mucha independencia para hacer su trabajo. El director técnico les da algunas indicaciones, pero son de tipo más bien general y no de detalle y él no juega. Su rol es mantener la moral del equipo alta y hacer que cada uno de los jugadores desarrolle su potencial al máximo.

En el modelo de grupo de teatro hay una dirección mucho más fuerte. El director tiene una idea muy clara de cómo la obra debe ser representada y se ensaya hasta su completa satisfacción. Los actores pueden introducir elementos personales de acuerdo a su propio talento, pero no tienen la libertad que tienen los jugadores de fútbol una vez que se inicia el partido.

Un equipo de desarrollo puede también funcionar asemejándose más a un modelo de equipo atlético o a un modelo de teatro. Históricamente, podría decirse que el desarrollo de un proyecto de *software* se asemejaba más al modelo de grupo de teatro en que el jefe de proyecto tomaba toda la responsabilidad. La tendencia en la actualidad, influenciado por las metodologías ágiles, es tender más a lo primero que a lo segundo y darle completa responsabilidad al equipo. De hecho en Scrum, no existe un “jefe de proyecto” que se consideraba un rol clave en cualquier desarrollo en las épocas anteriores.

6.4 El modelo de programador jefe (*Chief programming team*)

Este modelo está inspirado en que se ha observado que existen super programadores. Personas que pueden producir hasta 10 veces la cantidad de código de un profesional promedio, siendo este código incluso de mejor calidad y con menos errores. Parecería entonces un malgasto de este recurso asignarlo como “uno más” en el equipo. Lo que este modelo sugiere entonces es que esta persona diseñe y escriba la mayor parte del código, dejando al resto del equipo solamente para que lo apoye en todo lo que necesite: reuniones con clientes y con los ejecutivos, instalación de herramientas, actualización de *software*, etc. Todos actúan para facilitar el trabajo del “*chief programmer*”.

Lo más parecido a este modelo en otro dominio podría ser un neurocirujano de gran experiencia que se encarga de la mayor parte de una intervención quirúrgica compleja, dejando al resto del equipo solo tareas preparatorias, de apoyo durante la intervención o de cierre.

En el ámbito deportivo podríamos pensar en un jugador excepcional como por ejemplo Leo Messi que juega en el Barcelona. En ocasiones uno puede

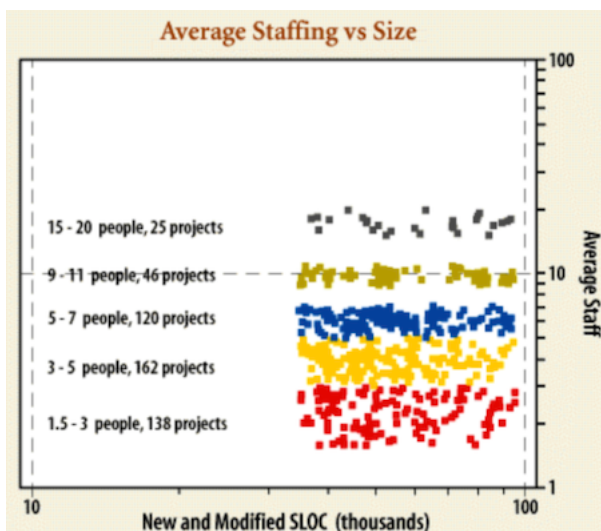
notar que el equipo juega para Messi. El resto de los jugadores hace las tareas rutinarias y al final todo se trata de pasarle el balón para que él pruebe hacer el gol.

Este modelo de desarrollo de *software* ha demostrado ser efectivo, pero tiene dos problemas enormes: primeramente, que estos super programadores no son muy comunes, por lo que no siempre puedes tener la suerte de contar con uno de ellos, y segundo, si es que tienes a uno, existe un riesgo muy elevado de perderlo con lo cual el equipo queda en un muy mal pie.

6.5 Los equipos deben ser pequeños

Hay suficiente evidencia que muestra las ventajas de los equipos relativamente pequeños. Un estudio reciente sobre 491 proyectos de *software* medianos (35 mil a 95 mil líneas de código fuente) lo demuestra empíricamente en forma muy clara. Se estudió la productividad, el tiempo de desarrollo y el esfuerzo para cada uno de estos proyectos.

La siguiente imagen muestra la distribución de los datos utilizados. Cada color representa proyectos con equipos de tamaños, desde muy pequeños (1 a 3 personas) hasta relativamente grandes (15 a 20 personas) según el tamaño del proyecto.



Los resultados son elocuentes. En términos de productividad, los equipos de hasta 7 personas son los que dieron mejores resultados. Aun cuando los de 5 a 7 tienen promedios algo más bajos que los aún más pequeños, ellos exhiben una variación menor, lo que es deseable.

En términos de esfuerzo, es impresionante observar como los grupos demasiado grandes tienen asociados esfuerzos muchísimos mayores. Recordemos

que en cada uno de estos segmentos hay proyectos de todos los tamaños, por lo que las diferencias de tamaño no explican estas cifras.

En cuanto al tiempo de desarrollo, nuevamente observamos que los equipos de 5 a 7 personas parecen ser los que lo hacen mejor seguidos por equipos algo menores.

Estos resultados, que coinciden con la experiencia de muchas empresas, dan cuenta de las ventajas en términos de menor necesidad de comunicar y coordinar de los equipos pequeños en comparación a los más grandes. Los equipos de 5 a 7 personas parecen ser mejores que equipos extremadamente pequeños, probablemente porque el equipo provee mayor protección a la pérdida o baja de productividad de algún miembro en particular y porque se puede construir un mayor sentido de identidad y cohesión.

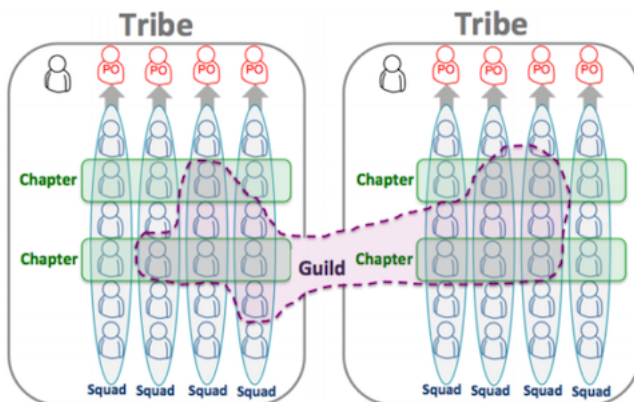
En definitiva, entonces por lo menos para proyectos de mediana envergadura deberíamos tender a equipos de entre 3 y 7 personas como máximo. La pregunta que surge de inmediato es entonces como se las arregla para organizarse cuando el proyecto requiere 50, 100 o 200 ingenieros. Esto es lo que trataremos de abordar en la siguiente sección.



6.6 Topología de Equipos de Desarrollo

La empresa Spotify, que maneja una de las aplicaciones para distribuir música más populares y exitosas del mundo, ha publicado en forma muy detallada la forma en que ellos organizan a los cientos de ingenieros que trabajan en este producto o más bien servicio.

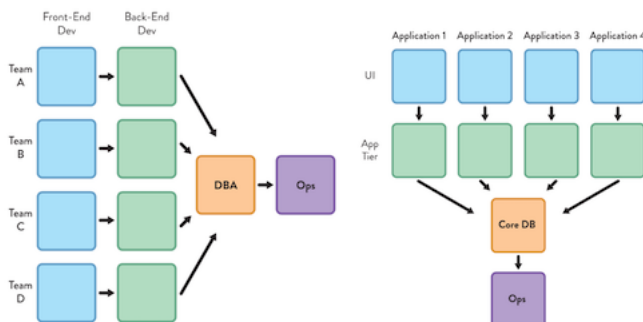
Hay equipos pequeños de 5 a 7 personas que ellos denominan *squads*, pero a la vez varias de estas *squads*, que trabajan en aspectos muy relacionados, se constituyen como tribus. Adicionalmente, especialistas de diversas *squads* de una tribu pueden constituir *chapters* e interesados en un aspecto más amplio de diversas tribus pueden formar *guilds*.



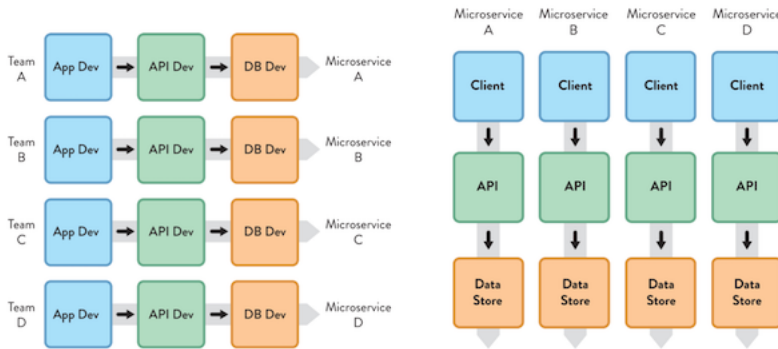
Hay diversas estrategias para organizar a la gente de modo de mantener la eficacia de los equipos pequeños, pero a la vez sacar máximo partido de los recursos humanos altamente especializados con que cuenta la organización.

Un aspecto muy interesante que hay que tener en cuenta es que, más allá de afectar la eficiencia, la manera como se organice a las personas va a influir en la arquitectura final del producto. Esto es conocido como la ley de Conway: *“Las organizaciones que diseñan sistemas tienden a producir diseños que son copias de las estructuras de comunicación de ellas.”*

Por ejemplo, una organización de 4 equipos, cada uno responsable del *front end* y del *back end*, pero no de la base de datos, mientras otro equipo se encarga de la base de datos y otro de la puesta en producción, dará origen a una arquitectura que es un espejo de esta organización.

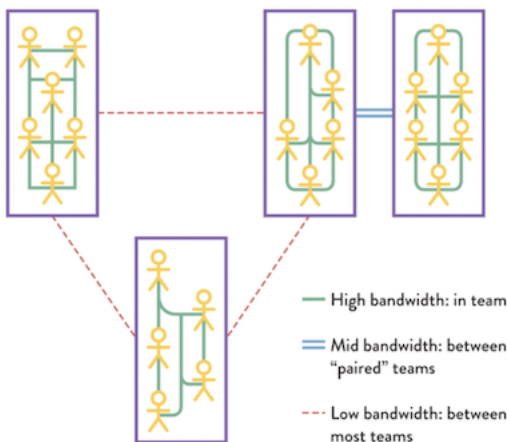


Si los equipos, en cambio, son responsables de todo el proceso, incluyendo el almacenamiento y la puesta en producción, ello puede dar origen a una arquitectura de microservicios como la que muestra la siguiente figura.



Cuando se tienen muchos equipos es posible que deba existir comunicación no solo entre los miembros de un equipo, sino también inter equipos. Sin embargo, no se requiere un ancho de banda tan alto en las comunicaciones que son entre equipos distintos.

La figura ilustra 3 situaciones de comunicación: dentro del equipo en que se requiere el máximo ancho de banda, entre equipos pareados (están trabajando en forma muy cercana en algo) y entre equipos en general.



El ancho de banda necesario puede manejarse por medio de ubicación geográfica o uso de herramientas de colaboración. El mayor ancho de banda se logra con la comunicación cara a cara y es por esta razón que los miembros de un equipo se suelen poner muy próximos el uno al otro. Un ancho de banda intermedio pueden ser proporcionados por herramientas como Google Meet, Zoom, etc. Un ancho de banda más bajo por herramientas como email o servicios de mensajería (Telegram, Slack, etc)

Skelton y Pais en su libro *Team Topologies*, identifican 4 topologías fundamentales y 3 modos de interacción entre equipos. Las 4 topologías son las siguientes:

- *stream-aligned team*
- *enabling team*
- *platform team*
- *complicated-subsystem team*

Los 3 modos de interacción son los siguientes:

- *collaboration*
- *X-as-a-Service*
- *facilitating*

A continuación revisamos primero los 4 tipos de equipo.

Stream-aligned team: este es el principal tipo de equipo que encontraremos en cualquier organización. Es el equipo que uno asocia con un equipo Scrum. Este tipo de equipo es responsable de llevar un flujo de trabajo de valor para el negocio, desde el inicio hasta el cliente. Puede ser un solo producto o servicio, un conjunto, un viaje del cliente, etc. Estos equipos son cercanos al cliente y rápidos en incorporar el feedback recibido y se les empodera para entregar valor rápido en forma lo más independiente posible.

Enabling team: los equipos de tipo *stream-aligned* trabajan bajo la presión de entregar valor. No pueden darse el lujo de investigar, aprender nuevas tecnologías, etc. Aquí viene al auxilio un segundo tipo de equipo que está compuesto por especialistas en un dominio técnico o en un producto determinado. Un *enabling team* está preparado para probar opciones y hacer sugerencias sobre herramientas, *frameworks*, etc. Son en esencia fuertemente colaborativos y deben mantenerse alertas y ser proactivos para entender lo que está pasando en los distintos *streams*. Estos equipos son mensajeros de las buenas noticias (hay una nueva herramienta que facilita) y también de las malas noticias (*framework X* deja de ser mantenido).

Complicated-subsystem team: Muchas veces un sistema incluye uno o más subsistemas de alta complejidad que requiere de conocimientos de especialistas en el área. Pensemos, por ejemplo, en una compañía que produce juegos que dependen de un “motor de física” que hace los cálculos de movimientos. O en el caso de Spotify, el subsistema recomendador que usa sofisticados algoritmos de inteligencia de máquina. El propósito de un equipo de esta naturaleza es precisamente construir y mantener estos subsistemas, por una parte, y también interactuar con otros equipos que hacen uso de estos subsistemas, ya sea publicando APIs como servicio o directamente colaborando con ellos.

Platform team

El propósito de estos equipos es construir y mantener servicios fundamentales para todos los otros equipos. Por ejemplo, servicios de autenticación,

servicios de almacenamiento de datos, servicios de mensajería, etc. La interacción con otros equipos es fundamentalmente en forma de publicar APIs que son consumidas por otros equipos en sus necesidades de desarrollo.

6.7 Modos de Colaboración

Estos diferentes tipos de equipo van a colaborar esencialmente de tres formas que examinamos a continuación:

- *Collaboration* - trabajo muy cercano de un equipo con un segundo equipo
- *X-as-a-service* - un equipo consume un servicio que otro equipo provee
- *Facilitating* - un equipo ayuda a un segundo equipo a superar algún problema o llevar a cabo una tarea

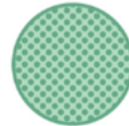
Los autores del libro antes mencionado usan la siguiente convención gráfica para denotar estos tres modos distintos de colaboración.



Collaboration



X-as-a-Service

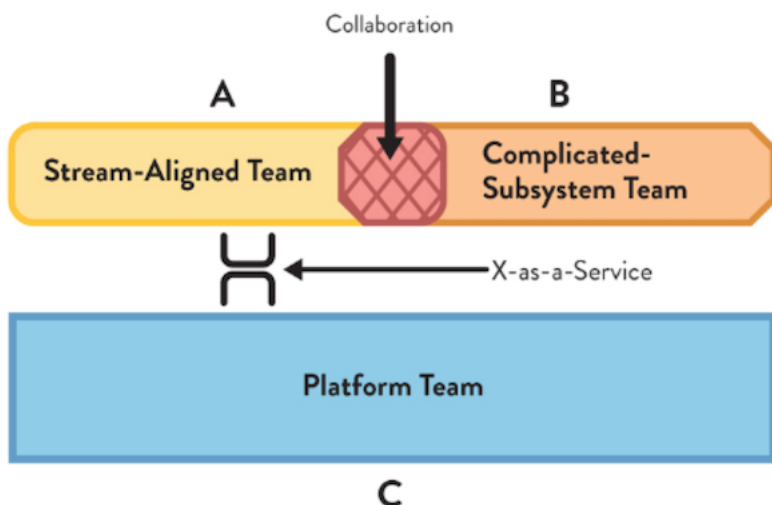


Facilitating

El modo de *Collaboration* se caracteriza por una alta interacción y respeto mutuo. Existe poca necesidad de transferir y hay innovación y descubrimiento rápido. Por otra parte, puede reducirse la efectividad de un equipo durante la colaboración, ya que hay que aprender e interiorizarse de más cosas. Se amplía también el ámbito de responsabilidad del equipo.

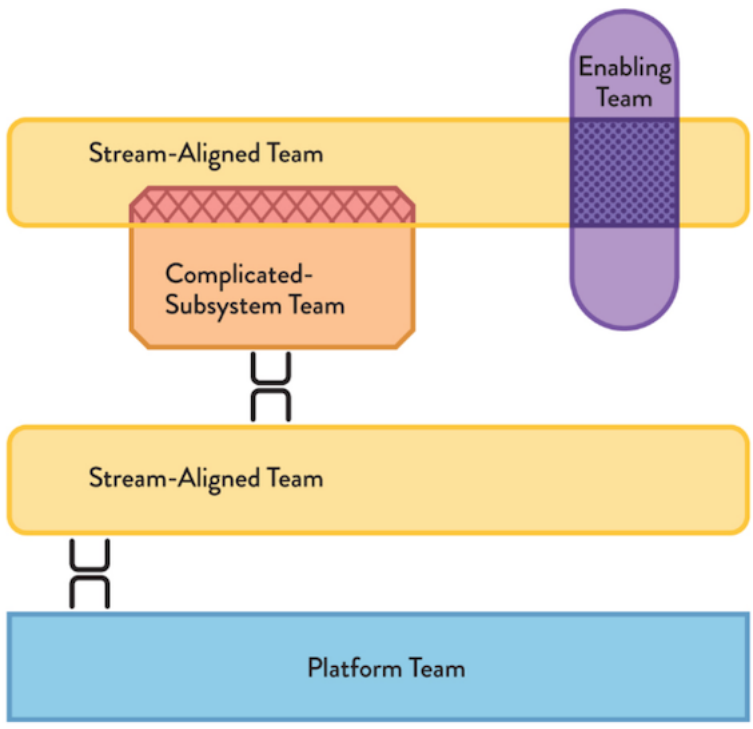
El modo *X-as-a-Service* se caracteriza por un especial cuidado en la experiencia de los usuarios consumidores del servicio. Las responsabilidades están muy claras y ni el equipo que consume el servicio ni el que lo proporciona necesita interiorizarse demasiado de los detalles del otro. Puede, sin embargo, producirse una menor innovación en torno a APIs que pueden no ser de lo más eficiente o efectivas.

Finalmentem el modo *Facilitation* al desbloquear a equipos permite en definitiva aumentar los flujos. Sin embargo, tener equipos que actúen principalmente en este modo implica asignar recursos humanos con experiencia a tareas que no son productivas en forma directa. Otro problema que puede haber es que en algunas ocasiones la interacción entre los ayudadores y los ayudados puede sentirse algo extraña.



La figura muestra un diagrama en que hay 3 equipos: A que es *stream-aligned*, B que es del tipo *complicated subsystem* y C que es de *platform*. El equipo A está interactuando en modo *collaboration* con el equipo B y además en forma de *X-as-a-Service* con el equipo C.

La interacción puede ser mucho más compleja e involucrar equipos de los cuatro tipos distintos, como muestra el siguiente ejemplo en que hay 5 equipos y todas las formas de colaboración entre ellos.



Estimaciones

7.1 Lo que deseamos estimar

Una planificación adecuada de un proyecto de desarrollo de *software* requiere conocer tempranamente dos magnitudes: el tiempo necesario para realizarlo, medido por ejemplo en semanas, y el esfuerzo involucrado en ello, medido por ejemplo en semanas-hombre. Lo primero porque pueden haber *deadlines* duros de entrega, compromisos de entrega e incluso multas en caso de atrasos. Lo segundo porque está directamente relacionado con los costos.

A pesar de que el foco principal está en estimar los números para esas dos magnitudes, es muy común que la primera estimación que hagamos corresponda al tamaño o envergadura del producto de *software* a desarrollar. Esto porque esta magnitud va a incidir directamente en las dos anteriores y es más fácil partir por allí.

Es importante hacer notar también que se hacen estimaciones desde antes que el proyecto parta hasta que el proyecto termina. Por ejemplo, las estimaciones preliminares deben ser inmediatamente corregidas al completar el primer sprint, al cambiar la composición del equipo o por alguna situación excepcional.

7.2 Las Magnitudes y sus Métricas

La primera magnitud a considerar es el tiempo. Es lo más sencillo y se suelen utilizar meses, semanas, días y horas. Es importante destacar que la sola elección de la unidad de tiempo va a transmitir en forma implícita un sentido de precisión en la estimación. Así, por ejemplo, si se plantea que un desarrollo durará 14 días, uno esperaría un error de más menos algunos días. Si,

en cambio, uno dice que serán 2 semanas, uno tiende a entender que puede haber un error menor.

La segunda magnitud es el esfuerzo. Las métricas usadas aquí son el hombre mes, hombre semana u hombre hora y que corresponde al producto del tiempo empleado por el número de personas que trabaja en ello. Por ejemplo, si una determinada pieza de *software* se construyó en 4 meses por un equipo de 5 personas, el esfuerzo asociado fue de 20 hombres mes.

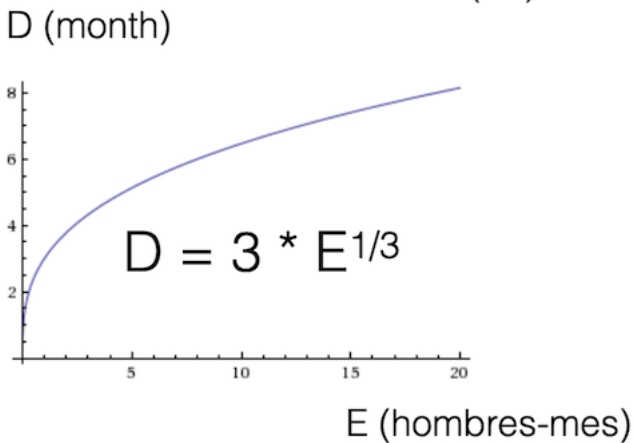
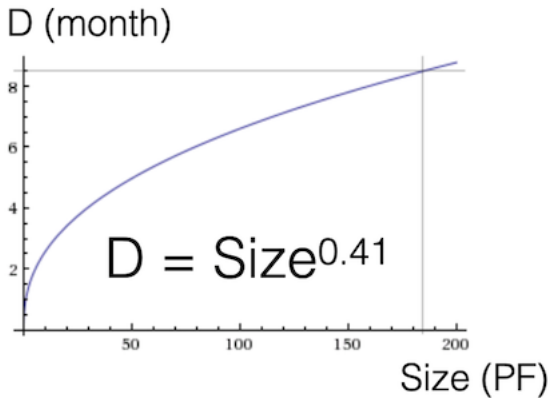
Es extremadamente importante al considerar el esfuerzo lo que Fred Brooks alertó en su célebre libro *El Mítico Hombre Mes* en el sentido que en el caso del *software* personas y tiempos no son intercambiables. Tomando el caso del ejemplo anterior en que el esfuerzo fue de 20 hombres mes, no necesariamente un equipo de 10 personas habría tardado solo dos meses en construirlo.

Tanto el esfuerzo como el tiempo dependen directamente del tamaño de la pieza de *software* o envergadura del proyecto. El tamaño se puede medir con distintas métricas, pero la más utilizada es la de líneas de código fuente (LOC). Esta métrica considera simplemente el número de líneas de código (posiblemente eliminando líneas vacías y comentarios) en el programa final. Más adelante retomaremos este tema y examinaremos otras métricas para medir el tamaño.

7.3 Del tamaño a la Duración y al Esfuerzo

Uno puede utilizar información empírica para construir modelos que permitan relacionar estas magnitudes. Lo ideal es tener modelos propios, pero si no se tienen pueden usarse los de la industria. A continuación se muestra la esencia de esta idea con un modelo sacado de la industria en USA.

El modelo relaciona la duración en meses con el tamaño en puntos de función (más adelante lo explicaremos) y la duración en meses en función del esfuerzo de desarrollo (ver figura).



Supongamos que hemos realizado una estimación de tamaño de 100 puntos de función. Entonces”

$$S = 100$$

$$D = 100^{**} 0.41 = 6.6 \text{ meses}$$

$$6.6 = 3 * E^{**} (1/3) \Rightarrow E = (6.6/3)^{**} 3$$

$$E = 10.6 \text{ hombres mes}$$

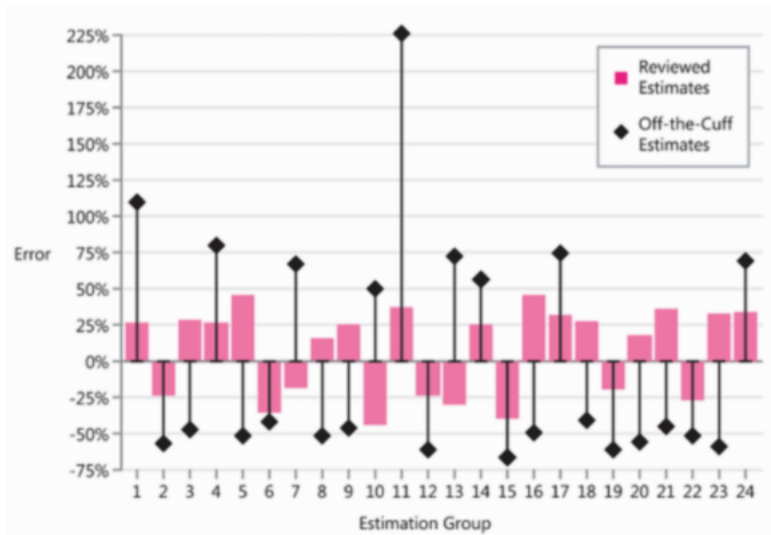
10.6 hombres mes con una duración de 6.6 meses sugiere usar a dos personas para este proyecto.

7.4 Evitar la respuesta improvisada

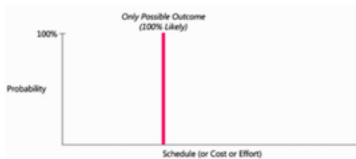
Este es un error en que incurren muchas veces los ingenieros poco experimentados. De pronto alguien pide una opinión del tipo “cuanto crees que tomaría hacer algo como ...” y, en lugar de callar, el ingeniero responde “e imagino que una semana”. La respuesta correcta debería ser “lo estudiaré y

te contesto en un rato más”. Incluso un análisis superficial, pero con calma, va a entregar números mucho mas cercanos a la verdad.

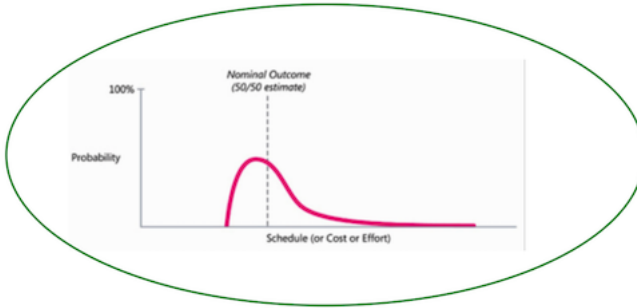
El gráfico siguiente muestra lo alejada que está la estimación rápida (rombos negros) de la estimación realizada en forma metódica más tarde.



Nunca hay que olvidar que una estimación siempre es una afirmación probabilística. Es decir, cuando uno afirma que el desarrollo tomará, por ejemplo, 14 semanas, lo que se está en realidad afirmando es que 14 es lo más probable, pero en realidad hay una distribución de probabilidad alrededor de ese valor (ver figura)



En realidad la curva será más bien como muestra la siguiente figura, ya que es mucho mas común que uno se quede corto con la estimación y no al revés.



Esto nos lleva a comparar el problema de subestimar con el de sobreestimar. Sobreestimar significa, por ejemplo, estimar el tiempo de desarrollo en 8 semanas cuando en realidad solo se requerían 6 semanas. Subestimar es lo contrario: se estimaron 6 semanas y en realidad se requerían 8.

7.5 Problemas de Subestimar y Sobreestimar

Obviamente que no puede ser bueno ninguna de las dos cosas, pero los problemas que trae el subestimar son mucho mayores que el de sobreestimar.

El sobreestimar el tiempo de desarrollo está asociado a dos problemas conocidos como la **ley de Parkinson** y la **ley de Goldratt**.

La ley de Parkinson establece que el trabajo se expandirá hasta usar todo el tiempo disponible.

La ley de Goldratt, también conocida como el síndrome de estudiante, dice que si hay demasiado tiempo se va a malgastar el tiempo al comienzo.

A pesar de estos problemas, la penalización por sobreestimar es mucho menor a la de subestimar. En el primer caso, la penalización crece linealmente con la magnitud de la sobreestimación, en cambio, en el segundo caso, la penalización puede crecer en forma exponencial por las razones que veremos a continuación.

La figura muestra como la penalización crece mucho más rápido cuando hay una subestimación en comparación con el caso contrario.

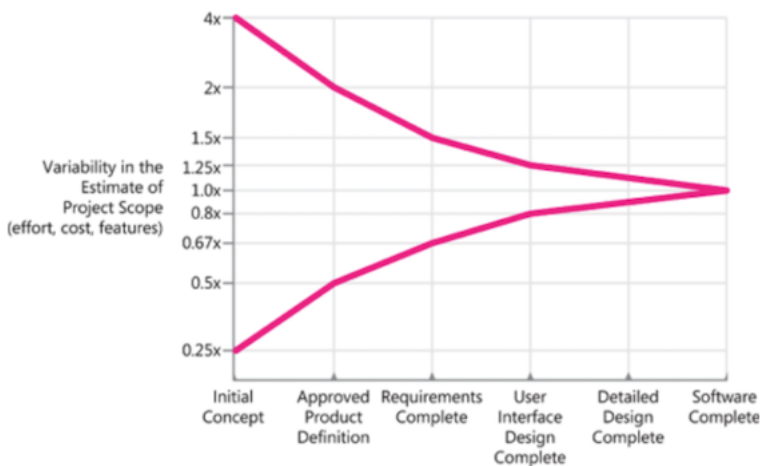


¿Por qué es tan problemático subestimar el tiempo (o el esfuerzo) de desarrollo? Hay muchas razones, pero básicamente trabajar en un modo en que el proyecto está permanentemente atrasado genera un enorme nivel de estrés en el equipo que puede traducirse en una baja de rendimiento general sin contar problemas de calidad o el riesgo de perder a parte del equipo a medio camino.

A veces se produce una suerte de efecto dominó. Se deben reducir tiempos de análisis y diseño, lo que a la larga genera mas atrasos. Hay que reagendar reuniones, lo que consume más tiempo del originalmente planeado, hay que ajustar los planes, etc.

7.6 El Cono de la Incertidumbre

Ya hemos dicho antes que se hacen estimaciones a lo largo de toda la duración de proyecto y que los errores en las estimaciones disminuyen a medida que el proyecto avanza. Inicialmente, puede haber un error de 4 veces hacia arriba o hacia abajo de la magnitud que termina siendo pero a medida que el proyecto avanza esta incertidumbre disminuye generando una curva que se conoce como el cono de incertidumbre. La figura muestra el cono para un modelo de proceso clásico de cascada pero es igualmente válido en el caso de desarrollos ágiles, solo que en lugar de tener etapas en el eje X tendremos Sprint 1, Sprint 2, etc.



Hay dos mecanismos importantes para ir reduciendo este cono:

- el desarrollo iterativo e incremental
- la descomposición en unidades pequeñas

Al primero ya nos hemos referido, pero volveremos más adelante. Lo segundo es porque es mucho más fácil estimar una tarea pequeña que una

grande. Por ejemplo, cuanto tiempo me toma construir un sitio web para una empresa de *delivery* puede ser complejo, pero estimar el tiempo para implementar la autenticación del usuario es mas tratable En este sentido, la división de la funcionalidad en base a relatos de usuario nos será de gran utilidad.

7.7 Fuentes de Error en las Estimaciones

Existen cuatro fuentes de error posibles relacionados con las estimaciones:

Información imprecisa sobre el proyecto

No se entendió bien lo que se quería hacer o el análisis no fue lo suficientemente profundo. Se dijo que era una página web simple y resultó que se trataba de un *ecommerce* completo.

Información imprecisa sobre las capacidades del equipo

Si los integrantes del equipo nunca han trabajado juntos o si lo han hecho, pero en otro tipo de proyecto es muy difícil estimar su productividad. ¿Podrán sacar 2, 3 o 6 relatos en un sprint? Es sabido que hay diferencias muy grandes de productividad entre programadores individuales y mucho más si incorporamos aspectos de trabajo en equipo (revisar la clase anterior)

Imprecisiones del proceso mismo de estimación

A estas alturas debería estar claro que el proceso de estimación puede tener muchas limitaciones: datos incompletos o que corresponde a información de la industria y no del equipo, modelos aproximados, uso de productividades constantes, no considerar el tipo de proyecto específico, etc.

El Proyecto es del tipo “blanco móvil”

Un proyecto de tipo blanco móvil es aquel que cambia permanentemente según como haya amanecido el encargado del proyecto. Se espera que ocurran cambios, pero a veces se está en presencia de una periodicidad en los cambios que hace casi imposible estimar nada.

7.8 Precisión, exactitud y uso de las métricas correctas

El ejemplo que comúnmente se da para comparar precisión con exactitud es el valor de PI. Si yo digo que el valor de PI es 3 no es muy preciso. Un valor de 3.14 es más preciso y un valor de 3.14159 es aún más preciso. Si yo digo que

el valor de PI es 5.123233341 estoy también comunicando precisión, pero el valor no es exacto.

En el caso de una estimación, la métrica o unidad utilizada transmite un sentido de precisión, por lo que hay que ser cuidadoso con ello. Si uno ha hecho una estimación de 3 meses para la duración del proyecto, pero lo comunica diciendo que tomará 90 días, un error de 7 días será un problema. Si, en cambio, dice que tomará 12 semanas y termina siendo 13, es mucho mejor tolerado. Si lo que se dijo son 3 meses, incluso un atraso de 2 semanas estará dentro de lo aceptable.

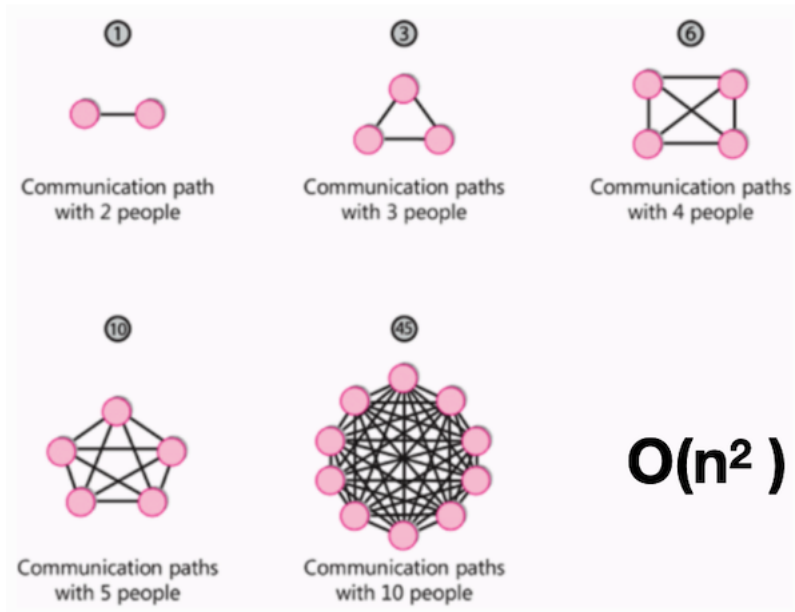
7.9 Todo parte con el tamaño

Es lo primero que se tiene a mano para llegar finalmente a tiempo de desarrollo y esfuerzo. Se sabe que a mayor tamaño mayor esfuerzo y mayor tiempo de desarrollo. Sin embargo, en el mundo del *software* ocurre algo inusual: la diseconomía de escala. Esto hace que mientras más grande el proyecto vamos a tener una menor productividad. Esto significa que un producto de 100.000 LOC probablemente requerirá más de 10 veces el tiempo que toma hacer uno de 10.000 LOC.

2020-09-29_06-46-32_PM.png

La razón de esta diseconomía de escala es que a medida que el proyecto crece necesitaremos más personas en los equipos, lo que se traducirá en mucho más tiempo de comunicación y coordinación entre ellos. Si tenemos dos nodos en un grafo, solo hay un arco posible que los conecte. Si hay 3 nodos son 3 arcos y si hay 5 son 10 arcos. En general, el número de arcos conectando a N nodos es $N(N-1)/2$ y este número crece con el cuadrado de N .

Origen de la diseconomía de escala: necesidad de comunicación y sincronización



Hablemos ahora de las unidades. Hemos dicho que las líneas de código fuente (LOC) son ampliamente utilizadas como una métrica del tamaño. Aparte de ser muy simple, esta métrica tiene otra gran ventaja: que una vez terminada la construcción del *software* se puede saber en forma muy precisa el tamaño. Basta contar las líneas de código. Sin embargo, la métrica tiene muchos inconvenientes también.

La primera dificultad es que si bien es fácil saber el valor al terminar el proyecto, es extremadamente difícil hacer una estimación del número de líneas de código al comenzar el proyecto. Adicionalmente, esta métrica es dependiente de demasiados factores:

- el lenguaje de programación que se esté usando (hay lenguajes mas verbosos que otros)
- el estilo de escritura del código (indentación, posición de las llaves, saltos de línea, etc)
- calidad del código ($i++$ vs $i = i + 1$)
- uso de comentarios y líneas en blanco

Necesitamos entonces métricas de tamaño que

- no dependan del lenguaje que se usará
- faciliten la estimación temprana

Puntos de Función

Esta es una métrica que fue extremadamente popular y de amplio uso en los 80's y 90's, pero lentamente ha ido siendo dejado de lado por los tremendos cambios que han ocurrido en el tipo de software y la forma en que se desarrolla. Esta métrica reúne los dos requisitos anteriores: puede ser usada en forma temprana y no depende del lenguaje que será utilizado.

Para entender esta métrica hay que entender el tipo de programa típico de esa época. Los elementos principales que caracterizaban el tamaño resultante eran cinco factores que eran fácilmente estimables:

- número y complejidad de entradas (inputs)
- número y complejidad de salidas (outputs)
- número y complejidad de consultas
- número y complejidad de archivos (o tablas) internos
- número y complejidad de archivos (o tablas) externos

La complejidad de cada uno de estos elementos se estimaba en tres niveles: baja, mediana y alta y de acuerdo a ello producía un puntaje de acuerdo a la siguiente tabla.

Tipo	Baja	Mediana	Alta
Entradas	x3	x4	x6
Salidas	x4	x5	x7
Consultas	x3	x4	x6
Archivos Internos (tablas)	x7	x10	x15
Archivos Externos (tablas)	x5	x7	x10

Así, 2 entradas de baja complejidad producen $2 \times 3 = 6$ puntos y 3 consultas de alta complejidad contribuyen con $3 \times 6 = 18$ puntos. Sumando todo se obtiene un número que se conoce como puntos de función no ajustados (PFNA) que serán reducidos o incrementados en hasta un 35% dependiendo de un factor de complejidad del programa en general.

Este factor de complejidad considera 14 áreas en que se asigna un puntaje de 0 a 5:

1. Comunicaciones
2. Funciones distribuidas
3. Objetivos de desempeño
4. Configuración sobrecargada
5. Tasa de transacciones
6. Entrada de datos *online*
7. Eficiencia para usuario
8. Actualización en línea
9. Proceso complejo
10. Reuso
11. Facilidad de instalación

12. Facilidad de operación
13. Varios sitios
14. Facilidad de mantención

lo que produce un número entero N que va desde 0 a 70. A partir de este número, el factor de complejidad se calcula como:

$$FC = 0.65 + N/100$$

De allí que

$$0.65 \leq FC \leq 1.35$$

El número final de puntos de función es entonces

$$PF = PFNA * FC$$

Relatos y Puntos de Relato (*storypoints*)

Una métrica que comparte con la de puntos de función, el que no depende del lenguaje y sirve para estimaciones tempranas es la de puntos de relato (*storypoints*). Sabemos que el número de relatos de usuario es un buen indicador de la magnitud de lo que queremos construir. Lo que agrega la métrica de puntos de relato es tomar en consideración que hay relatos más complejos de implementar que otros.

La idea entonces es ponderar la complejidad del relato incorporando un multiplicador. Estos multiplicadores crecen con la complejidad del relato en una escala de Fibonacci o Exponencial.

Al usar una escala de Fibonacci, un relato simple cuenta por 1 punto, uno mas complejo 2, el siguiente 3, luego 5, etc. Con una escala exponencial, los puntos de relato crecen más rápido: 1, 2, 4, 8, ...

Así, por ejemplo, supongamos que clasificamos los relatos en solo 4 tipos, podríamos usar solo multiplicadores 1 para el más simple y 5 para el más complicado.

Estimación Colaborativa

La idea es hacer que cada miembro del equipo haga una estimación personal y secreta del puntaje de un relato. Luego se descubren simultáneamente las “apuestas” y se genera una pequeña discusión tratando de llegar a un acuerdo. Por ejemplo, alguien que asignó un puntaje 1 puede no haber visto determinada complejidad que sí vio quien le puso 5, pero al conversar cambia de opinión. Se pueden hacer varias rondas hasta converger en un determinado valor.

Esta dinámica es tan popular que se venden cartas con los números (Fibonacci o Exponencial) para hacer las apuestas. Cada jugador (miembro del equipo) necesita un mazo de cartas.

2020-09-29_07-51-39_PM.png

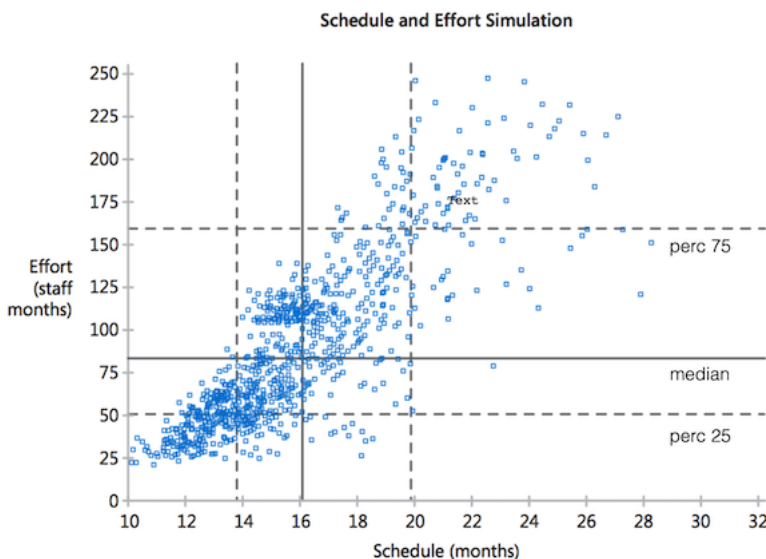
7.10 Del tamaño al esfuerzo

Sabemos que el esfuerzo no crece linealmente con el tamaño, pero cuando los números son relativamente pequeños se puede usar esa aproximación. Ello es equivalente a considerar un número de productividad, ya sea individual o del equipo medido en LOC por hombre-semana o puntos de relato por hombre-semana.

Esta cifra de productividad depende de muchos factores, incluyendo el desempeño individual de cada miembro del equipo y el funcionamiento como equipo, además de tipo de proyecto, plataforma de desarrollo, etc. Debido a esto, no resulta muy confiable usar parámetros de la industria en general y menos aún si es la industria norteamericana (se suele encontrar en estudios).

Lo mejor es utilizar cifras basadas en la información histórica del mismo proyecto en desarrollo (como han sido los sprints anteriores). Si no disponemos de ellas (proyecto no ha iniciado) se puede usar la información histórica de la organización (en otros proyectos similares). Solo en último término hay que recurrir a datos generales de la industria del tipo que muestra el estudio de la figura.

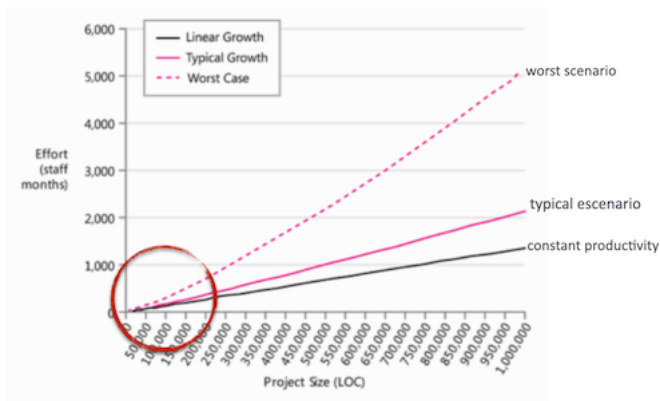
Los datos que se deben recolectar incluyen por supuesto líneas de código, tiempo de desarrollo, número de personas que participan. También conviene recolectar información sobre la calidad: número y tipo de defectos encontrados en relación a las líneas de código, etc.



Esta información debería recolectarse durante el proceso mismo en forma automatizada o si no es posible inmediatamente completado el proyecto. Si no se hace en ese momento, lo más probable es que no va a quedar registro alguno que pueda ser utilizado en el futuro.

Aun cuando sabemos que las cifras de productividad dependen mucho del equipo, muchas veces los equipos se mantienen o cambian solo ligeramente (uno o dos integrantes) por lo que el usar cifras anteriores de la organización es bastante útil.

En cuanto al supuesto de crecimiento lineal del esfuerzo en relación al tamaño, para tamaños relativamente pequeños no se induce un error demasiado grande. La figura muestra como la curva comienza a alejarse de la linealidad para programas de 250.000 LOC o más. El círculo muestra la zona donde normalmente estaremos operando.



Si se quiere mas precisión puede usarse una tabla con productividades crecientes del tamaño como la siguiente:

Team Size	Average Stories Delivered per Calendar Month
1	5
2-3	12
4-5	22
6-7	31
8	No data for projects of this size

A continuación se muestra un ejemplo de cómo se ajustan las estimaciones en base a la información obtenida en el mismo proyecto.

- Inicialmente, se inició con las estimaciones de puntos de relato para cada uno de los 60 relatos del sistema. Se usó un *scrum poker* con escala exponencial y el total de puntos fue de 180.
- Nos encontramos a punto de iniciar el segundo *sprint* (3 semanas) y, por lo tanto, ya sabemos lo que ocurrió en el primero. Estas son las

cifras: trabajaron 4 personas en estas primeras 3 semanas, quienes entregaron 27 puntos de relato.

- Esfuerzo = $3 \times 4 = 12$ semanas hombre
- Personas = 4
- Se produjeron 2.25 puntos por semana hombre
- Tiempo = 3 semanas
- Se produjeron 9 puntos por semana
- Con estas 2 cifras de productividad volvemos a nuestros 180 puntos del proyecto y hacemos las estimaciones globales
 - Esfuerzo = $180 / 2.25 = 80$ semanas hombre
 - Duración = $180 / 9 = 20$ semanas

Estimación por Analogía Simple

A veces se hace una estimación haciendo una directa analogía con un proyecto similar anterior. El proceso es como sigue:

- Etapa 1: Obtener cifras de tamaño, esfuerzo para proyecto similar
- Etapa 2: Poner las cifras del nuevo proyecto en términos relativos al anterior
- Etapa 3: Construir la estimación de tamaño para el proyecto nuevo
- Etapa 4: Construir estimación de esfuerzo basada en tamaño en comparación al antiguo
- Etapa 5: Asegurarse que son en verdad comparables /envergadura, tecnología, equipo, etc)

Un ejemplo se muestra a continuación.

Se pide desarrollar la versión 1.0 de una nueva aplicación Web llamada Triad. Se ha estimado que el modelo de datos incluye 15 clases de dominio y 14 tablas y comprende una interfaz compuesta por 19 páginas Web. Hay además que generar 14 gráficos y 16 reportes.

Se acaba de desarrollar una aplicación previa llamada AccSellerator, para la cual se requirió un esfuerzo de 30 meses hombre de la que se dispone de la información que muestra la tabla siguiente

Accelerator 1.0	
Database - 10 tables	→ 5.000 loc
Interface - 14 web pages	→ 14.000 loc
Graphs and Reports - 10 + 8	→ 9.000 loc
Foundation classes - 15	→ 4.500 loc
Business rules - ?	→ 11.000 loc

Hacemos la analogía entre lo que conocemos y lo que no conocemos extrayendo un factor multiplicativo

Subsystem	Actual Size of AccSellerator 1.0	Estimated Size of Triad 1.0	Multiplication Factor
Database	10 tables	14 tables	1.4
User interface	14 Web pages	19 Web pages	1.4
Graphs and reports	10 graphs + 8 reports	14 graphs + 16 reports	1.7
Foundation classes	15 classes	15 classes	1.0
Business rules	???	???	1.5

Aplicamos los factores a los tamaños conocidos de AccSellerator para obtener la estimación de tamaño de la nueva aplicación

Subsystem	Code Size of AccSellerator 1.0	Multiplication Factor	Estimated Code Size of Triad 1.0
Database	5,000	1.4	7,000
User interface	14,000	1.4	19,600
Graphs and reports	9,000	1.7	15,300
Foundation classes	4,500	1.0	4,500
Business rules	11,000	1.5	16,500
TOTAL	43,500	-	62,900

En definitiva, el tamaño de Triad será de 62.900 LOC, es decir, 1.45 veces el tamaño de AccSellerator para el cual se invirtieron 30 meses hombre. Si asumimos una relación lineal, se esperaría entonces que el desarrollo de Triad ocupe $30 \times 1.45 = 44$ meses hombre.



UML El Lenguaje de Modelación Unificado

8.1 Diseño en el contexto de procesos ágiles

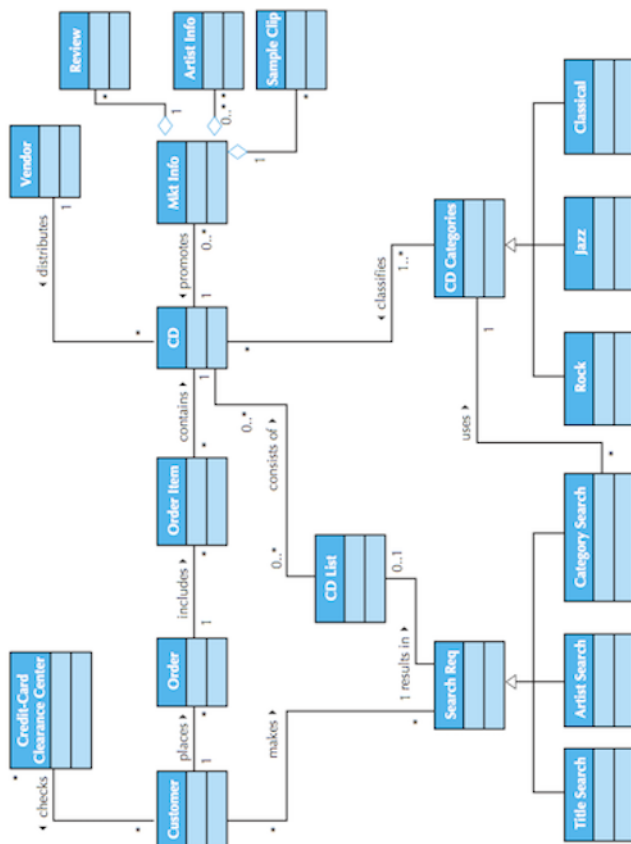
Tradicionalmente la etapa de diseño sigue a una etapa de análisis que a su vez cierra el levantamiento de requisitos. La diferencia esencial que se suele hacer entre análisis y diseño es que en el análisis nos enfocamos en “qué” mientras que en diseño el foco está en el “cómo”.

En el desarrollo ágil la separación entre análisis y diseño es muy sutil, casi imperceptible. Tanto durante el análisis como durante el diseño se busca construir un modelo: el modelo de análisis o el modelo de diseño. El modelo de análisis se parece mucho a un modelo de datos, mientras que el modelo de diseño incluye aspectos mucho más cercanos a la implementación.

8.2 Modelos de Dominio

El modelo de análisis, también llamado modelo de dominio, requiere identificar las clases de dominio y sus principales atributos y en este sentido se parece a un modelo de datos (modelo entidad relación), ya que las clases de dominio incluyen las entidades persistentes que deberán ser almacenadas (aunque puede incluir algunas otras adicionales también).

La figura muestra un ejemplo de modelo de dominio para una tienda que vende música en forma de CDs (¡todavía existen!). Puede observarse que se ha utilizado un diagrama de clases UML, pero no se han indicado ni atributos ni métodos de las clases. Lo esencial aquí es la identificación de las entidades y de las asociaciones entre ellas que son marcadas con líneas.



8.3 Cómo encontrar las clases (objetos) de dominio

Los candidatos deben buscarse en el trabajo realizado de levantamiento de requisitos funcionales, ya sea en los relatos de usuario, en los casos de uso o documentos clásicos de requisitos. Adicionalmente, podemos pensar en candidatos en el espacio general del problema (aquí sirve mucho la experiencia del analista), roles, eventos, ubicaciones geográficas o unidades organizacionales.

Pon atención en que estamos hablando de candidatos, ya que luego de esta etapa muchos de ellos finalmente no serán considerados como clases de dominio por distintas razones. Lo más común es que se trate mas bien de un atributo de una clase o que es una clase que ya fue considerada con otro nombre, etc.

Algunos autores sugieren revisar cuidadosamente el texto que describe los requisitos y asociar estructuras gramaticales como elementos del dominio: un sustantivo común sería una clase, un adjetivo un atributo, un verbo un

método, un verbo tener como agregación, etc. En mi opinión esto no es una muy buena idea, especialmente si las descripciones están en Castellano porque la flexibilidad del lenguaje admite muchísimas formas de decir lo mismo.

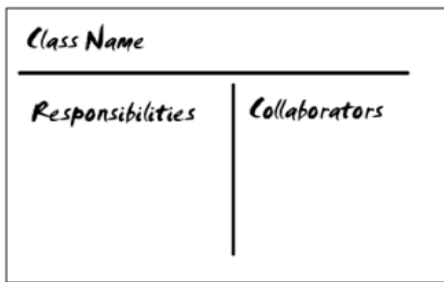
Algunos criterios para evaluar los candidatos encontrados son:

- se cumple que los objetos de esa clase guardan alguna información
- se puede pensar en más de un atributo
- se puede pensar en métodos
- todas las instancias de esa clase compartirán los mismos atributos y métodos

8.4 Clase, responsabilidad, colaborador (CRC)

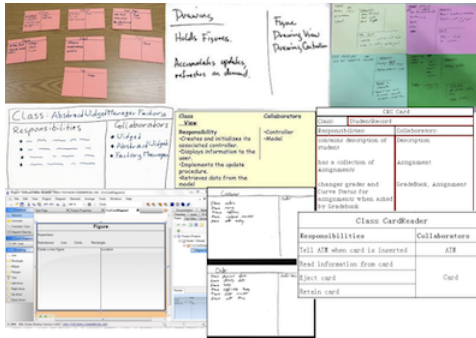
Las responsabilidades de un objeto tienen que ver con qué es lo que el objeto sabe (atributos) y qué es lo que el objeto es capaz de hacer (métodos). Los colaboradores son objetos de otras clases que son necesarios para que nuestro objeto pueda realizar lo que es capaz de hacer.

Una metodología muy popular que permite construir y mejorar el modelo de dominio es el de las tarjetas CRC (por *class - responsibility - collaborators*). Se construye una tarjeta por cada clase en que se pone el nombre de la clase, sus responsabilidades y los colaboradores (ver figura).



La idea es distribuir una tarjeta a cada persona e intentar llevar a cabo alguna funcionalidad. En el proceso puede encontrarse que faltaban o sobraban responsabilidades o colaboradores.

Es común el uso de tarjetas de 3x5 pulgadas para este fin, pero puede usarse cualquier otro medio. La siguiente figura muestra algunas tarjetas reales:



8.5 Del análisis al diseño

Ya hemos dicho que en la etapa de diseño pasamos del qué al cómo. Es particularmente importante tener criterios para decidir si un diseño es superior a otro (ambos responden el mismo qué pero con cómo diferentes)

La complejidad del problema necesariamente hará necesario descomponer la solución completa en elementos menores. Por ejemplo, 5 subsistemas cada uno con 4 componentes, cada una de las cuales tiene su propio diagrama de clases. La forma de descomponer el problema da origen a muchas posibles soluciones.

Los atributos que caracterizan a un buen diseño pueden ser:

- fácil de entender
- fácil de modificar y extender
- fácil de reutilizar en otro problema
- fácil de testear la implementación
- fácil integrar las distintas unidades
- fácil de implementar (programar)

8.6 Acoplamiento y Cohesión

Hay dos atributos que permiten sintetizar buena parte de lo que caracteriza un buen diseño: el acoplamiento y la cohesión. El acoplamiento representa el grado de interacción entre las distintas unidades y un bajo nivel de acoplamiento va a incidir en que el sistema sea más fácil de entender, modificar, extender y testear. La cohesión tiene que ver con que tanto tienen que ver entre sí las cosas que decidimos dejar juntas en una misma unidad. Una clase enfocada en una sola cosa es más fácil de entender, mantener y extender.

Lo que buscamos entonces es un diseño que tenga una alta cohesión y un bajo acoplamiento. Cabe hacer notar que estas dos cosas deben ser vistas

simultáneamente, ya que es fácil pensar en diseños de bajísimo o nulo acoplamiento pero con muy mala cohesión (por ejemplo todo en una sola unidad) o al revés, diseños de altísima cohesión pero enorme acoplamiento (clases extremadamente pequeñas).

Piensa en un tren. Los carros solo están conectados en un punto entre ellos (bajo acoplamiento) y cada carro lleva un contenido que es de un mismo tipo o para un mismo cliente (cohesión). El hecho de tener carros pequeños hace más fácil extender el tren, cambiar un carro por otro, etc.



Es imposible no tener acoplamiento, pero hay mejores y peores categorías de acoplamiento. Ordenados de peor a mejor los tipos de acoplamiento:

- acoplamiento por contenido (acceso a datos internos)
- acoplamiento por variables globales
- acoplamiento de control (una unidad influye en el flujo de control de otra)
- acoplamiento de datos (paso de parámetros necesarios)

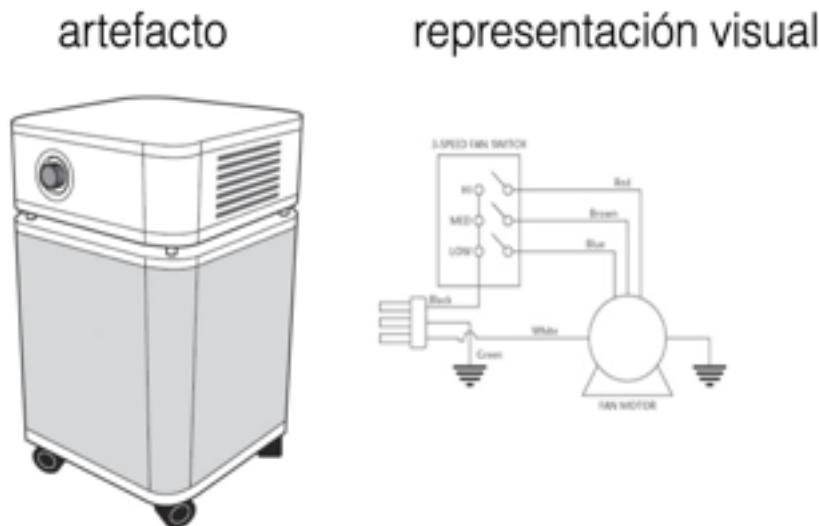
En cuanto a cohesión, las categorías desde menor (más malo) a mayor (de-seable) serían:

- coincidencial
- lógica
- temporal
- procedural
- comunicacional
- secuencial
- funcional

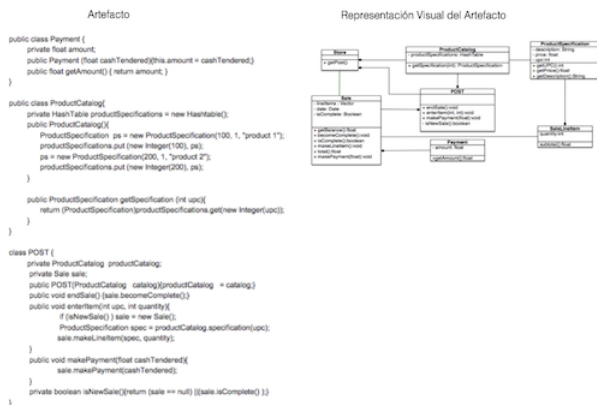
8.7 Un lenguaje visual para el software

La actividad de diseño se favorece enormemente al trabajar con un lenguaje gráfico de alto nivel. En otras áreas de la ingeniería se habla de artefactos y representación de ese artefacto. Por ejemplo, una máquina y su diagrama de conexión eléctrico (ver figura)

8.7 Un lenguaje visual para el software



Si el código mismo representa a un artefacto de *software*, entonces lo queremos es una representación visual del artefacto.



Al igual que como en el caso de describir una casa no basta un solo tipo de diagrama (planta, elevación, instalaciones, etc), en el caso del *software* también se requieren diversos tipos de diagramas. Para ello UML proporciona más de diez tipos distintos de diagramas pero nosotros solo veremos los mas utilizados: diagrama de clases, diagrama de secuencia, diagrama de estados, etc.

8.8 El diagrama de clases

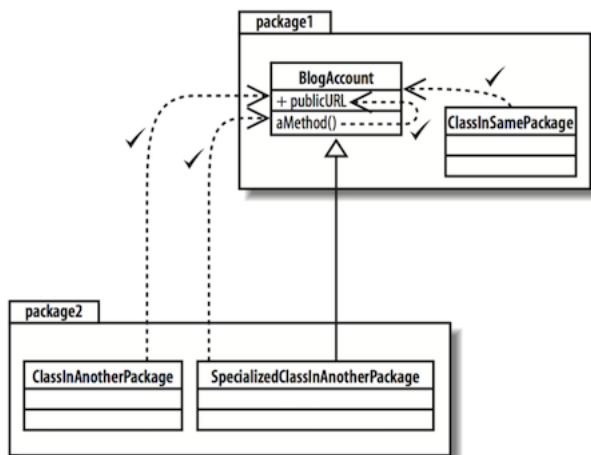
Hay rectángulos que denotan clases y líneas que representan asociaciones o dependencias entre las clases. Cada rectángulo puede tener hasta 3 compartimentos para albergar el nombre de la clase, sus atributos y sus métodos respectivamente (ver figura) clases. Las líneas representan asociaciones y pueden terminar con un adorno especial en un extremo (rombo, triángulo) que indica un tipo de asociación especial (ver figura) líneas.

8.9 Visibilidad

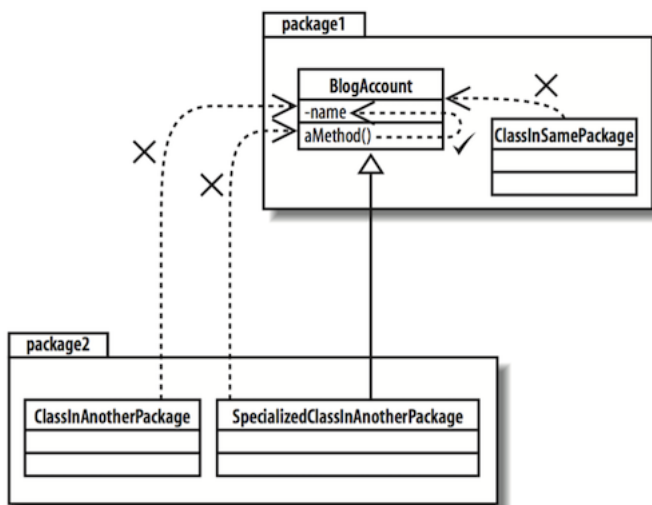
Los nombres de los atributos y los métodos de una clase pueden aparecer precedidos por un caracter especial: +, -, # y ~. Estos decoradores están asociados a lo que se conoce como la visibilidad del atributo o del método. La visibilidad indica desde donde está permitido acceder al elemento y desde donde no está permitido y ello es importante para asegurar que modificaciones posteriores no afecten a todo el código.

Un atributo siempre es accesible a los métodos de la misma clase al que pertenece, pero por lo general no es accesible a métodos de otras clases. Esta situación se indica como que el atributo es privado y se marca con un caracter "-". Si por alguna razón queremos que el atributo sea accesible desde todos lados (público) se indica con un caracter "+". Por lo general los métodos de una clase se hacen públicos, pero no necesariamente todos los métodos lo son. A veces queremos que el atributo sea privado con la excepción de las subclases (hijos) de la clase donde está definido. En ese caso se habla de una visibilidad protegida y se indica con un "#". Finalmente, es posible indicar un acceso libre a todas las clases que están encerradas en un mismo paquete y eso se indica con un caracter "~".

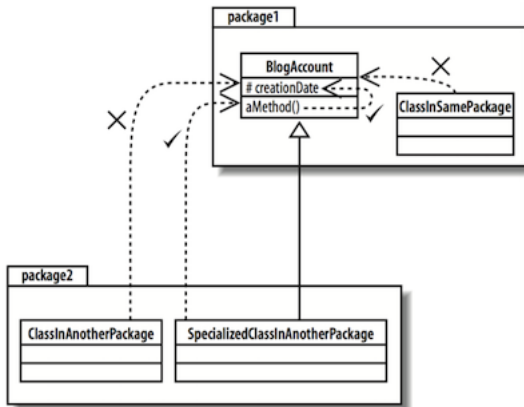
La figura siguiente ilustra un atributo público (+ publicURL) que puede ser accesado desde los métodos de la misma clase (aMethod), desde otra clase en el mismo paquete o desde cualquier clase en otro paquete.



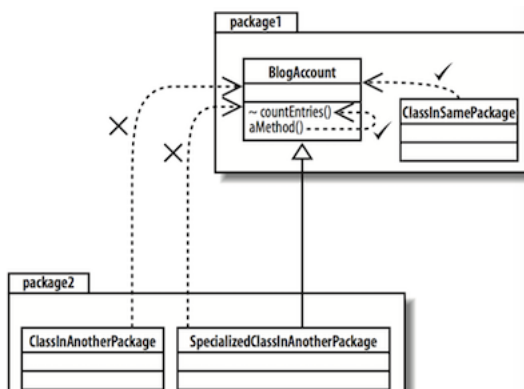
La siguiente figura muestra que sucede cuando un atributo (name) es declarado privado. Solo puede ser accedido por métodos de la misma clase.



En el caso de la visibilidad protegida (atributo #creationDate), además de los métodos de la misma clase, puede ser accedido desde métodos de sub-clases de BlogAccount como en este caso SpecializedClassInAnotherPackage, pero no desde otras clases, ni siquiera estando en el mismo paquete.



Finalmente, la figura siguiente muestra lo que sucede cuando a un elemento (en este caso el método `countEntries`) se le asigna visibilidad de paquete. En este caso, aparte de los métodos en la misma clase, sólo los métodos de clases en el mismo paquete pueden acceder a él.

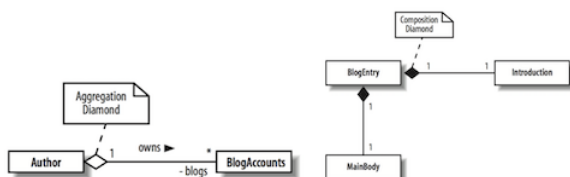


8.10 Asociaciones

Las clases de un modelo no son independientes. Para llevar a cabo una operación puede ser necesario que cooperen dos o más clases. La dependencia entre clases se indica mediante líneas que conectan las clases asociadas. Hay algunos tipos de asociación especiales que se indican con un símbolo en uno de los extremos de la línea que conecta las clases. Un rombo relleno al comienzo de la línea que conecta la clase A con la clase B denota que un objeto de la clase A contiene uno o más objetos de la clase B (composición). Si el rombo está vacío, la clase A contiene referencias a objetos de la clase B (agregación). Un triángulo se usa para modelar una relación de especialización (clase - subclase).

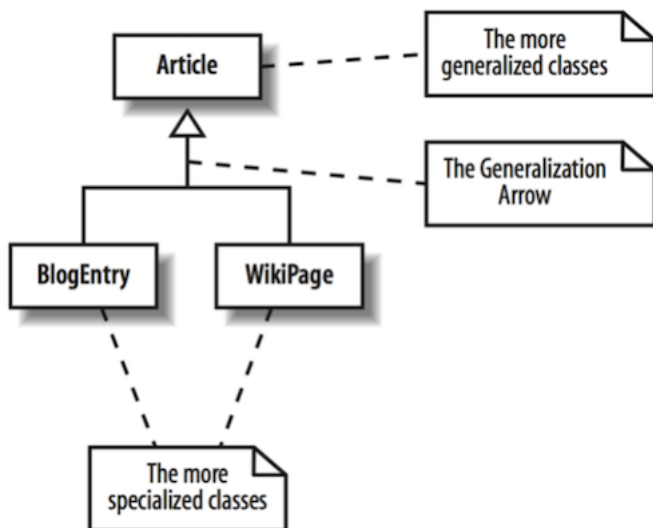
Las figuras muestran ejemplos de asociaciones de asociaciones de agregación

y composición.

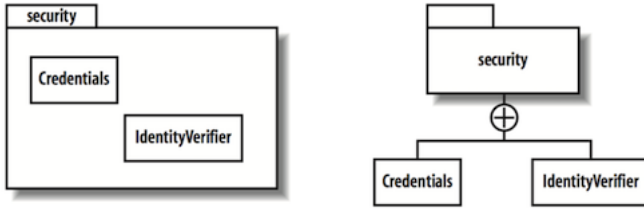


En el diagrama de la izquierda un autor tiene referencias a una o más cuentas. En el diagrama de la izquierda una entrada en el blog está compuesta de una introducción y un cuerpo. Tiene propiedad exclusiva de estos objetos y ellos no tienen sentido de existir fuera del objeto que los contiene.

La siguiente figura muestra un ejemplo de una asociación de generalización. Una entrada en un blog o una página wiki son casos especiales de un artículo. Esta es una relación de tipo-subtipo que a veces se suele llamar (equivocadamente en mi opinión) relación de herencia.

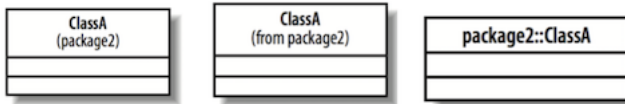


Para mostrar agrupaciones de clases en paquetes se usan carpetas. Debido a que los paquetes pueden contener otros paquetes, hay dos formas alternativas de mostrar en los diagramas (ver figura).



En el ejemplo, las clases `Credentials` y `IdentityVerifier` están dentro del paquete `security`.

Si uno quiere que en una clase (A) se haga explícito que pertenece a un paquete determinado (package 2), puede usarse una de las siguientes 3 formas:

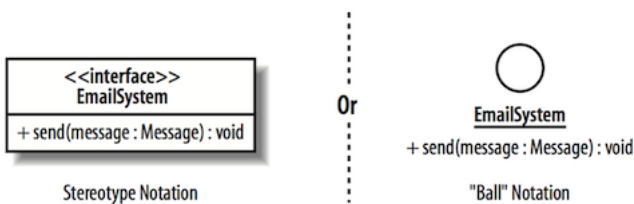


8.11 Interfaces

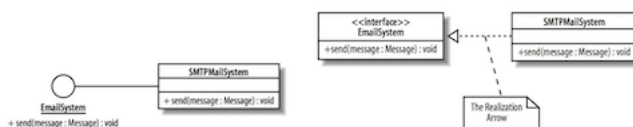
Una interfaz puede ser pensada como un contrato de disponibilidad de ciertas operaciones. Una interfaz define cierta funcionalidad en forma de métodos, pero no los detalles de la implementación. Una clase puede implementar una determinada interfaz, y en este caso está obligada a suministrar la implementación de cada uno de los métodos definidos en ella. Puede haber más de una clase que implementa la misma interfaz y también puede haber una clase que implementa más de una interfaz.

En algunos lenguajes de programación no hay soporte de interfaces (por ejemplo en Ruby) y se usa una clase abstracta para simular una interfaz.

En UML hay dos formas de mostrar una interfaz: igual que una clase, pero con estereotipo *interface* («*interface*») o simplemente como un círculo (ver figura).



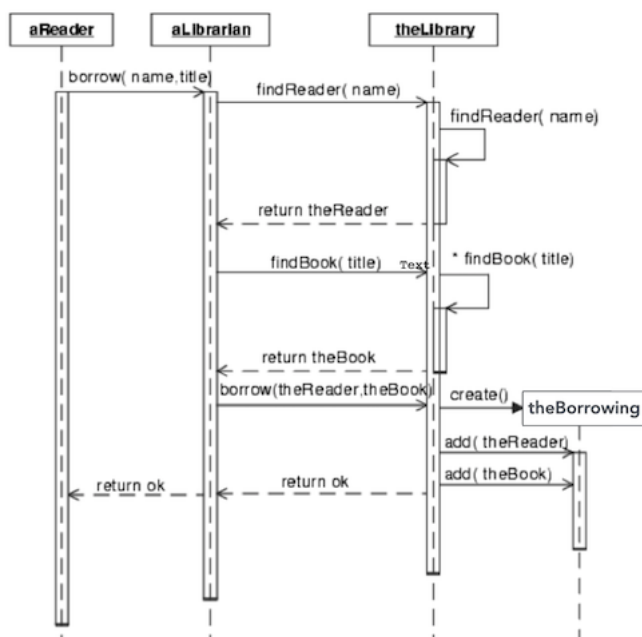
La manera como se muestra que una clase implementa una interfaz depende de cuál de las dos notaciones de interfaz se escogió



8.12 El diagrama de secuencia

Este tipo de diagrama UML es tal vez el segundo más utilizado después del diagrama de clases. Su utilidad radica en que nos permite entender la dinámica de la interacción de los objetos en el tiempo.

En este diagrama se muestra el intercambio de mensajes (flechas horizontales) entre un grupo seleccionado de objetos. El tiempo transcurre hacia abajo, por lo que una flecha más alta que otra indica que ello sucede con anterioridad. Veamos un ejemplo:



La figura muestra como se lleva a cabo la acción de crear un nuevo préstamo. Inicialmente, participan 3 objetos: uno de clase Reader (`aReader`), uno de clase Librarian (`aLibrarian`) y uno de clase Library (`theLibrary`). Inicialmente `aReader` envía el mensaje `borrow` a objeto `aLibrarian` pasando como parámetros el nombre del lector y el nombre del libro que le interesa. Un breve tiempo después (ver que la flecha parte más abajo) el objeto `aLibrarian` invoca el método `findReader` sobre el objeto `theLibrary`, pasando el

nombre del lector. El objeto `theLibrary`, después de buscar al lector, devuelve a `aLibrarian` el objeto que corresponde a ese lector (puede incluir muchos atributos). El objeto `Librarian` ahora invoca el método `findBook` sobre `theLibrary` y esta vez lo que retorna es el objeto que corresponde al libro pedido. Finalmente, el `Librarian` procede a enviar el mensaje para crear un nuevo préstamo sobre (`theBorrowing`) pasando como parámetros los objetos que corresponden al lector y al libro. El objeto `theLibrary` crea el objeto y agrega a él los datos del lector y del libro. Una vez que `aLibrarian` ha concluido su trabajo, retorna a `aReader` y la operación concluye.

Algunas observaciones importantes:

- normalmente, no se incluyen flechas para los retornos, salvo que se retornen objetos
- los rectángulos angostos sobre las líneas de tiempo denotan el período de tiempo en que el objeto está activo
- pueden haber invocaciones de métodos de un objeto sobre sí mismo

Patrones de Diseño

A fines de los 70, el arquitecto Christopher Alexander publica dos muy influyentes libros: *A Pattern Language* y *The Timeless Way of Building*. En estos libros Alexander cristaliza su idea de que en la arquitectura hay elementos de calidad objetivos, lo que se traduce en que ciertas soluciones se repiten una y otra vez. No exactamente iguales, pero sí la idea de la solución. Él acuña entonces el término de patrón como la solución de un problema en un contexto dado.

Las ideas de Alexander sobre patrones son tomadas por un grupo de muy influyentes personas en el ámbito del diseño de software y se dan cuenta de que son completamente válidas en el terreno del software. Cuando el objetivo buscado es lograr soluciones que puedan minimizar el acoplamiento manteniendo una adecuada cohesión, de modo que el software pueda ser extendido y modificado con facilidad, las soluciones que aparecen son siempre las mismas. Son soluciones de problemas en contexto, es decir, son patrones, y en este caso se habla de *software design patterns*.

Estos influyentes desarrolladores mencionados eran Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, a quienes se les conoce como la banda de los cuatro o en inglés *Gang of Four* (GOF), quienes se concentran en la tarea de identificar un primer grupo de patrones de diseño lo que da frutos en forma del fundamental libro *Design Patterns*, que se transforma rápidamente en “la biblia” del diseño de *software* con más de medio millón de copias vendidas y traducciones a 13 lenguajes diferentes. La gente suele referirse a este libro como el GOF por sus autores.

Asombrosamente, estos desarrolladores encuentran solo 23 patrones de diseño (ellos pensaban que habrían cientos) que son los que se describen en este libro. Cada uno de los patrones se presenta y describe en forma detallada, incluyendo su nombre (es muy importante), el propósito, el cómo logra ese

propósito y las restricciones y fuerzas involucradas en el problema y su solución. Los patrones se agrupan en 3 grandes grupos: patrones estructurales, patrones creacionales y patrones de comportamiento.

1. Abstract factory groups object factories that have a common theme.
2. Builder constructs complex objects by separating construction and representation.
3. Factory method creates objects without specifying the exact class to create.
4. Prototype creates objects by cloning an existing object.
5. Singleton restricts object creation for a class to only one instance.
6. Adapter allows classes with incompatible interfaces to work together by wrapping its own interface around that of an already existing class.
7. Bridge decouples an abstraction from its implementation so that the two can vary independently.
8. Composite composes zero-or-more similar objects so that they can be manipulated as one object.
9. Decorator dynamically adds/overrides behaviour in an existing method of an object.
10. Facade provides a simplified interface to a large body of code.
11. Flyweight reduces the cost of creating and manipulating a large number of similar objects.
12. Proxy provides a placeholder for another object to control access, reduce cost, and reduce complexity.
13. Chain of responsibility delegates commands to a chain of processing objects.
14. Command creates objects which encapsulate actions and parameters.
15. Interpreter implements a specialized language.
16. Iterator accesses the elements of an object sequentially without exposing its underlying representation.
17. Mediator allows loose coupling between classes by being the only class that has detailed knowledge of their methods.
18. Memento provides the ability to restore an object to its previous state (undo).
19. Observer is a publish/subscribe pattern which allows a number of observer objects to see an event.
20. State allows an object to alter its behavior when its internal state changes.
21. Strategy allows one of a family of algorithms to be selected on-the-fly at runtime.
22. Template method defines the skeleton of an algorithm as an abstract class, allowing its subclasses to provide concrete behavior.
23. Visitor separates an algorithm from an object structure by moving the hierarchy of methods into one object.

Nosotros no estudiaremos cada uno de ellos, sino solo cerca de la mitad (los

más frecuentes).

9.1 ¿Por qué vale la pena estudiar estos patrones?

La principal razón es poder reutilizar soluciones a problemas que han sido encontradas por otras personas a través de su experiencia. En lugar de dar con la solución después de muchos intentos de prueba y error, uno puede ir directamente a ella. Es importante contrastar la reutilización de una solución con la reutilización de una pieza de *software*. La ventaja de reutilizar *software* es que no solo está la solución, sino también la implementación de la solución lista para el uso. La desventaja es que muchas veces no nos sirve porque es otro lenguaje de programación, un contexto ligeramente distinto, etc. La ventaja de reutilizar la solución es que es casi siempre aplicable, puesto que en la implementación se incorporan los detalles.

Una segunda razón para estudiarlos es que permiten establecer una terminología común que hace el lenguaje mucho más preciso. Una terminología común favorece mucho las comunicaciones al interior del equipo. Observa el siguiente ejemplo:

Para los algoritmos criptográficos, afortunadamente, teníamos ya una clase que alguien había desarrollado. Por supuesto que no tenía la interfaz correcta, pero lo solucionamos rápidamente con un adaptador. Para facilitar a la empresa externa, el conectarse con nuestro producto, simplemente le proporcionamos una fachada de modo de que pueda tener una visión simplificada sin tener que entender todos los detalles de la aplicación. Más complejo resultó la implementación del esquema de “undo’s” ilimitados que el cliente insistió en tener, finalmente optamos por usar para ello un comando y así poder manejar las operaciones como si fuesen objetos. Evidentemente que el repositorio de datos se manejó como una instancia única, para lo cual el *singleton* fue de gran ayuda.

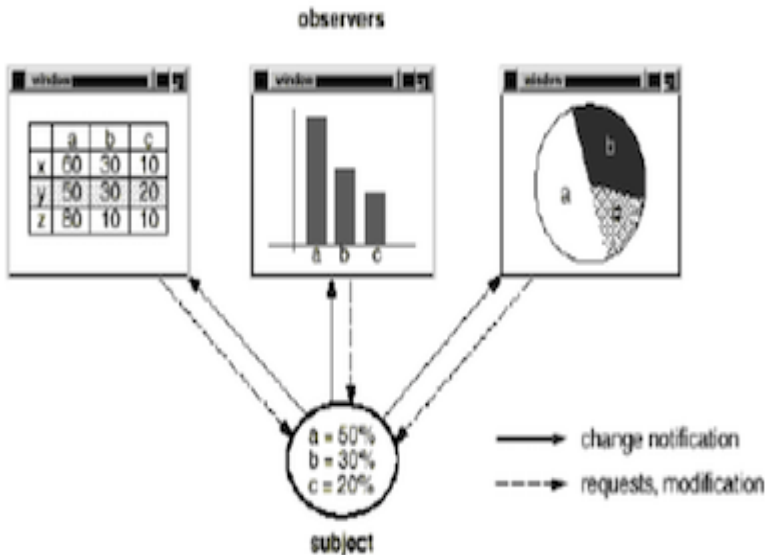
Otra razón para estudiar patrones de diseño es que se pueden aprender estrategias generales que uno puede utilizar en sus diseños, como por ejemplo:

- diseñar siempre para las interfaces
- favorecer la agregación por sobre la herencia cuando sea posible
- identificar lo que más cambia y encapsularlo

9.2 El Patrón Observador (*Observer*)

Es muy frecuente encontrarse con el siguiente escenario: hay un objeto (sujeto u observado) que tiene un estado del cual varios otros objetos (observadores) quieren estar pendientes para reaccionar cuando se produzcan cambios.

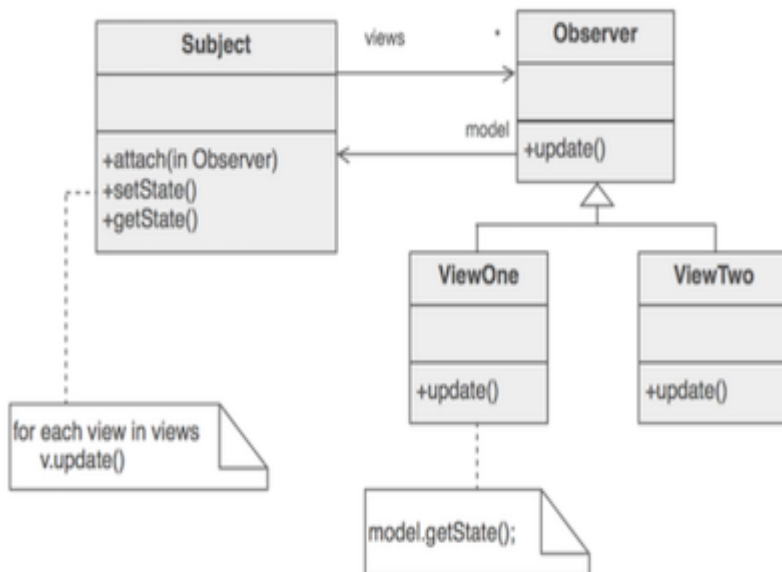
En el ejemplo de la figura, el sujeto es un objeto que tiene un estado representado por el valor de sus atributos a, b y c, y hay 3 observadores que corresponden a representaciones gráficas del estado de ese objeto. Lo que se desea es que cuando cambie alguno de los valores a, b o c ello se vea inmediatamente representado en los objetos observadores.



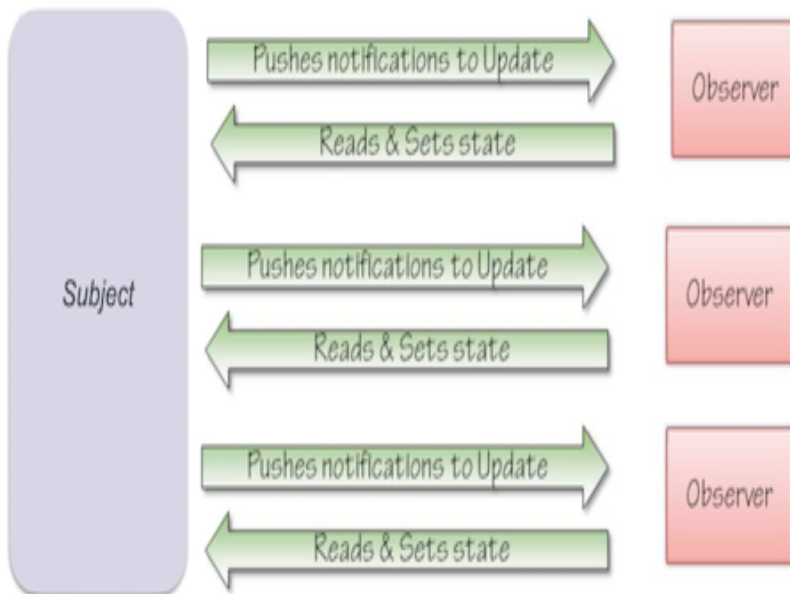
Este patrón se encuentra muy frecuentemente en productos de *software*. Por ejemplo, la relación entre un modelo y las vistas en que el modelo es el sujeto y las vistas son observadores.

El diagrama de clases UML siguiente muestra la solución. El sujeto debe proveer un método para que un objeto observador pueda registrarse o suscribirse (*attach*) y métodos que permitan obtener su estado (*getState*). El sujeto debe enviar un *update* a todos los objetos registrados como observadores cada vez que cambia su estado. Para ello, los objetos observadores deben implementar un método *update*. Este método a su vez lo primero que hará es obtener el estado actual desde el sujeto (*getState*) para proceder a actualizar.

9.2 El Patrón Observador (Observer)



A veces los observadores también son responsables de cambiar el estado (`SetState`) del sujeto, por ejemplo porque registraron un evento (un `click`) en la representación asociada a este objeto.



El uso de este patrón no está supeditada solo a escenarios con objetos gráficos. A continuación se muestra un ejemplo en que el sujeto es un objeto de la clase `Empleado` y la parte de su estado que nos interesa monitorear es el

suelo. Los objetos interesados en mantenerse informados de cambios en el sueldo del empleado son dos: uno de la clase `Payroll` (planilla de sueldos) y uno de la clase `Taxman` (responsable de los impuestos).

Nota que el objeto de la clase empleado se inicializa con la información de nombre, título, sueldo y además un arreglo de objetos observadores vacío. Se provee de métodos para registrar (`add_observer`) y para eliminar (`delete_observer`) a un observador y de un método que se encarga de notificar a todos los observadores (`notify_observers`) cuando hay cambios en el sueldo. Este método es muy simple y se limita a recorrer el arreglo de observadores del objeto empleado y enviar un `update` a cada uno de ellos. Por cierto tanto `Payroll` como `Taxman` implementan un método `update`. El método `notify_observers` es llamado cada vez que ejecuta el método que permite cambiar el sueldo (`salary=`)

```
class Employee
  def initialize( name, title, salary )
    @name = name
    @title = title
    @salary = salary
    @observers = []
  end
  def salary=(new_salary)
    @salary = new_salary
    notify_observers
  end

  def notify_observers
    @observers.each do |observer|
      observer.update(self)
    end
  end
  def add_observer(observer)
    @observers << observer
  end
  def delete_observer(observer)
    @observers.delete(observer)
  end
end

class Payroll
  def update( changed_employee )
    puts("Cut a new check for #{changed_employee.name}!")
    puts("His salary is now #{changed_employee.salary}!")
  end
end

class TaxMan
  def update( changed_employee )
    puts("Send #{changed_employee.name} a new tax bill!")
  end
end

fred = Employee.new('Fred', 'Crane Operator', 30000.0)

payroll = Payroll.new
fred.add_observer( payroll )

tax_man = TaxMan.new
fred.add_observer(tax_man)

fred.salary=80000

Cut a new check for Fred!
His salary is now 80000.0!
Send Fred a new tax bill!
```

9.3 El Patrón Método Plantilla (*Template Method*)

Muchas veces uno se enfrenta al siguiente escenario: hay una secuencia de pasos que siempre se repite, pero el detalle de cómo exactamente se hace cada paso puede cambiar. Por ejemplo, si quiero generar un reporte, a lo mejor voy a tener un encabezado, luego un preámbulo, luego el desarrollo y finalmente las conclusiones y el cierre. La idea de este patrón es encapsular toda la secuencia de operaciones como un método con operaciones abstrac-

tas y dejar que más tarde se puedan detallar las operaciones concretas que nos interesan.

A continuación se muestra una clase `Report` que incluye un método plantilla llamado `output_report` (marcado con otro color). Cada una de las operaciones que aparecen en la secuencia, excepto por `output_body`, corresponden a métodos abstractos (se espera que se implementen en subclases de `Report`).

```
class Report
  def initialize
    @title = 'Monthly Report'
    @text = ['Things are going', 'really, really well.']
  end

  def output_report
    output_start
    output_head
    output_body_start
    output_body
    output_body_end
    output_end
  end

  def output_body
    @text.each do |line|
      output_line(line)
    end
  end

  def output_start
    raise 'Called abstract method: output_start'
  end

  def output_head
    raise 'Called abstract method: output_head'
  end

  def output_body_start
    raise 'Called abstract method: output_body_start'
  end

  def output_line(line)
    raise 'Called abstract method: output_line'
  end

  def output_body_end
    raise 'Called abstract method: output_body_end'
  end

  def output_end
    raise 'Called abstract method: output_end'
  end
end
```

Si quiero un informe en formato HTML, puedo escribir una subclase de Report (HTMLReport) que tiene que proveer de implementaciones especiales de los métodos abstractos que son llamados en el método plantilla de Report. Si quiero un reporte en texto plano, escribo otra clase PlainTextReport y escribo otras versiones concretas de esos métodos abstractos.

```
class HTMLReport < Report
  def output_start
    puts('<html>')
  end
  def output_head
    puts('  <head>')
    puts("    <title>#{@title}</title>")
    puts('  </head>')
  end

  def output_body_start
    puts('<body>')
  end

  def output_line(line)
    puts("  <p>#{line}</p>")
  end

  def output_body_end
    puts('</body>')
  end

  def output_end
    puts('</html>')
  end
end

class PlainTextReport < Report
  def output_start
    end
  def output_head
    puts("**** #{@title} ****")
    puts
  end

  def output_body_start
    end

  def output_line(line)
    puts(line)
  end

  def output_body_end
    end

  def output_end
    end
end
```

Generar una versión distinta de reporte es ahora muy fácil ...

```
report = HTMLReport.new
report.output_report
```



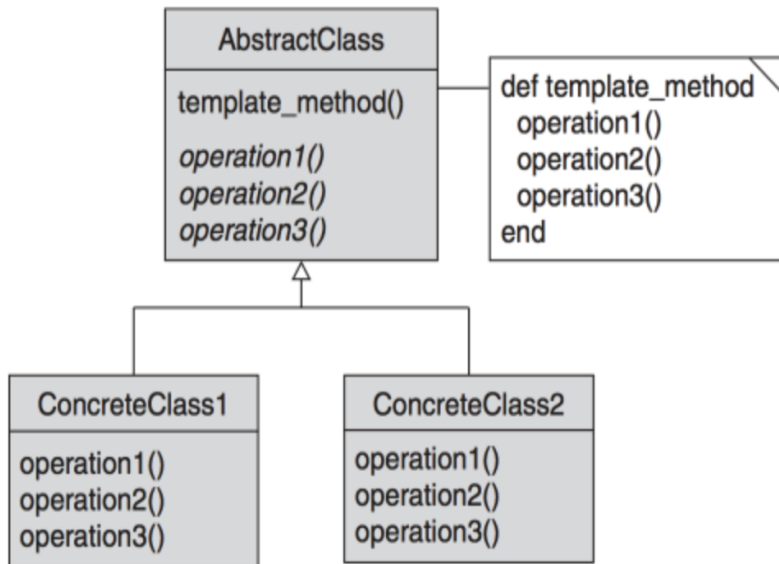
```
<html>
<head>
  <title> Montly Report </title>
</head>
<body>
  <p> Things are going </p>
  <p> really, really well </p>
</body>
</html>
```

```
report = PlainTextReport.new
report.output_report
```



```
**** Montly Report ****
Things are going
really, really well
```

La estructura del patrón descrita como diagrama UML se muestra en la siguiente figura. Se tiene un algoritmo/proceso/procedimiento que comprende una secuencia de pasos, lo que es capturado en un método plantilla con la secuencia conocida. Las operaciones concretas se especifican en las subclases de la clase abstracta donde vive el método plantilla.



Los métodos correspondientes a estas operaciones del método plantilla no

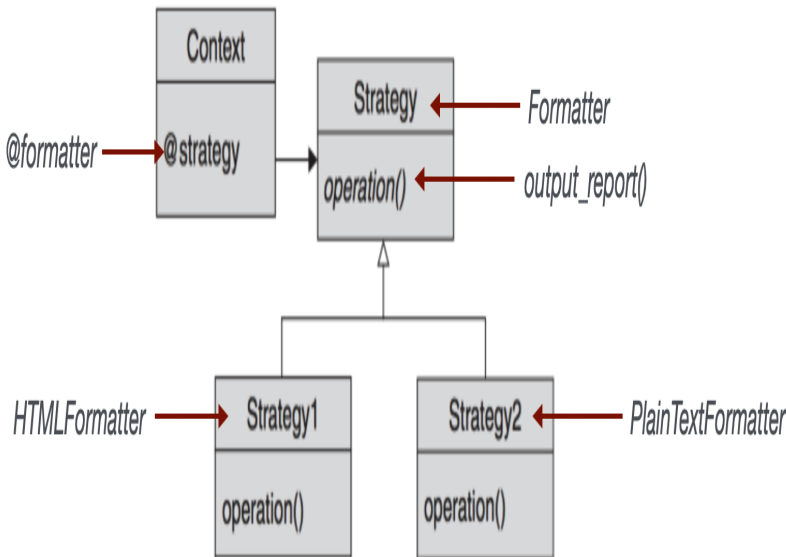
tienen por qué ser necesariamente abstractos. En el ejemplo del reporte podríamos haber dejado métodos que no hacen nada como *default* y así el `PlainTextReport` no hubiese tenido que redefinirlos.

9.4 El Patrón Estrategia (*Strategy*)

Este patrón resuelve un problema parecido al del método plantilla en el sentido de querer encapsular un algoritmo, pero lo hace de una manera completamente distinta. La idea es que ese algoritmo, en lugar de encapsularlo como un método en una clase, lo vamos a encapsular como un objeto que puede ser invocado desde la clase que quiere utilizarlo. En el GOF se hace la distinción en cuanto a que esta solución utiliza delegación en lugar de utilizar herencia.

En el contexto del ejemplo del reporte, lo que haremos será construir clases específicas con la manera en que queremos hacer el reporte. La clase `Reporte` mantendrá una referencia a un objeto estrategia el cual podrá invocar cuando lo necesite.

El diagrama UML siguiente muestra lo esencial de patrón estrategia. La clase `Context` sería nuestra clase `Report` y el atributo `strategy` apunta al objeto que utilizará para emitir el reporte. Puede haber una estrategia abstracta con especializaciones.



La implementación y uso de este patrón en el problema del reporte se muestra a continuación:

9.4 El Patrón Estrategia (*Strategy*)

```
class Report
  attr_reader :title, :text
  attr_accessor :formatter

  def initialize(formatter)
    @title = 'Monthly Report'
    @text = [ 'Things are going', 'really, really well.' ]
    @formatter = formatter
  end

  def output_report
    @formatter.output_report( @title, @text )
  end
end

class HTMLFormatter
  def output_report( title, text )
    puts('<html>')
    puts('  <head>')
    puts("    <title>#{title}</title>")
    puts('  </head>')
    puts('  <body>')
    text.each do |line|
      puts("    <p>#{line}</p>" )
    end
    puts('  </body>')
    puts('</html>')
  end
end

class PlainTextFormatter
  def output_report(title, text)
    puts("***** #{title} *****")
    text.each do |line|
      puts(line)
    end
  end
end
```

```
report = Report.new(HTMLFormatter.new)
report.output_report
```

```
report = Report.new(PlainTextFormatter)
report.output_report
```

Para terminar con este importante patrón pueden estudiar este nuevo ejemplo en que lo que se trata de encapsular como estrategia es el algoritmo de ordenamiento que se desea utilizar. La clase `Sorter` es inicializada con la estrategia deseada (un objeto de la clase `QuickSort` o un objeto de la clase `MergeSort`) y luego implementar el método `sort` simplemente invocando el `sort` sobre ese objeto (`theStrategy.sort`). Esa es la idea de delegación.

Una sutileza en este ejemplo es que existe una estrategia por defecto. Al inicializar el `Sorter`, si no se le pasa un objeto estrategia, utiliza un `Quicksort`.

Observa que la clase `Sorter` tiene un atributo de clase que es la estrategia por defecto y que es inicializada con un objeto `Quicksort`. El método de inicialización de instancias de `Sorter` asigna al atributo `theStrategy` el objeto estrategia que se le pase o, en caso que no venga, el objeto estrategia guardado como atributo de clase.

```

class QuickSort
  def sort arr
    ...
  end
end

class MergeSort
  def sort array
    ...
  end

  def merge left, right
    ...
  end
end

class Sorter
  @@default_strategy = Quicksort.new
  def initialize(strategy)
    @theStrategy = strategy || @@default_strategy
  end
  def sort arr
    thestrategy.sort(arr)
  end
end

```

Ahora podemos hacer :

```

arr = [3 5 2 6 8 9 3 5 8 2 ]
aSort = Sorter.new(MergeSort.new)
asort.sort arr
aSort = Sorter.new
asort.sort arr

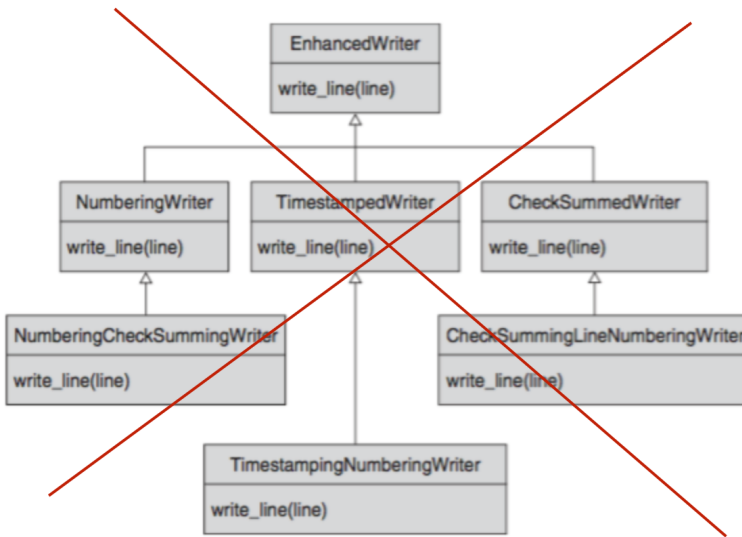
```

9.5 El Patrón Decorador (*Decorator*)

El problema que resuelve este patrón es el poder anexar responsabilidades en forma dinámica a un objeto. Digamos que yo tengo un personaje que sabe correr. Yo quiero que antes de comenzar a correr se ponga las zapatillas y al terminar se saque las zapatillas. O tengo una ventana de un programa que sabe dibujarse, pero quiero agregar el que antes de dibujar la ventana dibuje una barra de título y después de dibujar la ventana las barras de *scroll*.

La tendencia natural es pensar en mecanismos de herencia para agregar esta funcionalidad extra a un método, pero rápidamente nos daremos cuenta de que la idea no es buena. Hay dos problemas: primero que no nos permitiría hacer los agregados en forma dinámica mientras el programa está corriendo (aunque hay lenguajes de programación que si lo permiten) y segundo que no puedo combinar los elementos agregados con la flexibilidad necesaria.

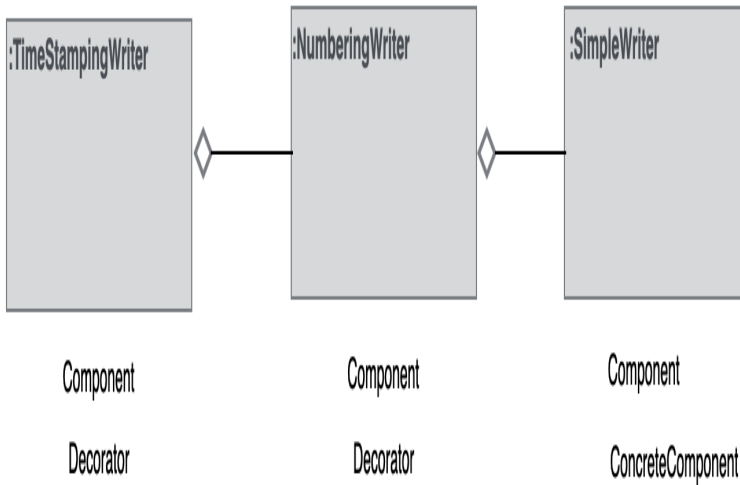
Imagina que tenemos una clase que llamaremos EnhancedWriter que es capaz de escribir una línea de texto y que lo especializamos en tres clases: NumberingWriter que le agrega un número de línea al comienzo de cada línea, un TimestampingWriter que le agrega un timestamp (fecha y hora actual) y un ChecksummingWriter que agrega un *checksum*. Perfecto, pero que pasa si quiero combinar y tener un texto numerado con *timestamps* u otras combinaciones necesitare crear nuevas clases (ver figura).



La siguiente figura muestra la esencia del patrón decorador como diagrama UML de clases. Es importante notar lo siguiente:

- Tanto la clase Decorator como ConcreteComponente son especializaciones de Component
- La clase Decorator incluye una referencia a un objeto Component (puede ser Decorator o ConcreteComponent)

El que un objeto Decorator pueda tener una referencia a otro objeto Decorator y así sucesivamente hasta que el último Decorator tiene una referencia a un ConcreteComponent permite producir lo que estamos buscando. El ejemplo de la figura muestra un objeto TimeStampingWriter que apunta a un decorador NumeringWriter quien finalmente apunta al SimpleWriter.



La figura siguiente muestra los detalles de implementación y la forma de uso para producir un writer con uno y dos decoradores

```

class QuickSort
  def sort arr
    ...
  end
end

class MergeSort
  def sort array
    ...
  end

  def merge left,right
    ...
  end
end

class Sorter
  @@default_strategy = Quicksort.new
  def initialize(strategy)
    @theStrategy = strategy || @@default_strategy
  end
  def sort arr
    thestrategy.sort(arr)
  end
end
    
```

Ahora podemos hacer :

```

arr = [3 5 2 6 8 9 3 5 8 2 ]
aSort = Sorter.new(MergeSort.new)
aSort.sort arr
aSort = Sorter.new
aSort.sort arr
    
```

El diagrama de secuencia UML ilustra la secuencia de mensajes entre los objetos involucrados para el segundo caso (con dos decoradores)

```

class QuickSort
  def sort arr
    ...
  end
end

class MergeSort
  def sort array
    ...
  end

  def merge left, right
    ...
  end
end

class Sorter
  @@default_strategy = Quicksort.new
  def initialize(strategy)
    @theStrategy = strategy || @@default_strategy
  end
  def sort arr
    thestrategy.sort(arr)
  end
end

```

Ahora podemos hacer :

```

arr = [3 5 2 6 8 9 3 5 8 2 ]
aSort = Sorter.new(MergeSort.new)
asort.sort arr
aSort = Sorter.new
asort.sort arr

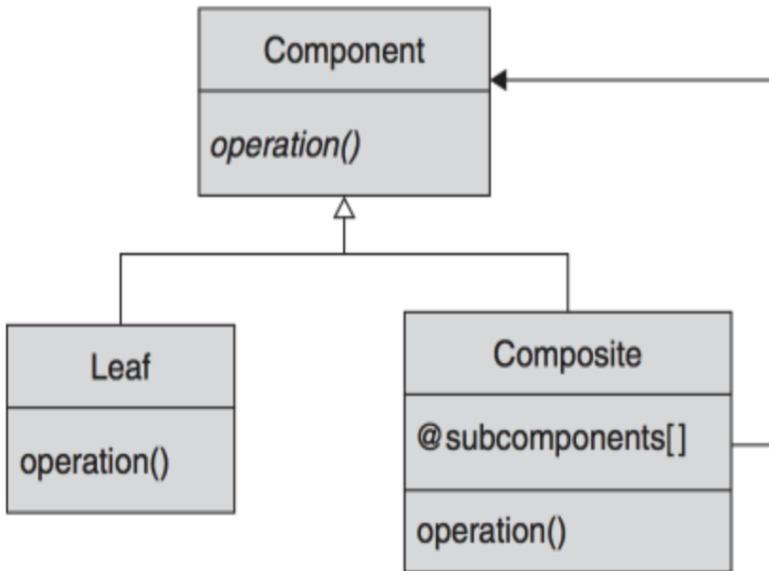
```

9.6 El Patrón *Composite*

Seguramente estás familiarizado con la estructura de árbol, estructura que es muy utilizada en Computación. Una de las cosas más interesantes de esta estructura es su carácter recursivo: un árbol es un nodo con hijos que son también árboles. Esto se puede dar en objetos de dominios muy diversos en que un objeto está compuesto por objetos que son similares al objeto que componen. Por ejemplo, una figura geométrica puede estar compuesta de un triángulo, un rectángulo y un círculo que son también figuras geométricas.

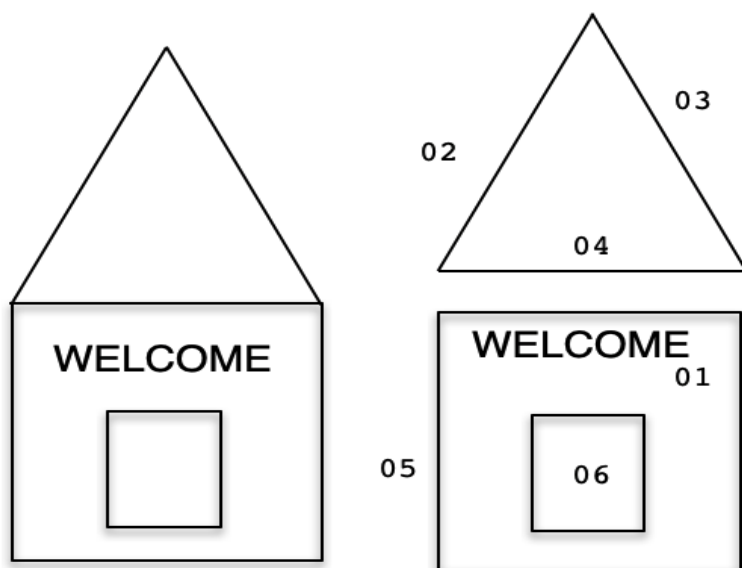
El patrón *composite* aplica siempre que estemos en presencia de objetos que tienen una estructura jerárquica en que los objetos y subobjetos se manejan de forma similar. En el ejemplo de la figura tenemos una casa que está compuesta de puertas y ventanas. Las puertas están compuestas de dos partes menores y las ventanas de 4 partes menores. Entonces, si quiero calcular el precio de la casa, la casa traspasará el mensaje a puertas y ventanas, las que a su vez lo traspasaran a sus constituyentes.

La descripción del patrón usando un diagrama de clases se muestra en el siguiente diagrama de clases UML.

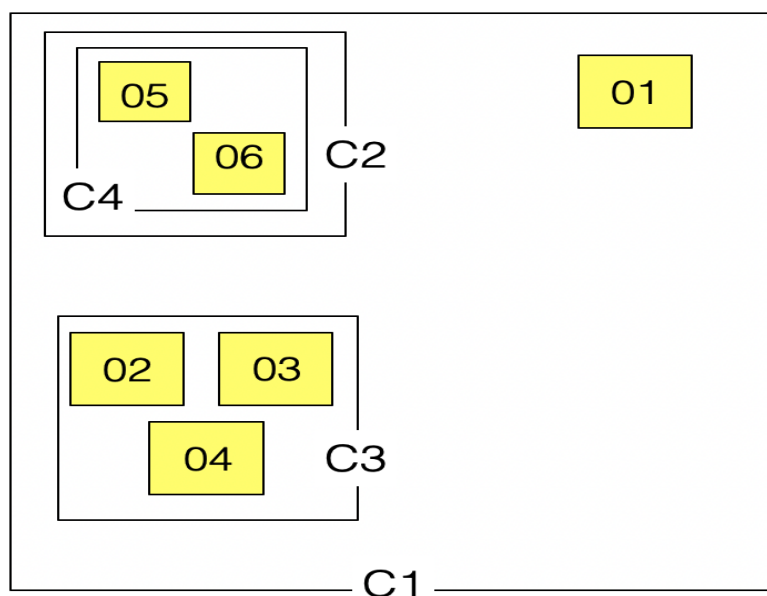


Un objeto `Component` puede ser especializado en dos formas: una `Leaf` o un `Composite`. En el caso que sea un `Composite`, tiene una referencia (*subcomponents*) a una lista de objetos que son a su vez `Component` (`Leaf` o `Composite`). Observa que tanto el objeto `Composite` como el objeto `Leaf` implementan `operation()`, pero en el caso de un `Composite` lo que hará es traspasar esa operación a cada una de las subcomponentes.

Veamos todo esto en acción con el ejemplo de objetos gráficos. Tenemos una figura muy sencilla, como la que se muestra a continuación, y que puede descomponerse en una primera instancia en 3 partes: la parte de arriba, la parte de abajo y el letrero “welcome”. Pero la parte de arriba está a su vez compuesta por 3 líneas u la parte de abajo por dos rectángulos.

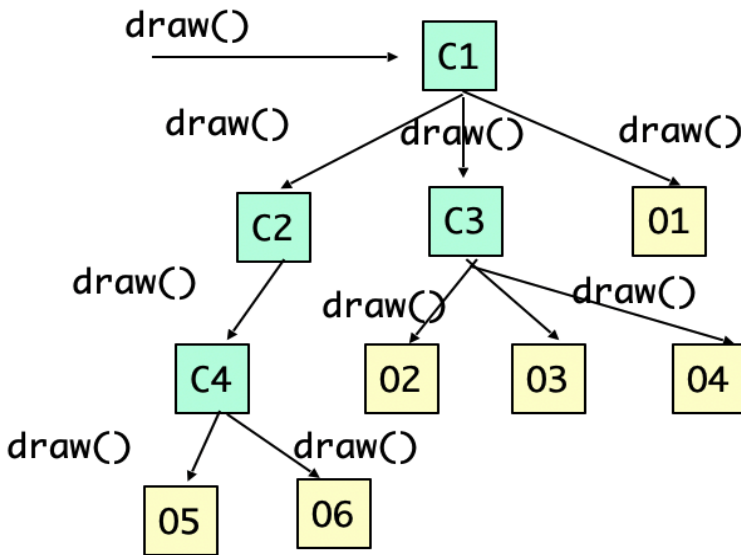


Asignando números a cada una de las partes podemos visualizar la figura en la forma que muestra el diagrama siguiente:



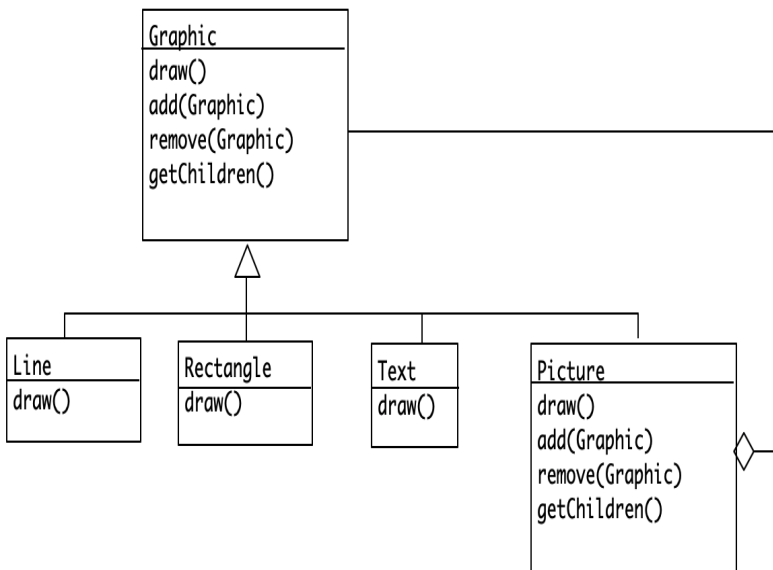
C1 es un Composite que representa a la figura completa y vemos que está compuesta de C3, C2 y 01. De estos 3 objetos, solo 01 es un Leaf. C3 es el Composite que corresponde a la parte de arriba y C2 a la parte de abajo.

En forma de un árbol la situación es como muestra el siguiente diagrama:



El objeto C1 recibe el `draw()` y procede a invocar el `draw` sobre cada uno de sus hijos. C2 y C3, que son *composite*, hacen lo mismo con sus hijos hasta que el método `draw` llega a un objeto *Leaf* que tiene que dibujarse.

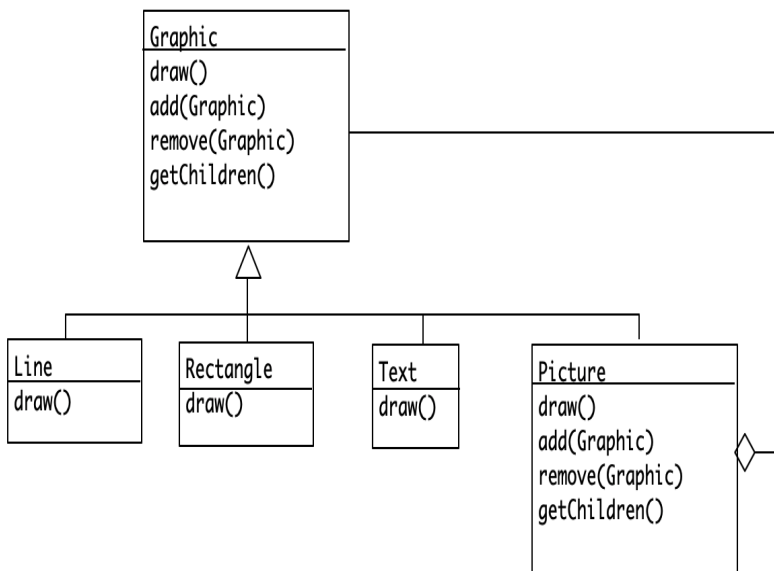
En este caso la estructura de clases podría ser la siguiente:



El siguiente ejemplo muestra un caso en que no hay una larga jerarquía de clases, sino solo dos niveles: la tarea de hacer una torta y las subtarefas de

mezclar ingredientes secos, agregar los líquidos y mezclar. Recordar que aunque parezca como demasiado usar el *composite* en este caso hay que siempre pensar en posibles cambios y extensiones del software. En un caso de recetas complejas, bien podría darse una estructura de 3 o más niveles.

En este caso el método común entre el todo y sus componentes es `get_time_required` que debe entregar el tiempo necesario para la tarea.



Y la implementación ...

```

class Task
  attr_reader :name
  def initialize(name)
    @name = name
  end
  def get_time_required
    0.0
  end
end
class AddDryIngredients < Task
  def initialize
    super('Add dry ingredients')
  end
  def get_time_required
    1.0 # 1 minute to add flour and sugar
  end
end
class AddLiquids < Task
  def initialize
    super('Add liquids')
  end
  def get_time_required
    2.0 # 2 minutes to add milk
  end
end
class Mix < Task
  def initialize
    super('Mix that batter up!')
  end
  def get_time_required
    3.0 # Mix for 3 minutes
  end
end

class MakeBatter < Task
  def initialize
    super('Make batter')
    @sub_tasks = []
    add_sub_task( AddDryIngredients.new )
    add_sub_task( AddLiquids.new )
    add_sub_task( Mix.new )
  end
  def add_sub_task(task)
    @sub_tasks << task
  end
  def remove_sub_task(task)
    @sub_tasks.delete(task)
  end
  def get_time_required
    time=0.0
    @sub_tasks.each {|task| time += task.get_time_required}
    time
  end
end

```

9.7 Para aprender más sobre patrones de diseño

Un sitio web muy bueno que enseña patrones de diseño en ruby es el siguiente:

- Refactoring Guru

Algunos libros sobre patrones

- Design Patterns (GOF) E. Gamma, R. Helm, R. Johnson y J. Vlissides
- Design Patterns Explained A. Shalloway and J. Trott
- Head First Design Patterns E. Freeman and B. Bates

Apendice - Ruby on Rails



Ruby: una revisión rápida

Lo primero que uno aprende en un nuevo lenguaje es imprimir el mensaje “Hello World” en consola. Entonces, empecemos con el primer ejemplo:

```
puts "Hello World!"
```

Uno puede ejecutar esta instrucción directamente en consola:

```
ruby -e 'puts "Hello Ruby!\n"'  
Hello Ruby!
```

Asimismo, uno puede guardarlo en un archivo llamado *hello.rb* y ejecutarlo de la siguiente forma:

```
ruby hello.rb  
Hello Ruby!
```

Para la primera clase del curso utilizaremos la segunda opción.

10.1 Comentarios

Para realizar un comentario de una sola línea se debe utilizar el carácter “#”

```
# Comentario de una línea  
puts "Hello World!"
```

El comentario también puede ir al final de una instrucción

```
puts "Welcome to Ruby!"      # comentario
```

Los comentarios de más de una línea deben ir entre las cadenas “=begin” y “=end”, por ejemplo:

```
=begin
Comentario de multiple lineas
Se puede poner codigo en ruby dentro del comentario
El código dentro de los comentarios no es interpretado
=end
```

10.2 Aspectos Básicos

Durante el curso utilizaremos tipos de datos basicos como por ejemplo: enteros (*integer*), reales (*float*), cadenas (*string*), arreglos (*array*), mapas (*hashes*) y simbolos (*symbol*). En ruby es un lenguaje orientado a objeto donde hasta los datos basicos son objetos, es decir, instancia de una clase. Por ejemplo, los enteros son objetos instanciados a partir de la clase *Number*:

```
123456
-543
```

Las cadenas son definidas a través de comillas simples o comillas dobles. Por ejemplo 'Texto' and "Texto", ambos crea un string. Para imprimir una cadena en consola es posible utilizar la funcion *puts*.

```
puts "Texto"
puts 'Texto'
```

En el ejemplo anterior ruby imprime dos veces la cadena "Texto". La diferencia en que crear una cadena permite (a) utilizar el caracter de escape "" y (b) interpolar la cadena con otros valores. Por ejemplo:

```
puts "texto\n"          # imprime "texto" y crea una nueva linea
puts "suma :#{1+2}"     # imprime "suma: 3"
puts 'suma :#{1+2}'     # imprime "suma :#{1+2}"
```

Para interpolar, se debe introducir la expresión a evaluar entre llaves utilizando el caracter "#" por delante. Ruby, primero evaluara la expresión, posteriormente concatenará el resultado.

Ruby cuenta con operadores aritmeticos y logicos para armar diferentes expresiones:

```
var1 = 20
var2 = 60
var3 = var1 + var2          # evalua a 80
var1 < 25 and var2 > 45    # evalua a verdad
```

Los operadores soportados son muy similares a la mayoría de los lenguajes.

10.3 Variables

Constantes. Las constantes en Ruby normalmente se escriben en puras mayúsculas. Por ejemplo:

```
MYCONSTANT = "hello"
```

En la practica, una “constante” en Ruby puede cambiar de valor (aunque va contra el concepto), pero el interprete de Ruby realizará el cambio de valor pero arrojará una alerta. Por ejemplo, si intentamos asignar un nuevo valor a la constante MYCONSTANT.

```
MYCONSTANT = "hello"
MYCONSTANT = "hello2"
```

Ruby lanzara la siguiente alerta.

```
(irb):34: warning: already initialized constant MYCONSTANT
```

Declarando Variables. Ruby es un lenguaje con sistemas de tipos dinamicos, es decir, que no se especifica el tipo de dato que una variable podra contener. Por lo que la asignación no varia entre diferente tipo de dato.

```
var1 = Person.new
var2 = 230
var3 = "hola"
var2 = Array.new
```

También es posible asigna dos valores de tipo diferente en a una misma variable. Note que esto en la practica no es recomendable ya que puede causar bugs.

10.4 Funciones

A continuación puede ver un ejemplo de como se puede crear una funcion en Ruby. Esta función recibe dos numeros y retorna la suma de los mismos.

```
def sum (n1, n2)
  n1 + n2
end

sum( 3 , 4)           # devuelve 7
sum("cat", "dog")     # devuelve "catdog"
```

En ruby todas las funciones retornan un valor, la regla general, es que retornan el valor resultante de evaluar la ultima expresión dentro de la función. En este ejemplo, la función *sum*, retorna el resultado de evaluar la suma de *n1* y *n2*. Si tu no estas comodo con esta, puedes utilizar el keyword *return* para hacer explicito el valor que retorna la función. Por ejemplo:

```
def multiply(val1, val2 )
  result = val1 * val2
  return result
end

value = multiply( 10, 20 )
puts value
```

Como vimos, en ruby los números y las cadenas también son objetos, por lo que en este ejemplo el `*` en realidad no es un operador es un método. Es más el `*` es un método polimorfo, ya que dependiendo del objeto que recibe el mensaje funcionara de forma diferente. Por ejemplo, si son cadenas el `*` concatenara las cadenas, y si son numeros el `*` los sumara.

A continuación otro ejemplo de función esta vez utilizando interpolación de cadenas.

```
def say_goodnight(name)
  "Good night, #{name}"
end
puts say_goodnight('Ma') # imprime Good night, Ma
```

En ruby los nombres de funciones y métodos pueden estar compuestos de cualquier caracter. Además, para llamar a un método no es necesario mandar los argumentos en parentesis, dicho esto, las siguiente expresiones son equivalentes:

```
puts say_goodnight('Ma') # imprime Good night, Ma
puts say_goodnight 'Ma'  # imprime Good night, Ma
```

Funciones anonimas como argumento. En ruby es posible enviar una función anonmia como argumento al final de la llamada. Considere el siguiente ejemplo:

```
def call_block #función llamada call_block
  yield("hello",2) # ejecutando la función anonima
end
call_block { | s, n | puts s*n, "\n" } # llamando a call_block y enviando argumentos

# imprime hellohello
```

El ejemplo imprime dos veces la palabra hello, así es, el operador `*` también es polimorfo.

10.5 Clases y Objetos

A continuación podrá encontrar un ejemplo donde se crea un clase *BanckAccount* y dos metodos: *initialize* y *test_method*.

```

class BankAccount
  def initialize()
  end

  def test_method
    puts "The class is working"
  end
end

```

El metodo *initialize* es llamado justo después que ruby crea un objeto. Para crear un objeto se debe llamar al método de clase *new* de la siguiente forma:

```
account = BankAccount.new()
```

En el ejemplo anterior se puede notar que en Ruby tampoco es obligatorio utilizar parentesis para los argumentos al momento de declarar un método. Es especialmente útil para los métodos que no tienen argumentos. Y para llamar un método se utiliza el “.” (punto).

```
account.test_method # imprime The class is working
```

Para identificar una variable como variable de instancia se debe utilizar el operador @.

```

class BankAccount
  def initialize (amount)
    @balance = amount
  end

  def deposit(amount)
    @balance = @balance + amount
  end
  def withdraw(amount)
    @balance = @balance - amount
  end
  def print
    puts "balance: #{@balance}"
  end
end

```

```

account = BankAccount.new(1324)
account.deposit(200)
account.withdraw(100)

```

El ejemplo anterior muestra la clase *BankAccount* con un atributo ?. *Note que en Ruby no es necesario hacer explícito al inicio de la clase que atributos tiene. En este ejemplo, amount es una variable local y @balance_ una variable de instancia.*

Visibilidad. Los atributos en ruby por defecto son protegidos, es decir, solo se los puede acceder desde la misma clase o desde las clases hijas. Por otro lado, los métodos tienen visibilidad pública por defecto, es decir, que los métodos pueden ser llamados desde cualquier clase del sistema.

Accesores. Como los atributos son protegidos, para acceder a su valor desde otras clases se puede crear métodos accesores:

```
class BankAccount
  def accountNumber= (number)
    @accountNumber = number
  end
  def accountNumber(number)
    return @accountNumber
  end
  ...
end
account= BankAccount.new(1223)
account.accountNumber= 3
account.accountNumber # => 3
```

En el ejemplo anterior creamos unos accesores, a diferencia de otros lenguajes no utilizamos las palabras *set* o *get*, en particular, para la asignación utilizamos el símbolo `=` en el nombre. Además es importante recordar que en ruby no es obligatorio poner los parentesis para llamar a un método.

El parser de Ruby también es muy flexible que considera las siguientes expresiones equivalentes.

```
account.accountNumber= 3
account.accountNumber = 3
```

10.6 Herencia

Para explicar como se implementa herencia en Ruby utilizaremos en el ejemplo de una clase llama *Song* definida a continuación:

```
class Song
  def initialize(name, artist, duration)
    @name = name
    @artist = artist
    @duration = duration
  end
end
song = Song.new("Bicylops", "Fleck", 260)
```

Primero, mencionar que la clase *Song* hereda de la clase *Object* definida en el core de Ruby, todas las clases en realidad heredan por defecto de esta clase.

Por esta razón cualquier clase creada ya soporta varios métodos, estos métodos son los que hereda de su clase padre *Object*, por ejemplo:

```
song.to_s # que devuelve #<Song:0xe6c>
```

El método *to_s* esta definido en la clase padre e devuelve la posición de memoria del objeto en formato cadena (string). Por lo anterior, si escribo un método con el mismo nombre en la clase *Song* en realidad estaria haciendo sobre-escritura.

```
class Song
  def initialize(name, artist, duration)
    @name = name
    @artist = artist
    @duration = duration
  end
  def to_s
    "Song: #{@name}--#{@artist} (#{@duration})"
  end
end
song = Song.new("Bicylops", "Fleck", 260)
song.to_s
# devuelve "Song: Bicylops--Fleck (260 )"
```

Para heredar explícitamente de otra clase se debe utilizar el operador "<" como se ve a continuación:

```
class KaraokeSong < Song
  def initialize(name, artist, duration, lyrics)
    super(name, artist, duration)
    @lyrics = lyrics
  end
end
song = KaraokeSong.new("My Way", "Sinatra", 225, "And now, the...")
song.to_s
#devuelve "Song: My Way--Sinatra (225)"
```

La clase *KaraokeSong* hereda de la clase *Song* y le agrega un atributo adicional *lyrics*. Note que desde el método *initialize* se llama al constructor de la clase padre para que inicialice los atributos restantes de la clase.

La clase *KaraokeSong* también puede sobre-escribir el método *to_s*, por ejemplo:

```
class KaraokeSong < Song
  # Format as a string by appending lyrics to parent's to_s value.
  def to_s
    super + " [#{@lyrics}]"
  end
end
```

```
end
end
```

La clase anterior sobre escribe *to_s*, este método devolvera el mismo *string* que la clase *Song* pero agregando el atributo *lyrics* al final. Recuerdo que en programación orientada objeto el operador *super* sirve para ejecutar el método de la clase padre. En este caso en particular, *super* llama al método *to_s* de la clase padre y luego concatena un *string* al final.

Para la primera parte del curso no se utiliza mucho sobre-escritura, pero si en la segunda parte del curso. Se recomienda repasar los conceptos de herencia llevados en programación avanzada.

Variables y metodos de clase. Anteriormentes, se vio como crear variables de instancia (atributos) y métodos de instancia. En ruby on rails, a veces es necesario llamar a métodos de clase, por lo que a continuación explicamos como se crean y se utilizan los métodos de clase.

El siguiente ejemplo, crea una variable de clase *plays* que tiene doble arroba al inicio, ademas, existe un método de clase *printPlaysReport* que tiene el key-work "self" al inicio.

```
class Song
  @@total_plays = 0
  def initialize(name, artist, duration)
    @name = name
    @artist = artist
    @duration = duration
    @plays = 0
  end
  def play
    @plays += 1
    @@total_plays += 1
  end
  def printReport
    puts "this song play #{@plays} times"
  end
  def self.printPlaysReport
    puts "all songs play {@@total_plays} times"
  end
end
```

El código anterior tiene dos variables *plays* y *total_plays* la primera contara cuantas veces un objeto en particular reprodujo una canción. Por otro lado, *total_plays* contara cuantas canciones se reprodujeron en total entre todos los objetos *Song*. Por ejemplo:

```
song1 = Song.new
song1.play
```

```

song2 = Song.new
song2.play

song1.printReport
# this song play 1 times
song2.printReport
# this song play 1 times
Song.printPlaysReport
# all songs play 2 times

```

Note que el método `printPlaysReport` se ejecuta sobre la clase `Song`, no así sobre una instancia como `song1` o `song2`. Desde un punto de vista, se podría decir que las variables de clase son variables compartidas entre todas las instancias de la clase.

10.7 Arreglos y Hashes

Los arreglos en ruby son instancias de la clase `Array` y las hashes (o mapas) son instancia de la clase `Hash`. Como en varios lenguajes uno puede crear un arreglo llamando a su constructor a través del operador `new`, o utilizando corchetes.

```

days_of_week = Array.new
days_of_week.empty? # devuelve true

```

En el ejemplo anterior se crea un arreglo vacío (tamaño 0), así mismo, se llama al método `empty?` de la clase `Array`, el mismo que retorna verdad (`true`). Otra cosa a mencionar es que en ruby los nombres de los métodos pueden tener caracteres especiales como `?`.

```

days_of_week = Array.new(7)
#[nil, nil, nil, nil, nil, nil, nil]

```

El ejemplo anterior crea un arreglo de tamaño 7, por defecto, ruby llena los 7 slots con `nil`. Es posible enviar como segundo argumento el valor por defecto con el cual quieres que se inicialicen los slots.

```

days_of_week = Array.new(7, "today")
#[ "today", "today", "today", "today", "today", "today", "today" ]

```

Finalmente, es posible crear el arreglo de forma explícita:

```

days_of_week = Array[ "Mon", "Tues", "Wed", "Thu", "Fri", "Sat", "Sun" ]
#[ "Mon", "Tues", "Wed", "Thu", "Fri", "Sat", "Sun" ]

```

Como un arreglo es una instancia de la clase `Array` tu puedes ejecutar cualquier método de esta clase sobre un arreglo. Por ejemplo:

```

days_of_week.at(0)      # devuelve "Mon"
days_of_week.size      # devuelve 7
days_of_week.empty?    # devuelve false

```

Ademas ruby te ofrece un poco de azucar sintactico para acceder a los elementos del arreglo de forma similar a la que se hace n otros lengaujes.

```

days_of_week = [ "Mon", "Tue", "Wed", "Thu", "Fri" ]
days_of_week[0]      # devuelve "Mon"
days_of_week[1]      # devuelve "Tue"

```

Otro dato curioso es que el + tambien sirve para concatenar arreglos:

```

days1 = ["Mon", "Tue", "Wed"]
days2 = ["Thu", "Fri", "Sat", "Sun"]
days = days1 + days2
# ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

```

Finalmente para modificar el elemento de un arreglo se puede utilizar simplemente el operador =

```

colors = ["red", "green", "blue"]
# ["red", "green", "blue"]
colors[1] = "yellow"      # devuelve "yellow"
colors
# ["red", "yellow", "blue"]

```

Hashes En los arreglos se puede acceder a los valores en base a los indices. Las estructuras *Hash* nos permite almacenar pares de datos key-value y hacer operaciones con estos datos.

```

inst = { "a" => 1, "b" => 2 }
inst["a"] # devuelve 1
inst["c"] # devuelve 2

# al especificar 0 en el constructor, el Hash inicializa los valores por 0
inst = Hash.new(0)
inst["a"] # devuelve 0
inst["a"] += 1
inst["a"] # devuelve 1

```

10.8 Estructuras de Control

If then else. A continuación se muestra un ejemplo de if-then-else en ruby.

```

if count > 10
  puts "Try again"

```

```

elsif tries == 3
  puts "You lose"
else
  puts "Enter a number"
end

```

While. A continuación se muestra un ejemplo de while en ruby.

```

while weight < 100 and num_pallets <= 30
  pallet = next_pallet()
  weight += pallet.weight
  num_pallets += 1
end

```

Iteradores. Los iteradores en realidad son métodos que reciben una función anonima al final, como se vio en un inicio. Por ejemplo:

```

animals = ["ant", "bee", "cat", "dog", "elk"]
animals.each {|animal| puts animal }

```

Each es un método que llama a la función anonima una vez por cada elemento dentro del arreglo. A continuación se muestran varios ejemplos de iteradores utiles en ruby.

```

3.times { print "X " }
1.upto(5) {|i| print i, " " }
99.downto(95) {|i| print i, " " }
50.step(80, 5) {|i| print i, " " }

```

10.9 Convenciones

A continuación algunas convenciones de nombre importantes:

- Variables Locales, en minuscula o con _ de inicio. Ejemplos: name, fish_and_chips, _26.
- Variables globales, empiezan con \$. Ejemplo: \$debug
- Variables de instancia, empiezan con @.
- Variables de clase, empiezan con @@.
- Constante, todo en mayuscula. Ejemplo: SINGLE, PI
- Nombres de clase, mayuscula inicial. Ejemplo: MyClass, FeedPerMile

En algunos casos, algunas companias cuando una variable tiene dos palabras utilizan guion bajo. Las clases con mas de dos palabras, cada palabra debe tener mayuscula inicial. Recuerde que en Ruby es valido (sobre todo en los métodos) utilizar caracteres como ?,! o =.

Nombres de Archivos Normalmente los nombres de los archivos son en pura minuscula y si son mas de dos palabras con un guion bajo para separar las palabras, por ejemplo, hard_drive.rb

10.10 Material Adicional

- Documentación de Ruby: <https://www.ruby-lang.org/es/documentation/>
- Prueba Ruby en tu navegador: <https://try.ruby-lang.org/>
- Aprende a programar: <https://pine.fm/LearnToProgram/>
- Otros libros: <https://github.com/EbookFoundation/free-programming-books/blob/main/books/free-programming-books-langs.md#ruby>

10.11 Ejercicios

Ejercicio 1 Escriba una función llamada `max_number` que reciba un arreglo de enteros positivos y devuelva el mayor.

Ejercicio 2 Escriba una función llamada `complete_decks_in` que recibe un arreglo de cadenas (que representan cartas) y retorna el número de mazos completos que se pueden armar con esas cartas.



RoR: Active Records

En el curso utilizaremos la implementación de Active Record en RoR para ayudarnos a guardar el estado de los objetos que creamos en una base de datos relacional. Active Record en RoR es un framework que nos permite definir de forma declarativa como un determinado objeto escrito en Ruby se guardara en una tabla en una base de datos relacional. Existen varios gestores de bases de datos relacionales (DBMS), por ejemplo, MySQL, PostgreSQL, o SQLite. Active Records permite trabajar con cualquiera de estos gestores de base datos.

El nombre de “Active Record” corresponde a un patron arquitectonico, originalmente descrito por Martin Fowler en el libro “Patterns of Enterprise Application Architecture”. El patron Active Record es una forma de modelar objetos que nos permitan interactuar con los datos almacenados en una base de datos. Es decir, estos objetos tienen la logica (métodos) que nos permiten entre otras operaciones crear, actualizar, buscar, editar, y eliminar datos en una base de datos. Active Record en RoR implementa este patron y da muchas facilidades para que los desarrollares puedan mapear los objetos con a una base de datos relacional.

11.1 Mapeo Objeto Relacional

El mapeo objeto relacional, comumente conocido como ORM, permite permitir enlazar los objetos dentro de una aplicación con tablas dentro de una base de datos relacional. Para utilizar Active Records no es necesario tener un conocimiento avanzado de bases de datos relacionales o de como realizar consultas en SQL. En este sección aprenderemos aspectos basicos de Active Records, así como de bases de datos, y consultas.

Mi primer Ejemplo

Asumimos que partimos con un proyecto ruby on rails creado. Para crear un proyecto rails solo debes ejecutar el siguiente comando en consola:

```
> rails new course
```

Este comando te creara varias carpetas y archivos utiles para el desarrollo de un proyecto web. Para entender mejor para que sirve cada carpeta y archivo recuerda que puedes leer el capitulo anterior que da una introducción a Ruby on Rails. Luego de crear el proyecto no olvides ingresar a la carpeta course recientemente creada:

```
> cd course
```

Para crear un modelo (una clase) en nuestro proyecto utilizaremos el siguiente comando en consola:

```
> rails generate model Student name rut
```

El comando anterior solicita generar un modelo que tiene dos atributos: name y rut. Si el comando funciona correctamente deberia salir el siguiente mensaje en consola:

```
invoke  active_record
create  db/migrate/20230208153647_create_students.rb
create  app/models/student.rb
invoke  test_unit
create  test/models/student_test.rb
create  test/fixtures/students.yml
```

El comando anterior crea la clase Student en el archivo app/models/student.rb como se ve a continuación:

```
class Student < ApplicationRecord
end
```

Ademas crea varios otros archivos que los analizaremos a lo largo de este capitulo. Primero enfoquemonos en al clase Student, que es una simple clase que hereda de la clase ApplicationRecord. Como sabemos en ruby no es necesario declarar los atributos que tendra la clase. Del mismo modo el comando anterior genera un script (db/migrate/20230208153647_create_students.rb) en ruby que permite crear una tabla en la base de datos.

```
class CreateStudents < ActiveRecord::Migration[7.0]
  def change
    create_table :students do |t|
      t.string :name
      t.string :rut
    end
  end
```

```

      t.timestamps
    end
  end
end

```

Para ejecutar este script uno debe ejecutar el siguiente comando en consola:

```
> rails db:migrate
```

Si el comando es ejecutado correctamente se mostrará el siguiente mensaje en consola:

```

== 20230208153647 CreateStudents: migrating =====
-- create_table(:students)
   -> 0.0010s
== 20230208153647 CreateStudents: migrated (0.0010s) =====

```

Como puedes ver el mensaje menciona que se creo una tabla llamada :students. Para poder ver exactamente que columnas tiene la tabla recién creada puedes abrir el archivo db/migrate/schema.rb. El mismo tiene la información completa de las tablas creadas en la base de datos, los nombres de las columnas entre alguna otra información como puedes ver a continuación.

```

ActiveRecord::Schema[7.0].define(version: 2023_02_08_153647) do
  create_table "students", force: :cascade do |t|
    t.string "name"
    t.string "rut"
    t.datetime "created_at", null: false
    t.datetime "updated_at", null: false
  end
end

```

Note que ruby aparte de las columnas name y rut creo dos columnas adicionales que es created_at y updated_at estas fueron creadas por la instrucción t.timestamps en el script. Hasta el momento hemos creado una clase Student en ruby y una tabla students en la base de datos, ahora lo unico que nos falta es aprender a mapear la informacion de los objetos de la clase Student a la base de datos. Gracias a que la clase Student hereda de ApplicationRecord tiene varios metodos de objeto y de clase utiles para realizar el mapeo y ademas hacer consultas a la base de datos.

RoR: Mi primera API

En este capítulo veremos con implementar un API en Ruby on Rails. API de sus siglas en inglés significa interfaz de programación de aplicaciones. Se trata de un conjunto de mecanismos que permiten la comunicación de dos aplicaciones que típicamente están en dos computadores diferentes. En programación web, normalmente se comunican dos aplicaciones: el navegador web (Chrome, Safari, Firefox, etc) y un servidor en la Nube. Podemos decir en palabras sencillas que el servidor es un programa ejecutándose en la nube que está a la espera que los clientes (otras aplicaciones) se comuniquen con él y le hagan solicitudes para recuperar o guardar información.

En el curso implementaremos un servidor en Ruby on Rails. Para comunicarse con él se mandará información en formato JSON mediante el protocolo HTTP. Por lo que este capítulo, primero describe el protocolo HTTP, luego da una pequeña introducción al formato JSON, finalmente, implementa un pequeño servidor que ofrece una interfaz para crear, recuperar y borrar estudiantes de base de datos.

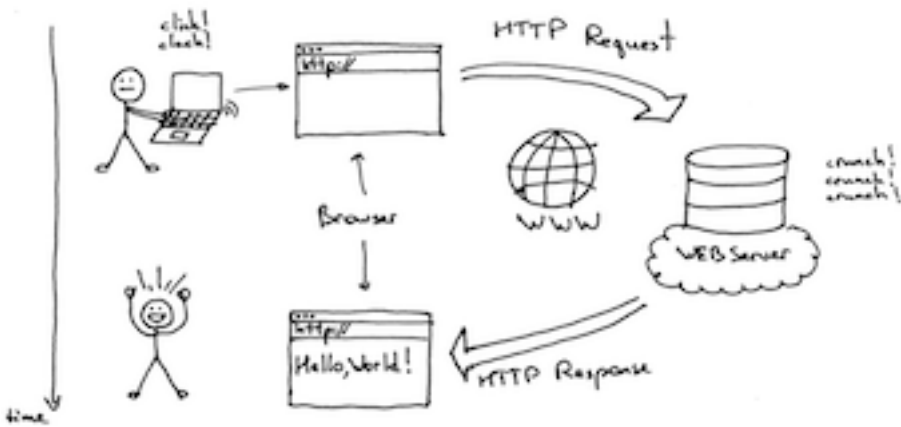
12.1 Protocolo de Transferencia de Hipertexto

HTTP significa Protocolo de Transferencia de Hipertexto, este protocolo es considerado una de las bases de la *web* a nivel mundial. Para entender HTTP, primero debemos entender que es un protocolo. Un protocolo es un conjunto de reglas, en particular, los protocolos de red son estándares y políticas formales que definen políticas de intercambio de información entre dos computadores.

Supongamos que enviamos el texto siguiente información desde un computador a otro:

```
01101000011011110110110001100001
```

El computador que recibe el mensaje tiene problemas al interpretar la información ya que solo recibe un conjunto de bits. Este conjunto de bits podría un numero, una cadena, dos numeros, un password, existen un monton de posibilidades. Por este motivo para que el computador que recibe el mensaje pueda saber de que trata es necesario también enviar meta-data (datos sobre los datos). Sin embargo, estos meta-datos también tendrían el mismo problema ya que solo estaríamos enviando una cade de bits mas grande. Por esta razón, es necesario definir protocolos que regulen como estructurar los datos y meta-datos que se envían entre dos computadores o procesos. En particular, el protocolo HTTP propone separar el envío de información en dos: HTTP Request y HTTP Response.



HTTP Request.. La cual establece que si alguien requiere solicitar información debe enviar los siguientes datos. La primera línea debe contener 3 elementos:

- *HTTP method:* que en la practica es un verbo o nombre (en formato string) que describe la acción que se va ejecutar. Los mas utilizados son "GET" que indica que se el que envia la solicitud esta esperando una respuesta y "POST" que indica que el que envia la información esta enviando datos que espera que se "empujen" en el servidor.
- *Request target:* que usualmente es una URL donde se especifica cual es el objetivo a ejecutar.
- *HTTP version.* Donde indica la version del protocolo HTTP que esta utilizando para enviar la petición. Las versiones de protocolo HTTP tiene ligeras variaciones por lo cual es importante informar cual se esta utilizando. Normalmente, se utiliza la version 1.1.

En las siguientes lineas se envían todos los "headers" necesarios. HTTP tiene una lista de headers posibles a enviar, uno podría ver un header como una

variable que fue inicializada con un valor por defecto. Si uno quieren utilizar otro valor distinto al por defecto, este se debe especificar al enviar el HTTP Request. Los headers que utilizaremos durante el curso casi siempre seran los mismos, por lo que no es necesario preocuparse mucho sobre cuantos tipos de headers existen, entre otras cosas.

Finalmente, el *body* (cuerpo) del mensaje va al final, debe existir un enter dentre los headers y el body.

```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.tutorialspoint.com
Content-Type: application/x-www-form-urlencoded
Content-Length: length
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```

```
licenseID=string&content=string&/paramsXML=string
```

HTTP Response. Del mismo modo la respuesta debe tener una estructura parecida.

- Una linea donde se especifica el estado de la solicitud. Existe un conjunto de estados predefinidos en HTTP entre los mas comunes es el estado 200 OK, que significa que la solicitud fue ejecutada correctamente;
- la respuesta tambien contienen varios headers;
- una linea en blanco;
- finalmente, en ocasioens tambien cuenta con un body.

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: text/html
Connection: Closed
```

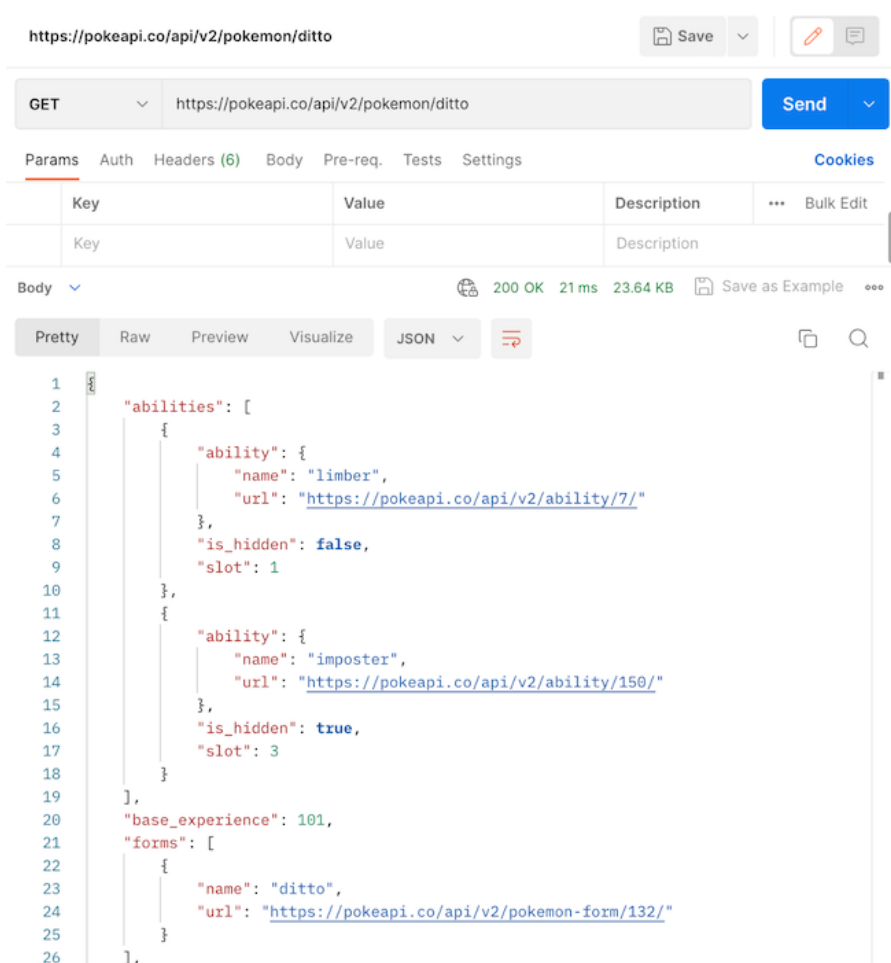
El mensaje anterior indica que el servidor realizo la petición de forma correcta.

12.2 Postman

En la actualizada existen varias API que utilizan el protocolo HTTP, por ejemplo, pokeapi. Poke API es una base de datos que tiene información de pokemon. Existen varias herramientas o programas que permite enviar HTTP

Requests y recibir HTTP Responses. En el curso utilizaremos Postman, un programa que proporciona una UI para armar una solicitud HTTP Request, enviarla y recibir respuestas HTTP. En esta sección utilizaremos Postman para solicitar información de pokemones a PokeAPI.

Postman es simple de utilizar, se debe utilizar las opciones del menu y buscar la opción de “crear HTTP Request”. Existen varias formas, una de ellas es ir a “File/New...” y luego presionar “HTTP Request”. Posteriormente, se debe llenar los datos de la HTTP Request en la UI proporcionada en postman. En nuestro ejemplo, solo necesitamos especificar que la operación es de tipo “GET”, poner la dirección URL la cual esta compuesta por el dominio de la aplicación y el http target que en este caso es “pokemon/ditto”. No se agrego ningun header.



Finalmente, se debe presionar el boton “Send” (en azul), en la parte inferior de postman se puede ver la respuesta (reponse) HTTP. Sepuede ver que el

estado de la respuesta es 200 OK. En la parte central se ve el “body” de la respuesta que es una cadena que contiene datos en formato JSON:

```
{
  "abilities": [
    {
      "ability": {
        "name": "limber",
        "url": "https://pokeapi.co/api/v2/ability/7/"
      },
      "is_hidden": false,
      "slot": 1
    },
    {
      "ability": {
        "name": "imposter",
        "url": "https://pokeapi.co/api/v2/ability/150/"
      },
      "is_hidden": true,
      "slot": 3
    }
  ],
  "base_experience": 101,
  "forms": [
    {
      "name": "ditto",
      "url": "https://pokeapi.co/api/v2/pokemon-form/132/"
    }
  ],
  ...
}
```

En esta sección se vio como enviar HTTP-Request y recibir HTTP-Response al API de pokemones. En este ejemplo, se vio solo consumir uno de los servicios de pokeAPI, pokeAPI tiene varios servicios que permiten: listar los pokemones existentes, obtener detalles de un pokemon en particular, entre otros servicios que se pueden acceder a través de la web (web services).

12.3 Hello World API

En la sección anterior vimos como consumir un servicio web, en esta sección aprenderemos a crear un servicio web sencillo. Como primer ejemplo básico crearemos un servicio web en Ruby on Rails que devuelve “Hola Mundo” cada vez que un cliente haga una petición. Antes de empezar a realizar las instrucciones de este tutorial usted debe tener instalado Ruby y Ruby on Rails. Para verificar su instalación ejecute el siguiente comando en consola

```
>ruby --version
ruby 3.1.2p20
>rails --version
Rails 7.0.4.2
```

Si usted esta trabajando con versiones de rails o ruby superiores no deberia tener ningún problema.

Paso 1. Crear un proyecto ruby en modo API:

```
rails new hello --api
```

La linea anterior debe crear un folder llamado “hello” y dentro de el varias carpetas y archivos. A medida que avancemos en el curso se ira detallando para que sirven cada uno de los archivos y folders generados. Una vez creado el proyecto debe ingresar dentro de la carpeta recién creada.

```
cd hello
```

Ya dentro de la carpeta se debe crear un controlador, en este caso nombraremos a nuestro controlador “Hello”. Un controlador en Rails es una clase cuyos objetos recibirán solicitudes (request) a través de sus métodos. Para crear el controlador debemos ejecutar el siguiente comando:

```
rails generate controller Hello
```

El comando anterior debe generarnos la siguiente clase en el folder “app/controllers/hello_controller.rb”.

```
class HelloController < ApplicationController
end
```

Inicialmente el controlador no tiene ningún comportamiento asociado (ningún método), nosotros implementaremos un método llamado “sayHi” que devolverá la cadena ‘Hello World’ al que realice una solicitud a través de este método.

```
class HelloController < ApplicationController
  def sayHi
    render json: 'Hello World!!'
  end
end
```

Las solicitudes en RubyOnRails se realizan mediante el protocolo HTTP. Las solicitudes HTTP viene acompañadas de “target” en formato “string”. El target indica que operación queremos ejecutar en el servidor. En nuestro ejemplo queremos que nuestra operación se pueda ejecutar utilizando el siguiente target: ‘http://localhost:3000/hi’. Para lo cual es necesario indicar a

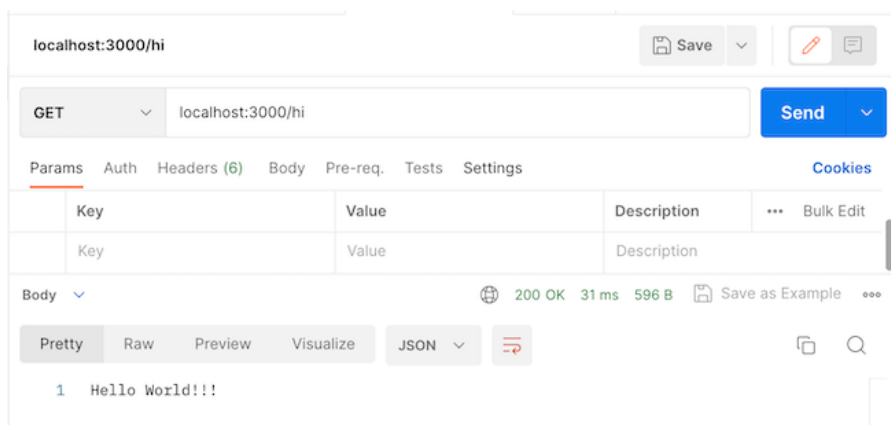
ruby que el target `/hi` ejecutara el método “sayHi” de nuestro controllador llamado ‘Hello’ para mapear el target con el metodo a ejecutar se debe editar el archivo “config/routes.rb” como se ve a continuación:

```
Rails.application.routes.draw do
  get 'hi', to: 'hello#sayHi'
end
```

Con lo anterior tenemos listo nuestro primer API, para probar su funcionamiento debemos ejecutar el siguiente comando en consola:

```
rails server
```

Para probar el API utilizaremos Postman, igual que en el ejemplo anterior solo necesario especificar el target y enviar la solicitud con el boton “Send”. Como se puede ver en la parte inferior el servidor respondio con un “Hello World!”



12.4 API – Estudiantes

En esta sección crearemos un API un poco mas complejo. La idea es tener un servidor que permita listar estudiantes, y buscar un estudiante mediante un ID.

Creando el Modelo.

Primero, crearemos un clase que nos permita modelar (Model) los datos y comporamentiento de un estudiante (en particular los datos). Para crear esta clase utilizaremos un comando en rails.

```
rails generate model Student name score
```

Si todo funciona correctamente Rails creara los siguientes archivos:

```

invoke  active_record
create  db/migrate/20230312193956_create_students.rb
create  app/models/student.rb
invoke  test_unit
create  test/models/student_test.rb
create  test/fixtures/students.yml

```

La ventaja de crear la clase `Student` utilizando el comando anterior es que Rails además de crear la clase `Student`, generará archivos que me ayudarán a guardar estudiantes en la base de datos y hacer pruebas de unidad automáticas. Por ejemplo, en este momento ya es posible crear la base de datos para almacenar estudiantes utilizando el siguiente comando:

```
rails db:migrate
```

Si el comando funciona correctamente el siguiente mensaje debería mostrarse:

```

== 20230312193956 CreateStudents: migrating =====
-- create_table(:students)
   -> 0.0007s
== 20230312193956 CreateStudents: migrated (0.0007s) =====

```

Note que la clase creada por Rails hereda de la clase `ApplicationRecord`

```

class Student < ApplicationRecord
end

```

Gracias a que hereda de esta clase (entre otras cosas) ya es posible crear objetos estudiantes, guardarlos y recuperarlos de la base de datos. Para probar con expresiones simples utilizaremos la consola de rails.

```
> rails console
```

Si se ejecuta correctamente, deberían observar el texto `irb:...>` en la consola. A continuación veremos algunos ejemplos, las siguientes expresiones permitirán crear un objeto estudiante y guardarlo en la base de datos.

```

student = Student.new(name:"Juan Pablo", score:100)
result = student.save

```

En el ejemplo anterior, la variable `result` será falso si es que Rails no pudo guardar al estudiante, por ejemplo, porque tal vez ya existe otro estudiante con el mismo ID en la base de datos o porque al estudiante le faltan algunos campos requeridos. A continuación se muestra otros métodos útiles que tiene la clase `ApplicationRecord` para (a) recuperar a un estudiante dado un ID, o para recuperar todos los estudiantes de la base de datos en una lista.

```

student = Student.find(1)
all_students = Student.all

```

La ultima instrucción devuelve una lista de objetos como se ve a continuación:

```
[#<Student:0x000000010a1c7478
  id: 1,
  name: "Juan Pablo",
  score: "100",
  created_at: Sun, 12 Mar 2023 19:53:07.784504000 UTC +00:00,
  updated_at: Sun, 12 Mar 2023 19:53:07.784504000 UTC +00:00>]
```

Note que cada estudiante dentro de la base de datos, aparte de name y score, tiene un ID, una fecha de creación y una fecha de actualización.

Creando un Controlador

Ahora procederemos a crear un controlador que tenga métodos (operaciones) que permitan manipular a los estudiantes dentro de la base de datos. En este ejemplo, haremos dos operaciones: (a) buscar estudiante por ID, y (b) listar todos los estudiantes. Para crear el controlador, al igual que el ejemplo anterior, tenemos que ejecutar la siguiente instrucción:

```
rails generate controller Student
```

Listando Estudiantes

Posteriormente, agregaremos una operación al controlador llamada “index” que sera encargada de listar a todos los estudiantes:

```
class StudentController < ApplicationController
  def index
    @all_students = Student.all
    render json:@all_students
  end
end
```

Finalmente, mapeamos el “target” al método del controlador respectivo.

```
Rails.application.routes.draw do
  get 'index', to:'student#index'
end
```

En el código anterior estamos mapeando la ruta “localhost:3000/index” al controlador ‘student’ (StudentController) y al método ‘index’. Para probar recuerde ejecutar:

```
rails server
```

Si accedemos al servicio web utilizando postman, la salida deberia ser la siguiente:

```
[
  {
    "id": 1,
    "name": "Juan Pablo",
    "score": "100",
    "created_at": "2023-03-12T19:53:07.784Z",
    "updated_at": "2023-03-12T19:53:07.784Z"
  }
]
```

Mostrando un estudiante

En este ejemplo, veremos como se puede enviar la información al servidor a través de la URL. Por ejemplo, queremos que cuando el cliente escriba “localhost:3000/estudiante/1”, muestre la información del estudiante con el ID igual a 1.

Primero debemos agregar la operación en el controlador:

```
class StudentController < ApplicationController
  # GET /student/:id
  def show
    @student = Student.find(params[:id])
    render json: @student
  end
end
```

Posteriormente, mapeamos la ruta a la operación:

```
Rails.application.routes.draw do
  get '/index', to: 'student#index'
  get '/student/:id', to: 'student#show'
end
```

Note que a diferencia del anterior ejemplo, esta vez mandamos un argumento a la operación como parte de la URL. En la ruta el argumento se define como “:id”, y para recuperar este argumento dentro del controlador utilizamos la siguiente expresión “params[:id]”.

Creando un estudiante

Para enviar información mas compleja a través de un HTTP Request utilizaremos el `body` y el formato `JSON`. Por ejemplo, podemos enviar la información de un nuevo estudiante en formato `JSON` de la siguiente forma:

```
{
  "student": {
    "name": "Pedro",
```

```

    "score": 51
  }
}

```

Note que en el caso de crear un estudiante, nosotros no queremos recuperar información (GET) sino introducir información en el servidor. Por lo que en este caso el HTTP Request debería ser de tipo POST. A continuación podrá encontrar la operación que permite registrar un nuevo estudiante.

```

class StudentController < ApplicationController
  # POST /student
  def create
    @student = Student.new(student_params)
    if @student.save
      render :json => @student
    else
      render :json => @student.errors, :status => :unprocessable_entity
    end
  end

  def student_params
    params.require(:student).permit(:name, :score)
  end
end

```

El método `student_params` permite recuperar la información del body del request y construir un hash con los datos del estudiante. Este hash es enviado al momento de crear al estudiante. En caso que el estudiante logre guardarse en la base de datos el método `save` devolverá “verdad”, “falso” en caso contrario. En caso que no sea posible guardar al estudiante devolveremos un HTTP Response, con status “unprocessable_entity” y enviaremos los errors que guarder Rails en la variable “errors”.

El ultimo paso es mapear la ruta del URL con la operación:

```

Rails.application.routes.draw do
  get '/index', to: 'student#index'
  get '/student/:id', to: 'student#show'
  post '/student', to: 'student#create'
end

```

Usted ya puede probar esta operación utilizando Postman, no olvide enviar el request utilizando el método POST y copiando el JSON del estudiante nuevo en el body. Si todo sale correctamente el HTTP response debería tener el siguiente body:

```

{
  "id": 2,

```

```
"name": "Pedro",  
"score": "51",  
"created_at": "2023-03-12T23:39:13.687Z",  
"updated_at": "2023-03-12T23:39:13.687Z"  
}
```

12.5 Ejercicios

Agregue la operación `filter` al controlador `Student`, el método `filter` debe recibir un numero como argumento, y retornar en formato JSON los datos de los estudiantes que tienen una nota igual al argumento del request (`params[:score]`). Crea la request en Postman para demostrar que el método `filter` funciona correctamente.